

The RadianceEngine Path Tracer

Physics, mathematics and implementation of a physically based path tracer
for surfaces and homogeneous participating media



The showcase scene after 60 seconds of progressive accumulation at 1280×720 . Left to right: frosted glass box under a chrome sphere, toy blocks of glossy to rough plastic, a blue absorbing glass sphere with its caustic, a jade dragon (scattering medium behind a rough dielectric boundary) on polished stone, a clear-coated car-paint sphere, a cyan emitter, a smoke volume with an orange emitter inside, a gold box and a velvet (sheen) bunny, all on a lacquered checkerboard floor; the right wall is a mirror.

RadianceEngine documentation · 26 September 2026

Describes `res/Shaders/PathTracer/` (`common.glsl`, `scene.glsl`, `bsdf.glsl`, `raygen.rgen`, `rayclosesthit.rchit`, `raymiss.rmiss`) and `Application/src/PathTracer/Scene.cpp` as of this date. All code listings are extracted automatically from those files.

About this document

This document explains how the RadianceEngine path tracer turns a scene description into an image, and why every step is done the way it is. It starts from the physics of light — what is measured, what equation it obeys — develops the Monte Carlo methods that solve that equation, and then walks through the complete scattering model of surfaces (the BSDF, lobe by lobe) and of participating media. Every formula is connected to the GLSL function that implements it, and the listings are the actual shader code.

The intended reader knows vector algebra, basic calculus and the notion of a probability density, and wants to understand a physically based renderer well enough to modify or extend it. No prior knowledge of rendering theory is assumed. Readers who already know radiometry, the rendering equation and Monte Carlo integration can skip Part I and use it as a reference for notation.

Structure

Part	Content
I · Foundations	What a pixel measures, radiometry, the rendering equation, Monte Carlo integration, importance sampling, multiple importance sampling, Russian roulette.
II · Camera and geometry	Camera rays and the thin lens, progressive accumulation, random numbers, ray-scene intersection and normals, robust ray offsets, the local shading frame.
III · The BSDF	Fresnel equations, microfacet theory (GGX distribution, Smith masking-shadowing, reflection and refraction BSDFs, visible-normal sampling), the complete material model with all lobes, BSDF sampling, and the energy-conservation tests.
IV · Light transport	Next event estimation, multiple importance sampling between lights and BSDFs, Russian roulette, and the full path loop.
V · Participating media	Radiative transfer, phase functions, free-flight sampling, spectral multiple importance sampling for coloured media, lighting in media, medium boundaries and the medium stack.
VI · Output and practice	Accumulation, tone mapping and colour encoding, numerical robustness, limitations and extensions.
Appendices	Formula summary, parameter reference, code map, detailed derivations, references.

Conventions

Coloured boxes mark four kinds of material:

KEY IDEA

The essential result of a section, in a form you can remember.

IN THE CODE

Where and how the result is implemented, including simplifications made on the GPU.

DERIVATION

A step-by-step argument that can be skipped on first reading.

CAVEAT

An approximation, limitation or pitfall.

Notation

Symbol	Meaning
ω_o	unit direction toward the viewer or the previous path vertex ("outgoing"); points away from the surface; in code <code>wo = -rayDirection</code>
ω_i	unit direction toward the light or the next path vertex ("incident"); points away from the surface
$\mathbf{n}, \mathbf{n}_g$	shading normal (interpolated), geometric normal (from the triangle)
$\theta, \mu = \cos \theta $	angle from the normal, and the absolute value of its cosine
$\mathbf{m}, \mathbf{h}$	microfacet normal; half vector between ω_o and ω_i
r, α	material roughness $r \in [0, 1]$ and GGX width $\alpha = \max(r^2, 10^{-3})$
η	relative index of refraction, inside over outside (material <code>IOR</code>)
F, F_0	Fresnel reflectance, and its value at normal incidence
D, G_1, G_2, Λ	microfacet normal distribution, masking, masking-shadowing, Smith auxiliary function
$f(\omega_o, \omega_i)$	bidirectional scattering distribution function (BSDF), unit sr^{-1}
L, L_e	radiance and emitted radiance (RGB triplets)
$p(\cdot), p_A(\cdot)$	probability density per unit solid angle; per unit area
β	path throughput (accumulated $f \cos \theta / p$ factors)
$\sigma_a, \sigma_s, \sigma_t$	absorption, scattering and extinction coefficients (per unit length), $\sigma_t = \sigma_a + \sigma_s$
$\varpi = \sigma_s / \sigma_t$	single-scattering albedo of a medium
g	anisotropy (mean cosine) of the Henyey–Greenstein phase function
$T(t)$	transmittance over a distance t
ξ, u	uniformly distributed random numbers in $[0, 1)$
$Y(\cdot)$	luminance, $0.2126R + 0.7152G + 0.0722B$

Contents

About this document

Part I · Foundations

1. What the renderer computes
 - 1.1 A pixel is an integral
 - 1.2 The algorithm at a glance
 - 1.3 Where things happen
 - 1.4 Units and colour
2. Radiometry
 - 2.1 Solid angle
 - 2.2 Flux, irradiance and radiance
 - 2.3 Radiance is constant along rays
 - 2.4 Lambertian emitters and the power of a light
 - 2.5 RGB and luminance
3. The rendering equation
 - 3.1 The BSDF
 - 3.2 The equation
 - 3.3 What makes a BSDF physically plausible
 - 3.4 Light paths
 - 3.5 The area form and direct lighting
4. Monte Carlo integration
 - 4.1 The estimator
 - 4.2 Importance sampling
 - 4.3 Generating samples
 - 4.4 Densities per solid angle and per area
 - 4.5 Multiple importance sampling
 - 4.6 One-sample MIS: sampling from a mixture
 - 4.7 Russian roulette
 - 4.8 The path tracing estimator

Part II · Camera and geometry

5. Camera, pixels and random numbers
 - 5.1 The pinhole camera
 - 5.2 Pixel jitter: antialiasing for free
 - 5.3 The thin lens
 - 5.4 Progressive accumulation
 - 5.5 Random numbers
 - 5.6 The ray generation entry point
6. Geometry: intersections and normals
 - 6.1 Acceleration structures and instancing
 - 6.2 Reconstructing the hit
 - 6.3 Why normals transform with the inverse transpose
 - 6.4 Geometric normals, shading normals and front faces
 - 6.5 Starting new rays without hitting the old surface
7. The local shading frame
 - 7.1 An orthonormal basis around the normal
 - 7.2 Directions and angles
 - 7.3 Reflection and refraction directions

Part III · The BSDF

8. Fresnel reflectance
 - 8.1 Where the Fresnel equations come from
 - 8.2 The dielectric Fresnel equations
 - 8.3 Total internal reflection
 - 8.4 Reflectance at normal incidence
 - 8.5 Schlick's approximation and metals
 - 8.6 Fresnel reflectance of diffuse light
9. Microfacet theory
 - 9.1 The microsurface model
 - 9.2 The GGX distribution
 - 9.3 Masking and shadowing: the Smith model

- 9.4 The microfacet reflection BRDF
- 9.5 The microfacet transmission BTDF
- 9.6 The distribution of visible normals
- 9.7 Sampling visible normals
- 9.8 Densities and sample weights
10. The material model
 - 10.1 Parameters
 - 10.2 Structure: lobes and weights
 - 10.3 The clear coat
 - 10.4 The metal lobe and multiple-scattering compensation
 - 10.5 The opaque dielectric: specular reflection over a diffuse substrate
 - 10.6 Sheen
 - 10.7 The glass lobe
 - 10.8 Two-sided surfaces and light leaks
 - 10.9 Near-specular materials and light sampling
 - 10.10 A worked example: the car paint
11. Sampling the BSDF
 - 11.1 Choosing a lobe
 - 11.2 Sampling each lobe
 - 11.3 Why the whole BSDF is evaluated after sampling
 - 11.4 The code
12. Verifying energy conservation
 - 12.1 The white furnace test
 - 12.2 Setup and results
 - 12.3 A CPU reference for the sampling code
 - 12.4 What still loses energy

Part IV · Light transport

13. Next event estimation
 - 13.1 The light list
 - 13.2 Choosing a point on a light
 - 13.3 The estimator
 - 13.4 What counts as a light
14. Combining light and BSDF sampling
 - 14.1 Two strategies for the same light
 - 14.2 Power heuristic weights
 - 14.3 Implementation
 - 14.4 When there is only one strategy
 - 14.5 The environment
15. Russian roulette
16. The complete path loop
 - 16.1 Overview
 - 16.2 Step by step
 - 16.3 The code
 - 16.4 The environment

Part V · Participating media

17. Radiative transfer
 - 17.1 Absorption, scattering and extinction
 - 17.2 The radiative transfer equation
 - 17.3 The integral form along a ray
18. Phase functions
 - 18.1 Definition
 - 18.2 The Henyey–Greenstein phase function
 - 18.3 Sampling Henyey–Greenstein by inversion
19. Free-flight distance sampling
 - 19.1 Sampling where the next interaction happens
 - 19.2 Pure absorbers
 - 19.3 Rays that leave the scene inside a medium

20. Coloured media: hero channels and spectral MIS

- 20.1 The problem
- 20.2 The channel as a sampling technique
- 20.3 Incremental implementation
- 20.4 Why per-event averaging is not enough
- 20.5 Numerical safety

21. Direct lighting in media**22. Medium boundaries and the medium stack**

- 22.1 Attaching media to geometry
- 22.2 Two kinds of boundaries
- 22.3 The medium stack
- 22.4 Rules and limitations
- 22.5 The global medium

23. Worked examples**Part VI · Output and practice****24. From radiance to pixels**

- 24.1 Exposure and tone mapping

24.2 sRGB encoding**24.3 Linear lighting and sRGB textures****25. Numerical robustness****26. Limitations and extensions**

- 26.1 Light transport
- 26.2 Materials
- 26.3 Media
- 26.4 Towards heterogeneous media

Appendices · Reference material**A. Formula summary****B. Parameter reference****C. Code map****D. Derivations**

- D.1 The half-vector Jacobian for reflection
- D.2 A general half-vector Jacobian, and refraction
- D.3 Free-flight sampling is unbiased

E. References

PART I

Foundations

Before looking at shader code we need three ingredients: the physical quantity the renderer computes (radiance), the equation that governs it (the rendering equation), and the numerical method used to solve that equation (Monte Carlo integration). The notation introduced here is used throughout the rest of the document.

1. What the renderer computes

1.1 A pixel is an integral

A pixel value is not the colour of a single ray. A sensor element integrates all the light that arrives over its footprint on the image plane — and, for a camera with a lens, over the whole lens aperture. For a pinhole camera and a box-shaped pixel, the value of pixel j is

$$I_j = \frac{1}{|P_j|} \int_{P_j} L_i(\mathbf{c}, \omega(\mathbf{u})) \, d\mathbf{u}, \quad (1.1)$$

where P_j is the pixel's square on the image plane, $\omega(\mathbf{u})$ the direction of the camera ray through the image point $\mathbf{u}$, and $L_i(\mathbf{c}, \omega)$ the radiance arriving at the camera position $\mathbf{c}$ from direction ω . That radiance is determined by the light transport in the scene (Section 3) and is itself an integral — over all the light paths that connect the camera to a light source. The renderer never evaluates these integrals exactly. It estimates them with random samples (Section 4): for every pixel and frame it picks a random point in the pixel, follows one random light path through the scene, and computes that path's contribution. The average of many such contributions converges to I_j .

1.2 The algorithm at a glance

The renderer is a *unidirectional path tracer*: paths start at the camera and are extended one scattering event at a time. At every surface or medium interaction it

1. connects the current vertex directly to a randomly chosen point on a light source — *next event estimation* (Section 13);
2. samples a continuation direction from the local scattering function — the BSDF at surfaces (Part III), the phase function inside media (Section 18);
3. combines both ways of finding light with *multiple importance sampling*, so that each is used where it works best (Section 14);
4. randomly terminates paths whose remaining contribution is small — *Russian roulette* (Section 15).

Participating media are homogeneous: constant coefficients inside a closed mesh. Distances between medium interactions are sampled analytically (Section 19), with spectral multiple importance sampling for coloured media (Section 20). Each frame adds `SamplesPerPixel` new paths per pixel to a running sum, so the image converges progressively while the camera stands still.

1.3 Where things happen

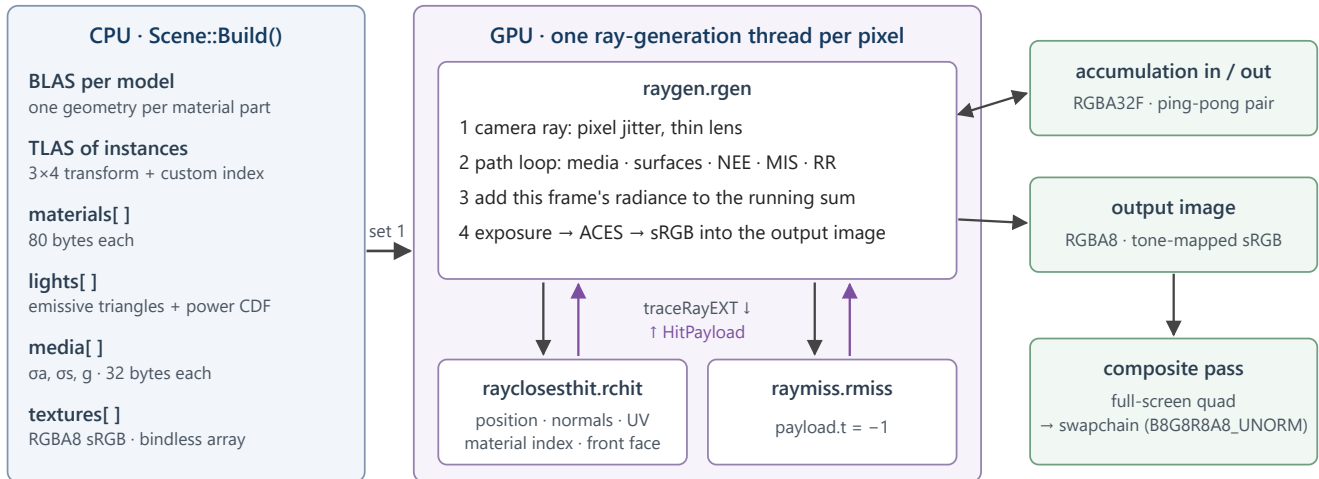


Figure 1. Data flow. The CPU builds acceleration structures and uploads the scene tables once. Each frame, one ray-generation thread per pixel runs the complete light path; the closest-hit shader only reconstructs intersections and does no shading. The running HDR sum lives in two RGBA32F images that swap roles every frame; the tone-mapped result is drawn to the swapchain by a composite pass.

The whole path is computed in the ray generation shader, one GPU thread per pixel. The closest-hit shader performs no shading; it reconstructs the intersection — position, normals, texture coordinates, material index, and which side of the triangle was hit — and returns it in the ray payload. This "megakernel" structure keeps the ray tracing pipeline non-recursive. The ray generation shader calls `traceRayEXT` once per path segment and once per shadow ray, and all path state lives in its local variables.

File	Responsibility
<code>common.glsl</code>	constants, the hit payload, random numbers, orthonormal frames, disk/hemisphere/triangle sampling, the power heuristic, robust ray offsets
<code>scene.glsl</code>	GPU layouts of meshes, materials, media and light triangles; descriptor set 1; the procedural sky
<code>bsdf.glsl</code>	Fresnel equations, the GGX microfacet model, and the material BSDF (<code>CreateBsdf</code> , <code>EvalBsdf</code> , <code>SampleBsdf</code>)
<code>raygen.rgen</code>	camera, path loop, next event estimation, media and medium stack, accumulation, tone mapping
<code>rayclosesthit.rchit</code>	intersection reconstruction (barycentric interpolation, normal transformation)
<code>raymiss.rmiss</code>	marks a miss with <code>payload.t = -1</code>
<code>Scene.cpp</code>	builds BLAS/TLAS, packs materials and media, builds the emissive-triangle list with its power CDF

Set	Binding	Resource	Updated
0	0	top-level acceleration structure	push descriptors, every frame
0	1, 2	accumulation image in / out (<code>rgba32f</code>)	
0	3	output image (<code>rgba8</code>)	
0	4	<code>GlobalData</code> uniform buffer (camera, frame index, bounces, exposure, samples)	
1	0	<code>meshPrimitives[]</code> : vertex/index buffer addresses and material per instanced part	descriptor set, on scene build
1	1	<code>materials[]</code>	
1	2	light count, total power, <code>lights[]</code>	
1	3	global medium index, medium count, <code>media[]</code>	
1	4	<code>textures[]</code> (variable-count bindless array, must be last)	

1.4 Units and colour

Lengths are in arbitrary scene units. The showcase room is 2.4 units wide, so it is natural to read one unit as one metre. Medium coefficients σ are measured per scene unit, so their visual effect depends on the size of the objects they fill (Section 23). Radiance is an RGB triplet in linear sRGB (Rec. 709) primaries, and each channel is treated as an

independent spectral sample; materials, lights and media are specified in RGB as well. The renderer therefore does not model dispersion or any wavelength dependence finer than three bands.

2. Radiometry

Radiometry is the measurement of light as energy. It tells us which quantity a path tracer transports and how that quantity behaves along rays and at surfaces.

2.1 Solid angle

The solid angle that an object subtends as seen from a point is the area of its projection onto the unit sphere around that point, measured in steradians (sr). The full sphere has 4π sr and a hemisphere 2π sr. In spherical coordinates with polar angle θ , measured from a chosen axis (usually the surface normal), and azimuth ϕ , a differential set of directions has

$$d\omega = \sin \theta d\theta d\phi. \quad (2.1)$$

A small surface patch of area dA at distance r , whose normal makes the angle θ' with the line of sight, subtends $d\omega = dA |\cos \theta'| / r^2$. This conversion between solid angle and area reappears whenever we switch from sampling directions to sampling points on lights.

2.2 Flux, irradiance and radiance

- **Radiant flux** Φ [W]: energy per unit time.
- **Irradiance** $E = d\Phi/dA$ [W m^{-2}]: flux arriving per unit area of a surface.
- **Radiance** $L = \frac{d^2\Phi}{dA \cos \theta d\omega}$ [$\text{W m}^{-2} \text{sr}^{-1}$]: flux per unit area *perpendicular to the direction of travel* and per unit solid angle.

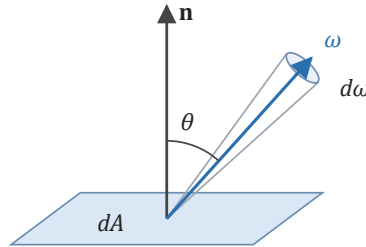


Figure 2. Radiance counts the flux that leaves (or arrives at) the patch dA into the small cone $d\omega$ around direction ω , per unit *projected* area $dA \cos \theta$ and per unit solid angle.

The factor $\cos \theta$ in the definition of radiance explains many formulas that follow. A beam of cross-section $dA_{\perp}$ hitting a surface at angle θ spreads over the larger area $dA = dA_{\perp} / \cos \theta$, so its irradiance is reduced by $\cos \theta$. The irradiance produced by radiance arriving from all directions of the hemisphere $\mathcal{H}^2$ above a surface is therefore

$$E(\mathbf{x}) = \int_{\mathcal{H}^2} L_i(\mathbf{x}, \omega) \cos \theta d\omega. \quad (2.2)$$

The product $\cos \theta d\omega$ is called the *projected solid angle*. Over the hemisphere it integrates to $\int_0^{2\pi} \int_0^{\pi/2} \cos \theta \sin \theta d\theta d\phi = \pi$, a number that appears in every Lambertian formula.

2.3 Radiance is constant along rays

Take two small patches dA_1 and dA_2 at distance r that see each other at the angles θ_1 and θ_2 from their normals. The flux leaving patch 1 toward patch 2 is $d^2\Phi = L_1 dA_1 \cos \theta_1 d\omega_1$, with $d\omega_1 = dA_2 \cos \theta_2 / r^2$:

$$d^2\Phi = L_1 \frac{dA_1 \cos \theta_1 dA_2 \cos \theta_2}{r^2}.$$

The expression is symmetric in the two patches. The same flux arrives at patch 2, so the radiance measured there looking back, $L_2 = d^2\Phi / (dA_2 \cos \theta_2 d\omega_2)$ with $d\omega_2 = dA_1 \cos \theta_1 / r^2$, equals L_1 . **In vacuum, radiance does not change along a ray.** This is why a path tracer can transport radiance by casting rays: the radiance leaving a surface point toward the camera is exactly the radiance the camera receives from it.

Two exceptions matter here. Participating media attenuate and add radiance along the ray (Part V). At a refractive interface the conserved quantity is L/η^2 rather than L : a beam entering a medium of higher refractive index is bent toward the normal, its solid angle shrinks by η_1^2/η_2^2 , and its radiance grows by the inverse factor (Section 9.5).

2.4 Lambertian emitters and the power of a light

A surface that emits the same radiance L_e into every direction of its front hemisphere is called Lambertian. Its exitance (emitted flux per unit area) follows from (2.2): $M = \int_{\mathcal{H}^2} L_e \cos \theta d\omega = \pi L_e$, so an emissive triangle of area A emits the flux $\Phi = \pi L_e A$. All emission in this renderer is Lambertian and one-sided: only the front face — the side the geometric normal points to — emits. Light selection (Section 13) picks triangles with probability proportional to $Y(L_e) A$; the common factor π cancels in the normalisation.

2.5 RGB and luminance

When one number is needed for an RGB quantity — how bright a light is, for example — the renderer uses luminance with the Rec. 709 weights (`Luminance` in `common.gls1`):

$$Y(R, G, B) = 0.2126 R + 0.7152 G + 0.0722 B. \quad (2.3)$$

Russian roulette uses the largest channel instead (`MaxComponent`), so that a path carrying only blue light is not terminated for having low luminance.

3. The rendering equation

3.1 The BSDF

How a surface scatters light is described by its *bidirectional scattering distribution function* $f(\mathbf{x}, \omega_o, \omega_i)$. It relates the radiance scattered toward ω_o to the irradiance arriving from a small cone around ω_i :

$$f(\mathbf{x}, \omega_o, \omega_i) = \frac{dL_o(\mathbf{x}, \omega_o)}{L_i(\mathbf{x}, \omega_i) |\cos \theta_i| d\omega_i} \quad [\text{sr}^{-1}]. \quad (3.1)$$

Both directions point *away* from the surface: ω_o toward the viewer (or the previous vertex of a path) and ω_i toward where the light comes from. When both lie on the same side of the surface, f describes reflection (a BRDF); on opposite sides it describes transmission (a BTDF). The BSDF covers both.

3.2 The equation

Energy balance at a surface point says that the radiance leaving it is the emitted radiance plus all scattered incident radiance (Kajiya 1986):

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\mathcal{S}^2} f(\mathbf{x}, \omega_o, \omega_i) L_i(\mathbf{x}, \omega_i) |\cos \theta_i| d\omega_i. \quad (3.2)$$

The integral runs over the whole sphere $\mathcal{S}^2$ so that transmitted light is included. In vacuum, the incident radiance is the outgoing radiance of the first surface in direction ω_i : $L_i(\mathbf{x}, \omega_i) = L_o(\mathbf{r}(\mathbf{x}, \omega_i), -\omega_i)$, where $\mathbf{r}$ is the ray casting function. The unknown L appears on both sides; (3.2) is an integral equation, and a path tracer solves it by recursively estimating the integral.

3.3 What makes a BSDF physically plausible

- **Positivity:** $f \geq 0$.
- **Energy conservation:** a surface cannot scatter more light than it receives. For every ω_o the *directional albedo*

$$\rho(\omega_o) = \int_{\mathcal{S}^2} f(\omega_o, \omega_i) |\cos \theta_i| d\omega_i \leq 1. \quad (3.3)$$

The white furnace test of Section 12 measures exactly this quantity.

- **Reciprocity:** for reflection $f(\omega_o, \omega_i) = f(\omega_i, \omega_o)$ (Helmholtz reciprocity). For refraction the generalised relation is $f(\omega_o, \omega_i)/\eta_o^2 = f(\omega_i, \omega_o)/\eta_i^2$, where η_o, η_i are the refractive indices on the sides of ω_o and ω_i (Veach 1997, ch. 5).

A few terms of this renderer's BSDF are deliberately not reciprocal (the clearcoat attenuation and the metal energy compensation). Section 10 points them out and explains why that is harmless for a path tracer that only traces from the camera.

3.4 Light paths

Substituting the equation into itself expands radiance into a sum over path lengths,

$$L = L_e + \mathcal{T}L_e + \mathcal{T}^2L_e + \mathcal{T}^3L_e + \dots,$$

where the transport operator $\mathcal{T}$ applies one bounce (the integral in (3.2)). The term $\mathcal{T}^k L_e$ is the light that reaches the camera after exactly k scattering events. Veach (1997) rewrites this sum as an integral over *path space*, the set of all polylines $\bar{x} = \mathbf{x}_0 \mathbf{x}_1 \dots \mathbf{x}_k$ from the camera to a light. A path tracer samples such polylines vertex by vertex, starting at the camera; each additional vertex multiplies in one more BSDF value, which gives the *path throughput* of Section 4.8.

3.5 The area form and direct lighting

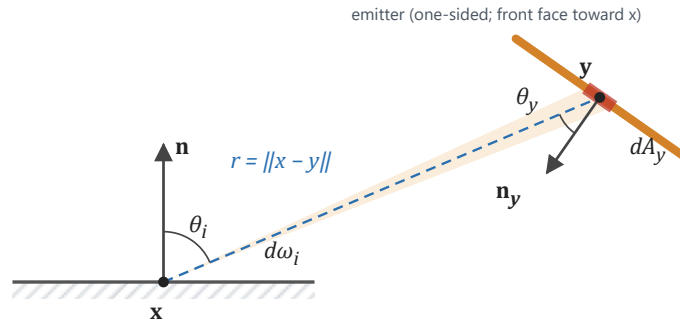


Figure 3. Changing variables from the solid angle $d\omega_i$ seen from $\mathbf{x}$ to the area dA_y on a light: $d\omega_i = dA_y |\cos \theta_y| / r^2$.

For light sources it is convenient to integrate over their surface instead of over directions. With the solid angle of a patch from Section 2.1,

$$d\omega_i = \frac{|\cos \theta_y|}{\|\mathbf{x} - \mathbf{y}\|^2} dA_y,$$

the *direct* illumination — the part of the integral in which ω_i points at an emitter — becomes

$$L_{\text{direct}}(\mathbf{x}, \omega_o) = \int_{A_{\text{lights}}} f(\mathbf{x}, \omega_o, \omega_i) L_e(\mathbf{y}, -\omega_i) V(\mathbf{x}, \mathbf{y}) \frac{|\cos \theta_i| |\cos \theta_y|}{\|\mathbf{x} - \mathbf{y}\|^2} dA_y, \quad (3.4)$$

with $\omega_i = (\mathbf{y} - \mathbf{x})/\|\mathbf{y} - \mathbf{x}\|$, θ_y the angle between the light's normal and $-\omega_i$, and the visibility V equal to 1 when nothing blocks the segment. Next event estimation (Section 13) is a Monte Carlo estimate of (3.4), in which the binary visibility is replaced by a transmittance when media or invisible boundaries lie between the two points.

4. Monte Carlo integration

4.1 The estimator

To estimate $I = \int_{\Omega} f(x) dx$, draw N independent samples X_k from a probability density p that is positive wherever $f \neq 0$, and average

$$\langle I \rangle_N = \frac{1}{N} \sum_{k=1}^N \frac{f(X_k)}{p(X_k)}. \quad (4.1)$$

The estimator is *unbiased*: $E[f(X)/p(X)] = \int \frac{f(x)}{p(x)} p(x) dx = I$. Its variance is $\text{Var}[f/p]/N$, so the standard error falls as $1/\sqrt{N}$: halving the noise requires four times as many samples. This is the noise you watch fade as the progressive image accumulates frames. The dimension of Ω does not enter this rate, which makes Monte Carlo the method of choice for the infinite-dimensional path integral.

4.2 Importance sampling

The variance of f/p is small when p is nearly proportional to f . If $p = f/I$ exactly, every sample returns I and the variance vanishes. We can never sample the complete integrand of a light path proportionally, but we can sample individual factors of it: directions proportional to the BSDF times the cosine (BSDF sampling), points proportional to the emitted power (light sampling), distances proportional to the transmittance in a medium (free-flight sampling). Most of the mathematics in Parts III–V consists of constructing such densities and evaluating them exactly.

4.3 Generating samples

The shaders produce uniform random numbers $\xi \in [0, 1)$ (Section 5.5). A sample with density p on the real line is obtained by the *inversion method*: with the cumulative distribution function $P(x) = \int_{-\infty}^x p$, the random variable $X = P^{-1}(\xi)$ has density p . For example, distances with density $p(t) = \sigma e^{-\sigma t}$ have $P(t) = 1 - e^{-\sigma t}$ and are generated as $t = -\ln(1 - \xi)/\sigma$ (Section 19). Multidimensional samples are built from several uniform numbers and a mapping $y = T(x)$; densities transform with the absolute Jacobian determinant,

$$p_Y(y) = \frac{p_X(x)}{|\det J_T(x)|}. \quad (4.2)$$

Every sampling routine in the renderer is such a mapping: the uniform disk and cosine-weighted hemisphere (Section 11.2), the uniform triangle (Section 13.2), GGX visible normals (Section 9.7) and Henyey–Greenstein directions (Section 18.3).

4.4 Densities per solid angle and per area

A density is always relative to a measure. BSDF sampling produces directions with a density p_ω per steradian, while light sampling produces points with a density p_A per unit area. To compare or combine them, as multiple importance sampling must, convert with the Jacobian from Section 3.5:

$$p_\omega(\omega_i) = p_A(\mathbf{y}) \frac{\|\mathbf{x} - \mathbf{y}\|^2}{|\cos \theta_y|}. \quad (4.3)$$

4.5 Multiple importance sampling

Often several sampling techniques are available and each is good in different situations. Sampling the light is excellent for a small bright light above a rough surface; sampling the BSDF is excellent for a glossy surface reflecting a large light. Multiple importance sampling (MIS; Veach & Guibas 1995) uses all techniques and weights their samples so that each dominates where it has low variance. With techniques $i = 1 \dots n$ drawing n_i samples $X_{i,j}$ each,

$$\langle I \rangle = \sum_{i=1}^n \frac{1}{n_i} \sum_{j=1}^{n_i} w_i(X_{i,j}) \frac{f(X_{i,j})}{p_i(X_{i,j})}, \quad \sum_i w_i(x) = 1 \text{ wherever } f(x) \neq 0, \quad (4.4)$$

is unbiased for any weights that sum to one. Two classic choices are

$$w_i^{\text{balance}}(x) = \frac{n_i p_i(x)}{\sum_k n_k p_k(x)}, \quad w_i^{\text{power}}(x) = \frac{(n_i p_i(x))^2}{\sum_k (n_k p_k(x))^2}. \quad (4.5)$$

Veach proved the balance heuristic to be close to optimal among all weighting schemes. The power heuristic sharpens the weights and in practice suppresses the rare high-variance samples of each technique even better. The renderer uses the power heuristic between light sampling and BSDF or phase function sampling (Section 14).

4.6 One-sample MIS: sampling from a mixture

Instead of drawing samples from every technique, one can pick a single technique at random, with probability c_i , and draw one sample from it. Using the balance heuristic, the estimator simplifies to

$$\frac{w_i(X) f(X)}{c_i p_i(X)} = \frac{f(X)}{\sum_k c_k p_k(X)}. \quad (4.6)$$

In words: we sample from the *mixture density* $\sum_k c_k p_k$, and we must divide by that full density — not by the density of the technique that happened to be chosen. This is how the renderer samples its multi-lobe BSDF: a lobe is selected at random, a direction is drawn from it, and then `EvalBsdf` computes the complete BSDF value together with the complete mixture density (Section 11.3).

4.7 Russian roulette

A path could bounce forever, but cutting it off after a fixed number of bounces biases the result. Russian roulette terminates randomly instead: continue with probability q and divide the throughput by q , otherwise stop. The expected value is unchanged, $q \cdot \frac{X}{q} + (1 - q) \cdot 0 = X$, while dim paths stop costing time. Section 15 gives the rule used.

4.8 The path tracing estimator

Applying (4.1) recursively to the rendering equation, with one sample per bounce, gives the path tracing estimator. At the k -th vertex a direction $\omega_{i,k}$ is sampled with density p_k , and every contribution found further along the path is multiplied by the *throughput*

$$\beta_k = \prod_{j=1}^k \frac{f(\mathbf{x}_j, \omega_{o,j}, \omega_{i,j}) |\cos \theta_{i,j}|}{p_j(\omega_{i,j})}. \quad (4.7)$$

A basic path tracer — without media, light sampling or MIS — looks like this:

```
L = 0;  β = 1;  ray = cameraRay(pixel)
loop:
  hit = trace(ray)
  if no hit: L += β · Lenv(ray.direction); break
  L += β · Le(hit, -ray.direction)
  (wi, pdf) = sampleBSDF(hit, -ray.direction)
  β *= f(hit, -ray.direction, wi) · |cos θi| / pdf
  ray = (hit, wi)
  if russianRoulette(β): break
```

The real loop (Section 16) adds light connections with MIS weights, media and boundaries, but its skeleton is exactly this.

PART II

Camera and geometry

This part follows a single camera sample from the pixel to the first surface: how the camera ray is generated, how frames are accumulated, where the random numbers come from, how an intersection is turned into a position and normals, how new rays avoid hitting the surface they start on, and how directions are expressed in the local frame in which the BSDF is evaluated.

5. Camera, pixels and random numbers

5.1 The pinhole camera

The camera is described by its position $\mathbf{c}$, an orthonormal basis (forward $\mathbf{f}$, right $\mathbf{r}$, up $\mathbf{u}$), the vertical field of view ϕ and the aspect ratio $a = \text{width}/\text{height}$. Place a virtual image plane at distance 1 along $\mathbf{f}$. Its half-height is $\tan(\phi/2)$ and its half-width $a \tan(\phi/2)$. A point $\mathbf{s} \in [0, 1]^2$ in normalised image coordinates maps to $\mathbf{d} = 2\mathbf{s} - 1 \in [-1, 1]^2$ and to the ray direction

$$\omega(\mathbf{s}) = \text{normalize}(\mathbf{f} + d_x a \tan(\phi/2) \mathbf{r} + d_y \tan(\phi/2) \mathbf{u}). \quad (5.1)$$

The CPU passes $\mathbf{u}' = -\mathbf{u}$ (`CameraUp = -m_Camera->Up` in `PathTracer.cpp`) because image rows grow downward while the world's up axis points upward. `tanHalfFov` and `aspectRatio` in the `GlobalData` uniform buffer are precomputed on the CPU.

5.2 Pixel jitter: antialiasing for free

The image coordinates of a sample are $\mathbf{s} = (\mathbf{p} + \boldsymbol{\xi})/(W, H)$, where $\mathbf{p}$ is the integer pixel index and $\boldsymbol{\xi}$ two uniform random numbers. Each sample therefore lands at a uniformly random position inside its pixel, which is a Monte Carlo estimate of the pixel integral (1.1) with a box filter. Edges are antialiased automatically as the samples accumulate; no separate antialiasing pass is needed.

5.3 The thin lens

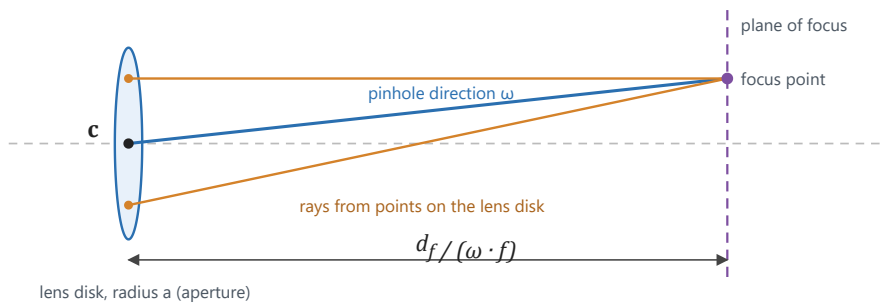


Figure 4. Thin lens. All rays that leave the lens disk toward the focus point of a pixel meet again on the plane of focus, so geometry at that distance is sharp; geometry in front of or behind it is blurred over a circle of confusion. The dimension line shows the distance to the focus point along ω .

A real lens gathers light over an aperture of finite size, which produces depth of field. The thin-lens model replaces the pinhole by a disk of radius a (aperture) perpendicular to the view axis and focuses at the distance d_f (focusDistance), measured along the view axis $\mathbf{f}$. For a pinhole direction ω the focus point is

$$\mathbf{P} = \mathbf{c} + \omega \frac{d_f}{\omega \cdot \mathbf{f}},$$

where the division converts the axial distance into a distance along ω . A random point on the lens, $\mathbf{c}' = \mathbf{c} + \ell_x \mathbf{r} + \ell_y \mathbf{u}'$ with (ℓ_x, ℓ_y) uniform on the disk of radius a (SampleUniformDisk, Section 11.2), becomes the ray origin, and the ray direction is $\text{normalize}(\mathbf{P} - \mathbf{c}')$. With Aperture = 0, the value the application currently sets, the code skips this step and the camera is an ideal pinhole.

5.4 Progressive accumulation

Each frame computes $S = \text{samplesPerPixel}$ independent estimates per pixel and averages them into x_N , the estimate of frame N . The ray generation shader reads the running sum of all previous frames from the accumulation image, adds x_N and writes the new sum into the other accumulation image (the two swap roles every frame, which avoids reading and writing the same image). The displayed value is the running mean

$$\bar{x}_N = \frac{1}{N} \sum_{k=1}^N x_k,$$

computed as the sum divided by frameIndex. Because every frame's samples are independent and unbiased, $\bar{x}_N$ is unbiased for every N and its variance falls as $1/(NS)$. The CPU resets FrameIndex to 1 — discarding the history — whenever the camera moves or the C key is held, and to 0 after a resize or a shader reload (the next frame then increments it to 1). A frame with index 1 ignores the input accumulation image.

5.5 Random numbers

Every random decision in the renderer consumes numbers from a per-thread 32-bit state that is advanced with the PCG hash of Jarzynski & Olano (2020), a permuted congruential step with an output permutation. The state is seeded from the pixel coordinates and the frame index by nested hashing, so neighbouring pixels and consecutive frames start from decorrelated states:

$$\text{seed} = h(x + h(y + h(\text{frame}))).$$

Random hashes the state again and converts the top 24 bits to a float: $\xi = \lfloor s/2^8 \rfloor \cdot 2^{-24}$. Every value in $[0, 1 - 2^{-24}]$ on this grid is exactly representable in single precision, so ξ is uniform and never equals 1. That matters for expressions such as $\ln(1 - \xi)$, which would otherwise produce $-\infty$.

common.glsl · PcgHash

```
uint PcgHash(uint value)
{
    uint state = value * 747796405u + 2891336453u;
    uint word = ((state >> ((state >> 28u) + 4u)) ^ state) * 277803737u;
    return (word >> 22u) ^ word;
}
```

common.glsl · Random

```
float Random(inout uint state)
{
    state = PcgHash(state);
    return float(state >> 8) * (1.0 / 16777216.0);
}
```

CAVEAT

The samples are independent (pseudo-random), not stratified. The error therefore falls as $N^{-1/2}$. Low-discrepancy sequences (Sobol, blue-noise dithered sequences) converge faster for the low-dimensional parts of the integrand — pixel position, lens position, the first bounce — and are a natural extension (Section 26).

5.6 The ray generation entry point

The complete entry point ties the pieces together: seeding, pixel jitter, the optional thin lens, the loop over samples per pixel, a guard against invalid samples (Section 25), accumulation and the conversion to display values (Section 24).

raygen.rgen · main

```

void main()
{
    ivec2 pixel = ivec2(gl_LaunchIDEXT.xy);
    uint rng = PcgHash(uint(pixel.x) + PcgHash(uint(pixel.y) + PcgHash(globalData.frameIndex)));

    vec3 cameraPosition = globalData.cameraPosition.xyz;
    vec3 forward = globalData.cameraForward.xyz;
    vec3 right = globalData.cameraRight.xyz;
    vec3 up = globalData.cameraUp.xyz;

    vec3 radiance = vec3(0.0);
    uint samples = max(globalData.samplesPerPixel, 1u);
    for (uint s = 0u; s < samples; s++)
    {
        vec2 uv = (vec2(pixel) + Random2(rng)) / vec2(gl_LaunchSizeEXT.xy);
        vec2 d = uv * 2.0 - 1.0;
        vec3 direction = normalize(forward + d.x * globalData.aspectRatio * globalData.tanHalfFov * right + d.y *
globalData.tanHalfFov * up);
        vec3 origin = cameraPosition;
        if (globalData.aperture > 0.0)
        {
            vec3 focusPoint = cameraPosition + direction * (globalData.focusDistance / dot(direction, forward));
            vec2 lens = SampleUniformDisk(Random2(rng)) * globalData.aperture;
            origin = cameraPosition + right * lens.x + up * lens.y;
            direction = normalize(focusPoint - origin);
        }

        vec3 sampleRadiance = TracePath(origin, direction, rng);
        if (any(isnan(sampleRadiance)) || any(isinf(sampleRadiance)))
            sampleRadiance = vec3(0.0);
        radiance += sampleRadiance;
    }
    radiance /= float(samples);

    vec3 accumulated = radiance;
    if (globalData.frameIndex > 1u)
        accumulated += imageLoad(accumulationImageIn, pixel).rgb;
    imageStore(accumulationImageOut, pixel, vec4(accumulated, 0.0));

    vec3 color = accumulated / float(max(globalData.frameIndex, 1u)) * globalData.exposure;
    imageStore(outputImage, pixel, vec4(LinearToSrgb(TonemapAces(color)), 1.0));
}

```

6. Geometry: intersections and normals

6.1 Acceleration structures and instancing

Each loaded model is one bottom-level acceleration structure (BLAS). A model whose triangles use several materials is split into *parts* — contiguous index ranges — and every part becomes one geometry of the BLAS. The top-level acceleration structure (TLAS) holds the instances, each with a 3×4 object-to-world matrix and a 24-bit *custom index*. `Scene::Build` appends one `MeshPrimitive` record per instanced part — vertex buffer address, index buffer address already offset to the part's first index, and material index — and sets each instance's custom index to the position of its first record. The closest-hit shader then finds the record of the hit triangle with

```
meshPrimitives[gl_InstanceCustomIndexEXT + gl_GeometryIndexEXT],
```

and reads the three vertices of triangle `gl_PrimitiveID` through buffer device addresses. A per-instance material override (the third argument of `AddInstance`) simply replaces the material index in these records.

6.2 Reconstructing the hit

The ray tracing hardware reports the hit distance and the barycentric coordinates (b_1, b_2) of the hit point on the triangle; the third is $b_0 = 1 - b_1 - b_2$. Any vertex attribute a is interpolated as $a = b_0a_0 + b_1a_1 + b_2a_2$. The shader interpolates the object-space position, the vertex normals and the texture coordinates, and transforms position and normals to world space:

rayclosesthit.rchit · main

```

void main()
{
    MeshPrimitive mesh = meshPrimitives[gl_InstanceCustomIndexEXT + gl_GeometryIndexEXT];
    VertexBuffer vertexBuffer = VertexBuffer(mesh.vertexAddress);
    IndexBuffer indexBuffer = IndexBuffer(mesh.indexAddress);

    uint baseIndex = gl_PrimitiveID * 3;
    Vertex v0 = vertexBuffer.vertices[indexBuffer.indices[baseIndex + 0]];
    Vertex v1 = vertexBuffer.vertices[indexBuffer.indices[baseIndex + 1]];
    Vertex v2 = vertexBuffer.vertices[indexBuffer.indices[baseIndex + 2]];

    vec3 bary = vec3(1.0 - attribs.x - attribs.y, attribs.x, attribs.y);
    vec3 localPosition = v0.position * bary.x + v1.position * bary.y + v2.position * bary.z;
    vec3 localNormal = v0.normal * bary.x + v1.normal * bary.y + v2.normal * bary.z;
    vec3 localGeometricNormal = cross(v1.position - v0.position, v2.position - v0.position);

    vec3 geometricNormal = normalize(vec3(localGeometricNormal * gl_WorldToObjectEXT));
    vec3 shadingNormal = dot(localNormal, localNormal) > 0.0 ? normalize(vec3(localNormal * gl_WorldToObjectEXT)) :
geometricNormal;
    if (dot(shadingNormal, geometricNormal) < 0.0)
        shadingNormal = -shadingNormal;

    payload.position = gl_ObjectToWorldEXT * vec4(localPosition, 1.0);
    payload.t = gl_HitTEXT;
    payload.geometricNormal = geometricNormal;
    payload.materialIndex = mesh.materialIndex;
    payload.shadingNormal = shadingNormal;
    payload.frontFace = dot(gl_WorldRayDirectionEXT, geometricNormal) < 0.0 ? 1u : 0u;
    payload.uv = v0.uv * bary.x + v1.uv * bary.y + v2.uv * bary.z;
}

```

The position is interpolated in object space and transformed, rather than computed as origin + t -direction. Both are mathematically equal, but the interpolated point lies on the triangle up to the rounding of a few operations, while origin + t -direction accumulates the error of the ray-triangle test along the whole ray length. The offset of Section 6.5 relies on this small error.

6.3 Why normals transform with the inverse transpose

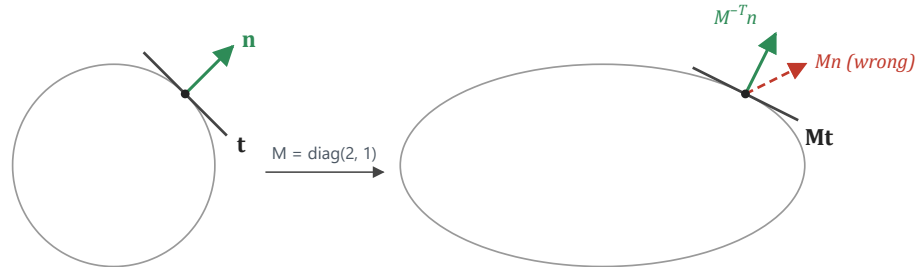


Figure 5. Under a non-uniform scale, tangents transform with M but normals must transform with M^{-T} to stay perpendicular to the surface.

Tangent vectors of a surface transform like positions: a tangent $\mathbf{t}$ becomes $M\mathbf{t}$, where M is the upper-left 3×3 block of the object-to-world matrix. A normal must stay perpendicular to all transformed tangents. We look for a matrix N with $(N\mathbf{n}) \cdot (M\mathbf{t}) = 0$ whenever $\mathbf{n} \cdot \mathbf{t} = 0$. Writing the dot product as a matrix product, $(N\mathbf{n})^T M\mathbf{t} = \mathbf{n}^T N^T M\mathbf{t}$, which vanishes for all such pairs if $N^T M = I$, i.e.

$$N = M^{-T}. \quad (6.1)$$

For rotations and uniform scales M^{-T} is proportional to M , which is why the mistake of transforming normals with M goes unnoticed until an object is scaled non-uniformly — like the room's walls and the pedestal in the showcase scene. The shader already has M^{-1} in `gl_WorldToObjectEXT` (a 3×4 matrix $[M^{-1} \mid -M^{-1}\mathbf{t}]$). Multiplying a vector from the left, $\mathbf{n} * \text{gl_WorldToObjectEXT}$, computes $(M^{-1})^T \mathbf{n}$ in its first three components — exactly (6.1) without an explicit inverse or transpose. The result is renormalised.

6.4 Geometric normals, shading normals and front faces

Two normals are computed at every hit:

- The **geometric normal** $\mathbf{n}_g$ is the normalised cross product of two triangle edges, $(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)$. It is exactly perpendicular to the triangle, and its orientation follows the vertex winding: for a closed mesh wound counter-clockwise when seen from outside, it points outward. The renderer relies on this orientation to decide whether a ray enters or leaves an object (media, Section 22) and which side of an emitter emits.
- The **shading normal** $\mathbf{n}$ interpolates the vertex normals, so that a coarse triangle mesh looks smooth. It is flipped, if necessary, into the same hemisphere as $\mathbf{n}_g$, so that both normals always agree on which side is "outside".

The payload field `frontFace` records whether the ray arrived from the side $\mathbf{n}_g$ points to: `dot(rayDirection, n_g) < 0`. Neither normal is flipped toward the ray; the BSDF handles rays arriving from either side (Section 10.8).

6.5 Starting new rays without hitting the old surface

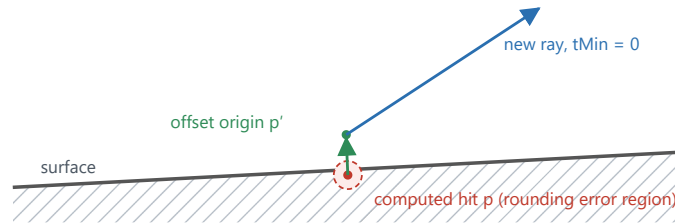


Figure 6. The computed hit point can lie slightly on the wrong side of the surface. Moving the origin a few units in the last place along the geometric normal puts it safely on the intended side.

A computed intersection point is not exactly on the surface: it carries rounding error proportional to the magnitude of its coordinates. A ray started from it may immediately re-intersect the same triangle ("shadow acne", speckled self-shadowing). The classic fix, a small minimum ray distance `tMin`, depends on the scene scale and fails for grazing rays. The renderer instead uses the method of Wächter & Binder (2019): it pushes the origin along the geometric normal by a fixed number of *units in the last place* (ULPs), by adding an integer to the bit pattern of each floating-point coordinate. Far from the origin, one ULP is proportionally large, so the offset scales with the error it has to cover. Near zero, where ULPs become tiny, a small absolute offset of 2^{-16} units is used instead. All rays are then traced with `tMin = 0`.

common.glsl · OffsetRay

```
vec3 OffsetRay(vec3 p, vec3 n)
{
    const float origin = 1.0 / 32.0;
    const float floatScale = 1.0 / 65536.0;
    const float intScale = 256.0;
    ivec3 offset = ivec3(intScale * n);
    vec3 moved = vec3(
        intBitsToFloat(floatBitsToInt(p.x) + (p.x < 0.0 ? -offset.x : offset.x)),
        intBitsToFloat(floatBitsToInt(p.y) + (p.y < 0.0 ? -offset.y : offset.y)),
        intBitsToFloat(floatBitsToInt(p.z) + (p.z < 0.0 ? -offset.z : offset.z)));
    return vec3(
        abs(p.x) < origin ? p.x + floatScale * n.x : moved.x,
        abs(p.y) < origin ? p.y + floatScale * n.y : moved.y,
        abs(p.z) < origin ? p.z + floatScale * n.z : moved.z);
}
```

The side to offset to depends on where the new ray goes:

Situation	Offset direction
reflection (sampled ω_i on the side of ω_o)	$+\mathbf{n}_g$ if $\omega_i \cdot \mathbf{n}_g > 0$, otherwise $-\mathbf{n}_g$
transmission (refraction through the surface)	the side of ω_i , i.e. through the surface
crossing an invisible (null) boundary	away from the side the ray came from
shadow ray toward a light	the side of the light direction

7. The local shading frame

7.1 An orthonormal basis around the normal

BSDFs are easiest to write in a coordinate system in which the shading normal is the z axis. Then $\cos \theta = \omega_z$, $\sin^2 \theta = \omega_x^2 + \omega_y^2$, and isotropic functions depend only on z components and dot products. `BuildFrame` constructs two tangents $\mathbf{t}$, $\mathbf{b}$ that complete $\mathbf{n}$ to a right-handed orthonormal basis, using the branch-free construction of Duff et al. (2017). With $s = \text{sign}(n_z)$, $a = -1/(s + n_z)$ and $b = n_x n_y a$:

$$\mathbf{t} = (1 + s n_x^2 a, s b, -s n_x), \quad \mathbf{b} = (b, s + n_y^2 a, -n_y).$$

The earlier construction by Frisvad fails near $n_z = -1$; using the sign s removes that singularity without a branch. `ToLocal` projects onto the three axes (multiplication with the transpose of the frame matrix) and `ToWorld` multiplies with the frame.

common.glsl · BuildFrame

```
mat3 BuildFrame(vec3 n)
{
    float s = n.z >= 0.0 ? 1.0 : -1.0;
    float a = -1.0 / (s + n.z);
    float b = n.x * n.y * a;
    return mat3(vec3(1.0 + s * n.x * n.x * a, s * b, -s * n.x), vec3(b, s + n.y * n.y * a, -n.y), n);
}
```

7.2 Directions and angles

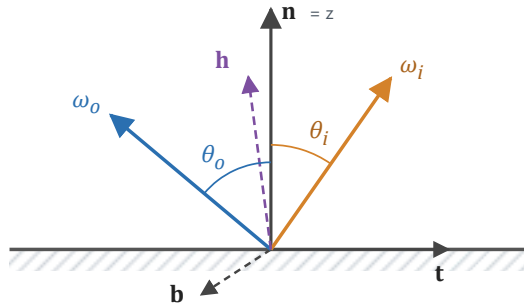


Figure 7. The local frame. Both ω_o and ω_i point away from the surface; their polar angles are measured from the shading normal $\mathbf{n} = \mathbf{z}$. The half vector $\mathbf{h}$ bisects them.

At a path vertex, `wo = -direction` is the reversed direction of the arriving ray, and the sampled `wi` becomes the direction of the next ray. In local coordinates, $\omega_z > 0$ means "on the side the shading normal points to", which for closed meshes is the outside. A ray that hits a surface from behind has $\omega_{o,z} < 0$; the BSDF handles that case explicitly (Section 10.8). The geometric normal is transformed into the same local frame (`ngLocal`), because the BSDF needs it to detect directions for which shading and geometric normals disagree.

7.3 Reflection and refraction directions

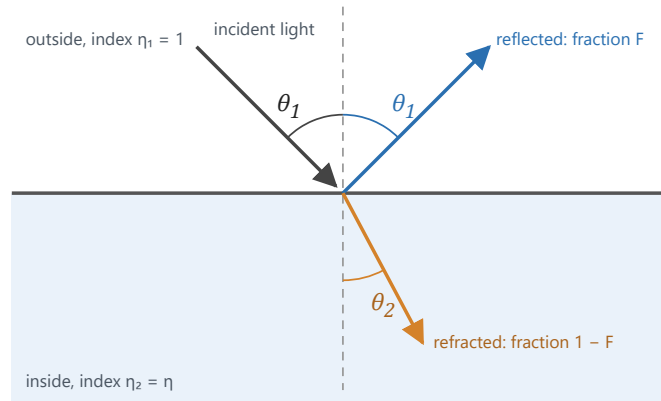


Figure 8. Reflection and refraction at a smooth interface. The reflected ray leaves at the angle of incidence; the refracted ray obeys Snell's law $\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2$. The Fresnel reflectance F (Section 8) splits the energy between the two.

With all directions pointing away from the surface, mirror reflection of ω_o about a normal $\mathbf{m}$ is

$$\omega_r = 2(\omega_o \cdot \mathbf{m}) \mathbf{m} - \omega_o, \quad (7.1)$$

which the code writes as `reflect(-wo, m)` (GLSL's `reflect(I, N)` expects an incident vector pointing toward the surface). Refraction follows Snell's law $\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2$. With the relative index $\eta = \eta_2/\eta_1$ and $\cos \theta_1 = \omega \cdot \mathbf{m} > 0$, the transmitted direction is

$$\omega_t = -\frac{\omega}{\eta} + \left(\frac{\cos \theta_1}{\eta} - \cos \theta_2 \right) \mathbf{m}, \quad \cos \theta_2 = \sqrt{1 - \frac{1 - \cos^2 \theta_1}{\eta^2}}. \quad (7.2)$$

It follows from splitting ω into components along and perpendicular to $\mathbf{m}$: the tangential component shrinks by $1/\eta$ (that is Snell's law), and the normal component is chosen so that ω_t has unit length on the other side. When the square root's argument is negative — $\sin \theta_2$ would exceed 1 — no transmitted direction exists: *total internal reflection*. `Refract` implements (7.2) for either side: if ω lies below $\mathbf{m}$ it swaps $\eta \rightarrow 1/\eta$ and flips $\mathbf{m}$, and it returns `false` on total internal reflection.

bsdf.glsl · Refract

```
bool Refract(vec3 w, vec3 n, float eta, out vec3 wt)
{
    float cosThetaI = dot(n, w);
    if (cosThetaI < 0.0)
    {
        eta = 1.0 / eta;
        cosThetaI = -cosThetaI;
        n = -n;
    }
    float sin2ThetaT = max(0.0, 1.0 - cosThetaI * cosThetaI) / (eta * eta);
    if (sin2ThetaT >= 1.0)
        return false;
    float cosThetaT = sqrt(max(0.0, 1.0 - sin2ThetaT));
    wt = -w / eta + (cosThetaI / eta - cosThetaT) * n;
    return true;
}
```

PART III

The BSDF

The BSDF decides what a surface looks like. This part builds it from the ground up: the Fresnel equations that govern reflection at a single smooth interface, the microfacet theory that turns a rough surface into a statistical collection of tiny smooth interfaces, the complete multi-lobe material used by the renderer, the algorithm that samples it, and the tests that verify it conserves energy.

8. Fresnel reflectance

8.1 Where the Fresnel equations come from

Light is an electromagnetic wave. At the boundary between two media with refractive indices η_1 and η_2 , Maxwell's equations require the tangential components of the electric and magnetic fields to be continuous. Solving these boundary conditions for an incident plane wave gives the amplitudes of the reflected and transmitted waves. The result depends on the polarisation: the component whose electric field is perpendicular to the plane of incidence (s, "senkrecht") reflects differently from the component parallel to it (p). The reflected *power* fraction is the square of the amplitude ratio.

8.2 The dielectric Fresnel equations

For light arriving at angle θ_1 and refracting to θ_2 (Snell's law), with the relative index $\eta = \eta_2/\eta_1$, the amplitude reflection coefficients are

$$r_{\parallel} = \frac{\eta \cos \theta_1 - \cos \theta_2}{\eta \cos \theta_1 + \cos \theta_2}, \quad r_{\perp} = \frac{\cos \theta_1 - \eta \cos \theta_2}{\cos \theta_1 + \eta \cos \theta_2}, \quad (8.1)$$

and unpolarised light — the assumption throughout the renderer — is reflected with the average

$$F(\theta_1, \eta) = \frac{1}{2} (r_{\parallel}^2 + r_{\perp}^2), \quad \text{transmitted fraction } 1 - F. \quad (8.2)$$

Dielectrics (glass, water, plastic, varnish) absorb nothing at the interface itself, so exactly $1 - F$ of the energy crosses it.

IN THE CODE

`FresnelDielectric(cosThetaI, eta)` takes the cosine relative to the outward normal and $\eta = \text{inside/outside}$. A negative cosine means the light arrives from inside; the function then swaps to $1/\eta$ and uses $|\cos \theta_1|$, so one function covers both sides of every interface.

bsdf.glsl · FresnelDielectric

```

float FresnelDielectric(float cosThetaI, float eta)
{
    cosThetaI = clamp(cosThetaI, -1.0, 1.0);
    if (cosThetaI < 0.0)
    {
        eta = 1.0 / eta;
        cosThetaI = -cosThetaI;
    }
    float sin2ThetaT = (1.0 - cosThetaI * cosThetaI) / (eta * eta);
    if (sin2ThetaT >= 1.0)
        return 1.0;
    float cosThetaT = sqrt(max(0.0, 1.0 - sin2ThetaT));
    float rParallel = (eta * cosThetaI - cosThetaT) / (eta * cosThetaI + cosThetaT);
    float rPerpendicular = (cosThetaI - eta * cosThetaT) / (cosThetaI + eta * cosThetaT);
    return 0.5 * (rParallel * rParallel + rPerpendicular * rPerpendicular);
}

```

8.3 Total internal reflection

From inside a denser medium, Snell's law demands $\sin \theta_2 = \eta \sin \theta_1$ with $\eta > 1$ now being inside/outside. Above the critical angle $\theta_c = \arcsin(1/\eta) \approx 41.8^\circ$ for glass — no transmitted wave exists and $F = 1$: total internal reflection. It is what makes the edges of the glass sphere and the jade dragon bright, and what traps light inside dense scattering media.

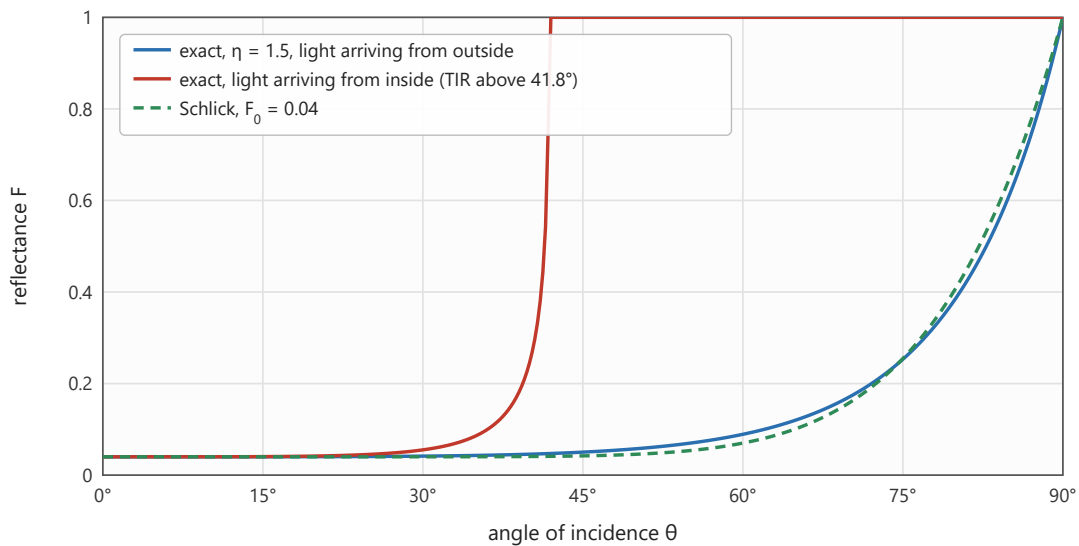


Figure 9. Fresnel reflectance of glass ($\eta = 1.5$) as a function of the angle of incidence, for light arriving from outside (blue) and from inside (red, with total internal reflection above 41.8°). Schlick's approximation (green, dashed) is close to the exact curve for dielectrics.

8.4 Reflectance at normal incidence

At $\theta_1 = 0$ both polarisations agree and (8.2) reduces to

$$F_0 = \left(\frac{\eta - 1}{\eta + 1} \right)^2. \quad (8.3)$$

Material	η	F_0
water	1.33	0.020
glass, plastic, clear coat	1.50	0.040
jade (scene)	1.62	0.056
diamond	2.42	0.172

Every dielectric reflects only a few percent at normal incidence and everything at grazing incidence. This rise toward grazing angles is why even rough, diffuse-looking objects show a bright rim, and why the floor reflects the room strongly when seen at a shallow angle.

8.5 Schlick's approximation and metals

Schlick (1994) approximates the Fresnel curve by

$$F(\theta) \approx F_0 + (1 - F_0)(1 - \cos \theta)^5. \quad (8.4)$$

Conductors (metals) have a complex refractive index $\eta + i\kappa$: the transmitted wave is absorbed within nanometres, so metals have no diffuse or transmitted light, their reflectance is high, and it varies with wavelength — which gives gold and copper their colour. Instead of specifying η and κ per colour channel, the widely used *metallic workflow* specifies the reflectance at normal incidence directly as a colour, $F_0 = \text{BaseColor}$, and uses Schlick's curve (8.4) to rise to white at grazing incidence. This is how the metal lobe is evaluated (`FresnelSchlick`). It is an approximation: real metals show a slight dip before grazing and a hue shift toward the edge that (8.4) cannot express, but the error is small compared to the gain in simplicity.

bsdf.glsl · FresnelSchlick

```
vec3 FresnelSchlick(vec3 f0, float cosTheta)
{
    return f0 + (1.0 - f0) * SchlickWeight(cosTheta);
}
```

8.6 Fresnel reflectance of diffuse light

Section 10.5 needs the fraction of *uniformly diffuse* light that an interface reflects. Averaging F over the cosine-weighted hemisphere gives the diffuse Fresnel reflectance

$$F_{dr}(\eta) = \frac{1}{\pi} \int_{\mathcal{H}^2} F(\theta, \eta) \cos \theta \, d\omega = 2 \int_0^1 F(\mu, \eta) \mu \, d\mu. \quad (8.5)$$

For light arriving from the outside of glass it is small, $F_{dr}(1.5) \approx 0.092$. The renderer evaluates the polynomial fit of d'Eon & Irving (2011) — accurate to about 0.1% for $1 \leq \eta \leq 2$, as used in Mitsuba — which only needs $1/\eta$:

$$F_{dr}(\eta) \approx 0.919317 - \frac{3.4793}{\eta} + \frac{6.75335}{\eta^2} - \frac{7.80989}{\eta^3} + \frac{4.98554}{\eta^4} - \frac{1.36881}{\eta^5}.$$

bsdf.glsl · FresnelDiffuseReflectance

```
float FresnelDiffuseReflectance(float eta)
{
    float i = 1.0 / eta;
    return 0.919317 + i * (-3.4793 + i * (6.75335 + i * (-7.80989 + i * (4.98554 - 1.36881 * i))));
}
```

DERIVATION · INTERNAL DIFFUSE REFLECTANCE

Diffuse light arriving from *inside* is reflected much more strongly, because everything beyond the critical angle is totally reflected. The internal value follows from the external one without another integral. Imagine thermal equilibrium: isotropic radiance L outside and $\eta^2 L$ inside (radiance scales with η^2 , Section 2.3). The flux crossing inward per unit area is $\pi L(1 - F_{dr}^{\text{ext}})$, the flux crossing outward $\pi \eta^2 L(1 - F_{dr}^{\text{int}})$. In equilibrium both are equal, so

$$1 - F_{dr}^{\text{int}} = \frac{1 - F_{dr}^{\text{ext}}}{\eta^2}. \quad (8.6)$$

For glass, $F_{dr}^{\text{int}} = 1 - 0.908/2.25 \approx 0.596$: sixty percent of diffuse light inside a glass layer is reflected back at the interface.

9. Microfacet theory

9.1 The microsurface model

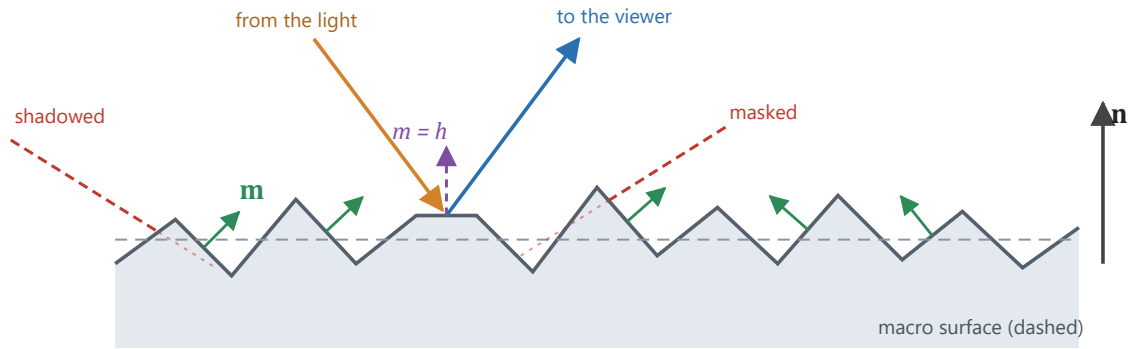


Figure 10. A rough surface as a collection of perfectly smooth microfacets with normals $\mathbf{m}$ (green). Light from the light is reflected to the viewer only by microfacets whose normal equals the half vector $\mathbf{h}$. Some microfacets are hidden from the viewer (masked) or from the light (shadowed) by neighbouring ones.

A rough surface is modelled as a height field of tiny, perfectly smooth facets — much smaller than a pixel, yet much larger than the wavelength of light, so that geometric optics applies to each facet. Each microfacet reflects and refracts like a smooth interface (Section 8). The microsurface is described statistically by its *normal distribution function* $D(\mathbf{m})$: the microfacet area whose normals lie in a small solid angle $d\omega_m$ around $\mathbf{m}$ is $D(\mathbf{m}) d\omega_m dA$ per macro-surface area dA . Since the microfacets, projected onto the macro surface, must exactly cover it,

$$\int_{\mathcal{H}^2} D(\mathbf{m}) (\mathbf{m} \cdot \mathbf{n}) d\omega_m = 1. \quad (9.1)$$

Two further ingredients describe which microfacets participate in a given pair of directions: *masking* (hidden from the viewer) and *shadowing* (hidden from the light). The standard microfacet BSDF accounts for exactly one reflection or refraction per microfacet. Light that bounces between microfacets before escaping is ignored — the source of the energy loss of rough surfaces that Section 10.4 compensates.

9.2 The GGX distribution

The renderer uses the GGX distribution (Trowbridge & Reitz 1975; Walter et al. 2007):

$$D(\mathbf{m}) = \frac{\alpha^2}{\pi ((\mathbf{m} \cdot \mathbf{n})^2 (\alpha^2 - 1) + 1)^2} = \frac{1}{\pi \alpha^2 \cos^4 \theta_m (1 + \tan^2 \theta_m / \alpha^2)^2} \quad (\mathbf{m} \cdot \mathbf{n} > 0). \quad (9.2)$$

The width parameter α controls the spread of normals. At $\alpha = 1$, $D = 1/\pi$ is constant; as $\alpha \rightarrow 0$, D becomes a spike around $\mathbf{n}$ and the surface a mirror. Compared with the older Beckmann distribution, GGX has long tails: highlights have a bright core and a wide soft halo, which matches measured materials well.

The material parameter is the *roughness* $r \in [0, 1]$ with $\alpha = r^2$. This remapping (Burley 2012) makes equal steps in r look like roughly equal steps in appearance. The shader clamps $\alpha \geq 10^{-3}$ (`MIN_ALPHA`): even a "perfect" mirror is a very narrow GGX lobe, so no special code path for delta distributions is needed. The densities involved become large (of order $1/(\pi \alpha^2) \approx 3 \times 10^5$), but they appear in the numerator and denominator of every sample weight and cancel.

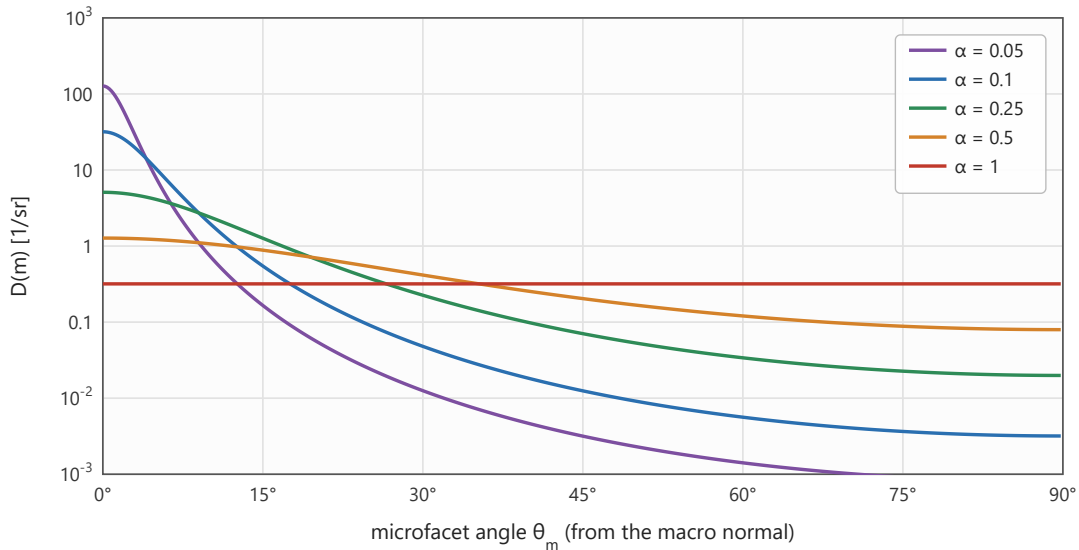


Figure 11. GGX normal distribution D as a function of the microfacet angle, for five widths α (logarithmic scale). Smooth surfaces concentrate the normals near $\theta_m = 0$; at $\alpha = 1$ the distribution is constant.

bsdf.glsl · GgxD

```
float GgxD(vec3 m, float alpha)
{
    float a2 = alpha * alpha;
    float d = m.z * m.z * (a2 - 1.0) + 1.0;
    return a2 / (PI * d * d);
}
```

9.3 Masking and shadowing: the Smith model

The masking function $G_1(\omega, \mathbf{m})$ is the fraction of microfacets with normal $\mathbf{m}$ that are visible from direction ω . The Smith model assumes that the height of a microfacet is uncorrelated with its normal. Under that assumption masking depends on $\mathbf{m}$ only through the back-facing test, and

$$G_1(\omega, \mathbf{m}) = \frac{\chi^+(\omega \cdot \mathbf{m})}{1 + \Lambda(\omega)}, \quad (9.3)$$

where $\chi^+(x)$ is 1 for $x > 0$ and 0 otherwise, and Λ depends on the slope distribution. For GGX,

$$\Lambda(\omega) = \frac{-1 + \sqrt{1 + \alpha^2 \tan^2 \theta}}{2}. \quad (9.4)$$

Λ is 0 at normal incidence (nothing is masked) and grows without bound toward grazing directions.

Both directions are needed at once for reflection: a microfacet must be visible from the viewer *and* from the light. Low-lying microfacets tend to be both masked and shadowed, so the two events are correlated. The *height-correlated* masking-shadowing function (Heitz 2014) accounts for that:

$$G_2(\omega_o, \omega_i, \mathbf{m}) = \frac{\chi^+(\omega_o \cdot \mathbf{m}) \chi^+(\omega_i \cdot \mathbf{m})}{1 + \Lambda(\omega_o) + \Lambda(\omega_i)}. \quad (9.5)$$

The simpler product $G_1(\omega_o)G_1(\omega_i)$ treats the events as independent and darkens rough surfaces too much. The renderer uses (9.5) for reflection and, following PBRT-v4, also for transmission; Heitz derives a slightly different exact form for transmission, which differs little in practice.

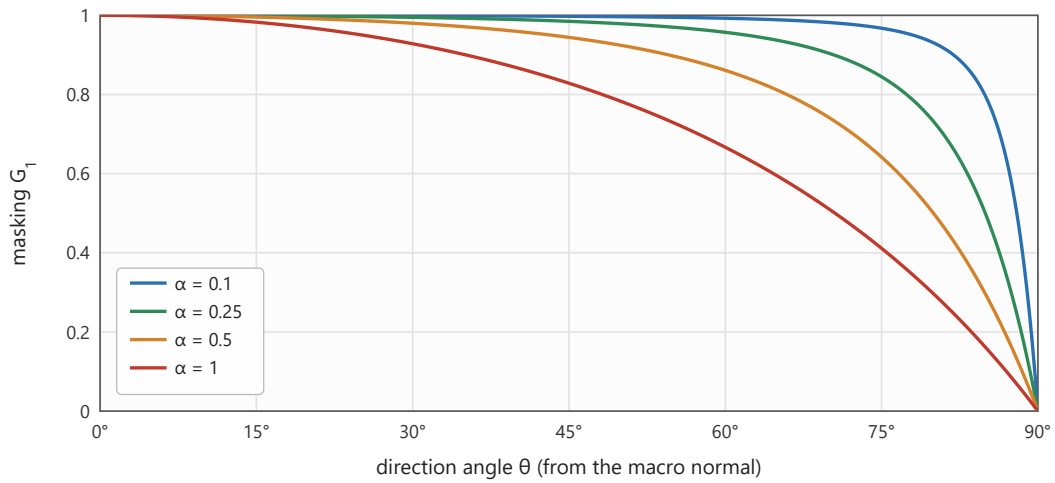


Figure 12. Smith masking G_1 for GGX. Rough surfaces hide a noticeable fraction of their microfacets even at moderate angles; all curves drop to zero at grazing incidence.

The Smith masking function satisfies an important identity: the projected area of all *visible* microfacets, seen from ω , equals the projected area of the macro surface,

$$\int_{\mathcal{H}^2} G_1(\omega, \mathbf{m}) \max(0, \omega \cdot \mathbf{m}) D(\mathbf{m}) d\omega_m = \cos \theta_\omega. \quad (9.6)$$

This identity is what determines Λ for a given D , and it makes the distribution of visible normals (Section 9.6) a proper probability density.

bsdf.glsl · GgxLambda

```
float GgxLambda(vec3 w, float alpha)
{
    float cos2 = max(w.z * w.z, 1e-12);
    float tan2 = max(0.0, 1.0 - cos2) / cos2;
    return 0.5 * (-1.0 + sqrt(1.0 + alpha * alpha * tan2));
}
```

bsdf.glsl · GgxG1

```
float GgxG1(vec3 w, float alpha)
{
    return 1.0 / (1.0 + GgxLambda(w, alpha));
}
```

bsdf.glsl · GgxG2

```
float GgxG2(vec3 wo, vec3 wi, float alpha)
{
    return 1.0 / (1.0 + GgxLambda(wo, alpha) + GgxLambda(wi, alpha));
}
```

9.4 The microfacet reflection BRDF

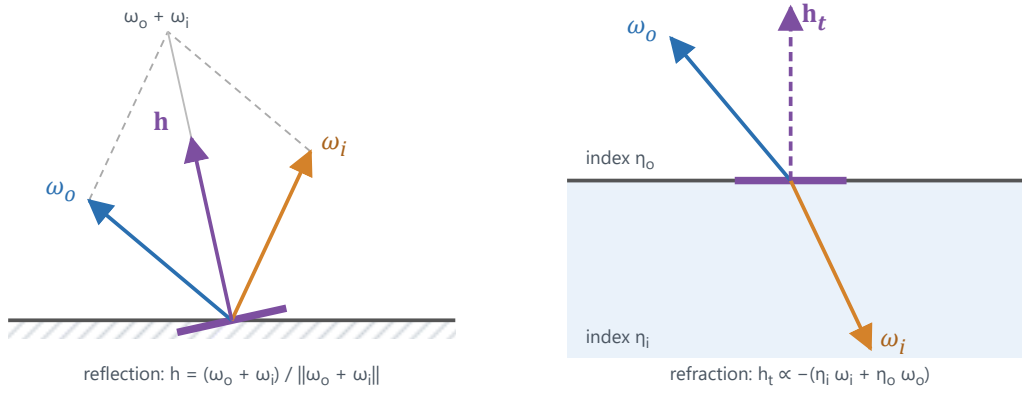


Figure 13. Left: for reflection, only microfacets whose normal equals the half vector $\mathbf{h}$ reflect ω_o into ω_i . Right: for refraction, the generalised half vector $\mathbf{h}_t$ plays the same role; a microfacet with that normal refracts one direction exactly into the other.

For a pair of directions on the same side, only the microfacets whose normal is the half vector

$$\mathbf{h} = \frac{\omega_o + \omega_i}{\|\omega_o + \omega_i\|} \quad (9.7)$$

reflect light from ω_i into ω_o . Counting the flux they receive and reflect yields the Torrance–Sparrow microfacet BRDF:

$$f_r(\omega_o, \omega_i) = \frac{F(\omega_o \cdot \mathbf{h}) D(\mathbf{h}) G_2(\omega_o, \omega_i, \mathbf{h})}{4 |\cos \theta_o| |\cos \theta_i|}. \quad (9.8)$$

DERIVATION

Take a macro patch dA lit by radiance L_i from a cone $d\omega_i$. Microfacets with normals in $d\omega_m$ around $\mathbf{m} = \mathbf{h}$ have area $D(\mathbf{m}) d\omega_m dA$; they receive the irradiance $L_i (\omega_i \cdot \mathbf{m}) d\omega_i$, reduced by shadowing, and reflect the fraction F . The reflected flux is

$$d\Phi_r = F L_i (\omega_i \cdot \mathbf{m}) d\omega_i D(\mathbf{m}) d\omega_m dA G_2.$$

(The masking part of G_2 removes the flux that is blocked on the way to the viewer.) This flux leaves into the solid angle $d\omega_o$ of directions that mirror ω_i about the normals in $d\omega_m$. The reflection map doubles angles, which gives the Jacobian $d\omega_o = 4(\omega_i \cdot \mathbf{m}) d\omega_m$ (Appendix D.1). The reflected radiance is flux per projected area and solid angle:

$$dL_o = \frac{d\Phi_r}{dA \cos \theta_o d\omega_o} = \frac{F D G_2 L_i d\omega_i}{4 \cos \theta_o}.$$

Dividing by the incident irradiance $L_i \cos \theta_i d\omega_i$ gives (9.8).

9.5 The microfacet transmission BTDF

For a pair of directions on opposite sides, a microfacet transmits ω_i into ω_o only if its normal is the generalised half vector (Walter et al. 2007)

$$\mathbf{h}_t = -\frac{\eta_i \omega_i + \eta_o \omega_o}{\|\eta_i \omega_i + \eta_o \omega_o\|}, \quad (9.9)$$

where η_o and η_i are the refractive indices on the sides of ω_o and ω_i (the sign is irrelevant; the code flips it into the upper hemisphere). Repeating the flux argument with the refraction Jacobian (Appendix D.2) gives

$$f_t(\omega_o, \omega_i) = \frac{|\omega_i \cdot \mathbf{h}_t| |\omega_o \cdot \mathbf{h}_t|}{|\cos \theta_i| |\cos \theta_o|} \frac{(1 - F(\omega_o \cdot \mathbf{h}_t)) D(\mathbf{h}_t) G_2(\omega_o, \omega_i, \mathbf{h}_t)}{(\omega_i \cdot \mathbf{h}_t + \omega_o \cdot \mathbf{h}_t / \eta')^2} \frac{1}{\eta'^2}, \quad \eta' = \frac{\eta_i}{\eta_o}. \quad (9.10)$$

The last factor is the radiance scaling of Section 2.3. Light that crosses from the side of ω_i (index η_i) to the side of ω_o (index η_o) has its solid angle expanded or compressed, and its radiance changes by $(\eta_o/\eta_i)^2$. When the camera looks into glass from air, $\eta' = \eta$ and the radiance coming out is reduced by $1/\eta^2$; when a path inside glass looks out, $\eta' = 1/\eta$ and it is increased. Renderers that trace light from the light sources ("importance transport") must omit this factor. This renderer only traces from the camera, so the factor is always applied.

IN THE CODE · EvalDielectric

1. Reflection if $\cos \theta_o \cos \theta_i > 0$, else transmission.
2. η' (etap) is 1 for reflection, η if ω_o is outside, $1/\eta$ if ω_o is inside.
3. $\mathbf{wm} = \text{normalize}(\mathbf{wi} * \text{etap} + \mathbf{wo})$ is the half vector (9.7) for reflection and $-\mathbf{h}_t$ of (9.9) for transmission (up to the common factor η_o); it is flipped to $z > 0$.
4. Configurations in which $\mathbf{m}$ faces away from either direction are rejected: no microfacet can produce them.
5. The Fresnel term is evaluated at the microfacet, $\text{FresnelDielectric}(\text{dot}(\mathbf{wo}, \mathbf{wm}), \text{eta})$.
6. Reflection returns (9.8); transmission returns (9.10) multiplied by the tint (Section 10.7). The pdf includes the probability F or $1 - F$ with which SampleDielectric chooses the branch.

bsdf.glsl · EvalDielectric

```
vec3 EvalDielectric(vec3 wo, vec3 wi, float alpha, float eta, vec3 tint, out float pdf)
{
    pdf = 0.0;
    float cosO = wo.z;
    float cosI = wi.z;
    bool reflection = cosO * cosI > 0.0;
    float etap = reflection ? 1.0 : (cosO > 0.0 ? eta : 1.0 / eta);
    vec3 wm = wi * etap + wo;
    if (cosO == 0.0 || cosI == 0.0 || dot(wm, wm) == 0.0)
        return vec3(0.0);
    wm = normalize(wm);
    if (wm.z < 0.0)
        wm = -wm;
    if (dot(wm, wi) * cosI < 0.0 || dot(wm, wo) * cosO < 0.0)
        return vec3(0.0);

    float F = FresnelDielectric(dot(wo, wm), eta);
    if (reflection)
    {
        pdf = GgxVisiblePdf(wo, wm, alpha) / (4.0 * abs(dot(wo, wm))) * F;
        return vec3(GgxD(wm, alpha) * GgxG2(wo, wi, alpha) * F / abs(4.0 * cosI * cosO));
    }

    float denom = dot(wi, wm) + dot(wo, wm) / etap;
    denom *= denom;
    pdf = GgxVisiblePdf(wo, wm, alpha) * abs(dot(wi, wm)) / denom * (1.0 - F);
    float ft = GgxD(wm, alpha) * (1.0 - F) * GgxG2(wo, wi, alpha) * abs(dot(wi, wm) * dot(wo, wm) / (cosI * cosO * denom));
    return tint * (ft / (etap * etap));
}
```

9.6 The distribution of visible normals

Not all microfacets are seen from a direction ω_o : some face away, some are masked. The normals that a ray from ω_o actually hits follow the *distribution of visible normals* (Heitz & d'Eon 2014)

$$D_{\omega_o}(\mathbf{m}) = \frac{G_1(\omega_o, \mathbf{m}) \max(0, \omega_o \cdot \mathbf{m}) D(\mathbf{m})}{\cos \theta_o}. \quad (9.11)$$

By the identity (9.6) it integrates to one, so it is a probability density over $\mathbf{m}$. Sampling microfacet normals from D_{ω_o} — instead of from $D(\mathbf{m}) \cos \theta_m$, as in the older approach of Walter et al. — has two advantages: it never produces back-facing microfacets (wasted samples), and it makes the sample weights bounded (Section 9.8).

9.7 Sampling visible normals

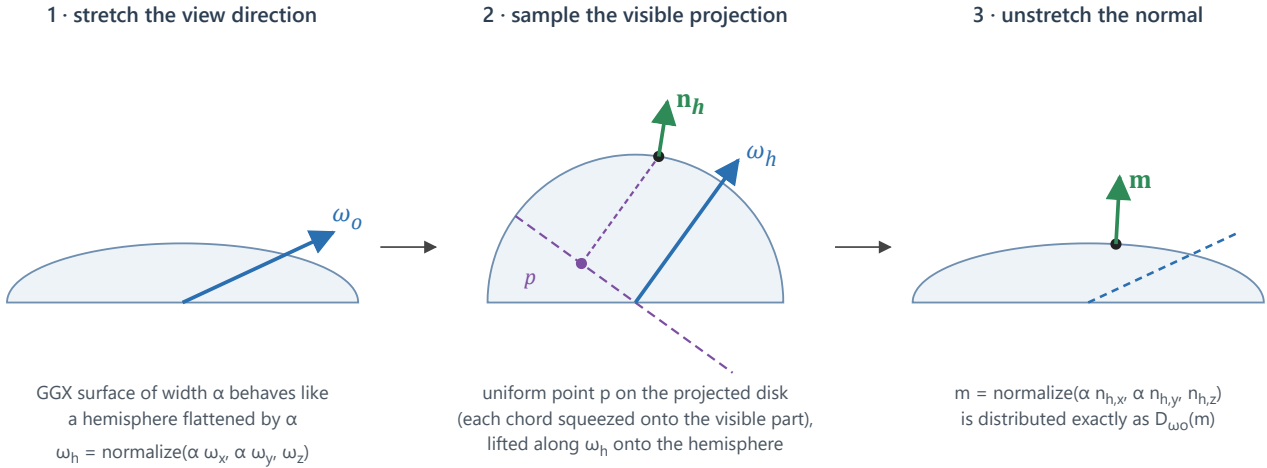


Figure 14. Visible-normal sampling (Heitz 2018) in three steps. Stretching turns the GGX configuration into a hemisphere; the visible normals of a hemisphere are uniformly distributed over its projected area, which is easy to sample; the sampled hemisphere normal is transformed back to the GGX configuration.

Heitz (2018) found an exact and cheap sampling algorithm for D_{ω_o} based on a geometric fact: the GGX distribution of width α is the distribution of normals of a hemisphere flattened by the factor α along z (a half ellipsoid). Linear transformations map such a configuration to the unit hemisphere, whose visible normals are trivial to sample:

1. **Stretch.** Directions transform like vectors, so ω_o becomes $\omega_h = \text{normalize}(\alpha \omega_x, \alpha \omega_y, \omega_z)$ in the hemisphere configuration. If ω_h points below the surface — ω_o inside glass — it is negated; see the note below.
2. **Sample the projected area.** Seen from ω_h , the unit hemisphere projects onto a half disk (the dome) plus a half ellipse with semi-minor axis $\omega_{h,z}$ (the rim). Its visible normals are distributed uniformly over this projected region. The algorithm samples a point (p_x, p_y) uniformly on the unit disk (`SampleUniformDisk`) and linearly remaps every vertical chord of the disk, $[-h, h]$ with $h = \sqrt{1 - p_x^2}$, onto the visible chord $[-\omega_{h,z}h, h]$. The map has a constant Jacobian, so the point stays uniformly distributed over the visible region:

$$p_y \leftarrow (1 - s)h + s p_y, \quad s = \frac{1}{2}(1 + \omega_{h,z}).$$
3. **Lift.** The point is projected along ω_h onto the hemisphere, $\mathbf{n}_h = p_x \mathbf{T}_1 + p_y \mathbf{T}_2 + \sqrt{1 - p_x^2 - p_y^2} \omega_h$, where $\mathbf{T}_1, \mathbf{T}_2$ complete ω_h to an orthonormal frame. The hemisphere normal at that point is $\mathbf{n}_h$ itself.
4. **Unstretch.** Normals transform with the inverse transpose of the stretch (Section 6.3), which again multiplies x and y by α : $\mathbf{m} = \text{normalize}(\alpha n_{h,x}, \alpha n_{h,y}, \max(10^{-6}, n_{h,z}))$.

The resulting $\mathbf{m}$ has exactly the density $D_{\omega_o}(\mathbf{m})$.

NOTE · DIRECTIONS FROM BELOW

When ω_o lies below the macro surface (a path inside glass), the algorithm samples the visible normals for $-\omega_o$. The microfacets visible from $-\omega_o$ are exactly those whose back side is seen from ω_o below the surface, the ones that can refract or reflect light arriving from inside. `GgxVisiblePdf` uses absolute values of $\cos \theta_o$ and $\omega_o \cdot \mathbf{m}$, consistent with this choice, and `EvalDielectric` rejects microfacets facing the wrong way. This is the convention of PBRT-v4.

bsdf.glsl · SampleGgxVisibleNormal

```
vec3 SampleGgxVisibleNormal(vec3 w, float alpha, vec2 u)
{
    vec3 wh = normalize(vec3(alpha * w.x, alpha * w.y, w.z));
    if (wh.z < 0.0)
        wh = -wh;
    vec3 t1 = wh.z < 0.99999 ? normalize(cross(vec3(0.0, 0.0, 1.0), wh)) : vec3(1.0, 0.0, 0.0);
    vec3 t2 = cross(wh, t1);
    vec2 p = SampleUniformDisk(u);
    float h = sqrt(max(0.0, 1.0 - p.x * p.x));
    p.y = mix(h, p.y, (1.0 + wh.z) * 0.5);
    float pz = sqrt(max(0.0, 1.0 - dot(p, p)));
    vec3 nh = p.x * t1 + p.y * t2 + pz * wh;
    return normalize(vec3(alpha * nh.x, alpha * nh.y, max(1e-6, nh.z)));
}
```

9.8 Densities and sample weights

Reflecting ω_o about a sampled visible normal gives $\omega_i = 2(\omega_o \cdot \mathbf{m})\mathbf{m} - \omega_o$. By the half-vector Jacobian, the density of ω_i is

$$p_r(\omega_i) = \frac{D_{\omega_o}(\mathbf{h})}{4|\omega_o \cdot \mathbf{h}|} = \frac{G_1(\omega_o) D(\mathbf{h})}{4|\cos \theta_o|}. \quad (9.12)$$

With the BRDF (9.8), the weight of such a sample is

$$\frac{f_r |\cos \theta_i|}{p_r} = \frac{F D G_2 / (4|\cos \theta_o| |\cos \theta_i|) \cdot |\cos \theta_i|}{G_1(\omega_o) D / (4|\cos \theta_o|)} = F(\omega_o \cdot \mathbf{h}) \frac{G_2(\omega_o, \omega_i)}{G_1(\omega_o)} \leq F. \quad (9.13)$$

KEY IDEA

With visible-normal sampling, the normal distribution D cancels completely and the sample weight is bounded by the Fresnel term. No matter how sharp the lobe is, a reflection sample can never carry an exploding weight. This is why the renderer can treat mirrors as GGX with $\alpha = 10^{-3}$ without producing fireflies.

For refraction through a sampled normal, the Jacobian (Appendix D.2) gives

$$p_t(\omega_i) = D_{\omega_o}(\mathbf{h}_t) \frac{|\omega_i \cdot \mathbf{h}_t|}{(\omega_i \cdot \mathbf{h}_t + \omega_o \cdot \mathbf{h}_t / \eta')^2}, \quad (9.14)$$

and the weight is again G_2/G_1 times the transmitted fraction and the radiance factor $1/\eta'^2$. `SampleDielectric` chooses between reflection and refraction with probabilities F and $1 - F$, evaluated at the sampled microfacet. The Fresnel factors cancel from the weights, leaving G_2/G_1 for reflection and $G_2/G_1 \cdot \text{tint}/\eta'^2$ for transmission.

bsdf.glsl · GgxVisiblePdf

```
float GgxVisiblePdf(vec3 w, vec3 m, float alpha)
{
    return GgxG1(w, alpha) / max(abs(w.z), 1e-8) * GgxD(m, alpha) * abs(dot(w, m));
}
```

bsdf.glsl · SampleDielectric

```
bool SampleDielectric(vec3 wo, float alpha, float eta, vec3 u, out vec3 wi)
{
    vec3 wm = SampleGgxVisibleNormal(wo, alpha, u.xy);
    float F = FresnelDielectric(dot(wo, wm), eta);
    if (u.z < F)
    {
        wi = reflect(-wo, wm);
        return wo.z * wi.z > 0.0;
    }
    return Refract(wo, wm, eta, wi) && wo.z * wi.z < 0.0;
}
```

10. The material model

10.1 Parameters

Every surface uses the same parametric material, in the spirit of the Disney "principled" BSDF (Burley 2012) and its descendants. A material is a blend of a small number of physically motivated lobes, controlled by artist-friendly parameters:

Parameter	Range	Meaning
BaseColor ρ	$[0, 1]^3$	diffuse albedo of dielectrics, reflectance F_0 of metals, transmission tint of glass; multiplied by the texture
Metallic m	$[0, 1]$	blend between dielectric (0) and conductor (1)
Roughness r	$[0, 1]$	microfacet roughness of metal, specular and glass lobes; $\alpha = \max(r^2, 10^{-3})$
Transmission t	$[0, 1]$	blend between opaque dielectric (0) and glass (1)
IOR η	≥ 1 (clamped to 1.01)	relative refractive index of the dielectric base and of glass
Clearcoat c	$[0, 1]$	strength of a clear dielectric coat (IOR 1.5) on top of everything
ClearcoatRoughness r_c	$[0, 1]$	roughness of the coat, $\alpha_c = \max(r_c^2, 10^{-3})$
Sheen C_s	RGB	colour of the grazing-angle sheen of cloth-like surfaces
Emission L_e	RGB, unbounded	emitted radiance of the front face
Texture	id	sRGB texture that multiplies BaseColor (texture 0 is white)
Medium	id	participating medium that fills the inside of the (closed) mesh (Part V)
NullSurface	bool	the surface is an invisible boundary that only changes the medium (Section 22)

On the GPU a material is an 80-byte record. `FetchMaterial` reads it, applies the texture (sampled at mip level 0, hardware sRGB decoding gives linear values), and fills the `SurfaceMaterial` struct from which the BSDF is built.

scene.glsl · struct Material

```
struct Material
{
    vec3 baseColor;
    float metallic;
    vec3 emission;
    float roughness;
    vec3 sheen;
    float transmission;
    float ior;
    float clearcoat;
    float clearcoatRoughness;
    uint textureIndex;
    int mediumIndex;
    uint flags;
    uint pad0;
    uint pad1;
};
```

raygen.rgen · FetchMaterial

```

SurfaceMaterial FetchMaterial(uint index, vec2 uv)
{
    Material m = materials[index];
    SurfaceMaterial material;
    material.baseColor = m.baseColor * textureLod(textures[nonuniformEXT(m.textureIndex)], uv, 0.0).rgb;
    material.metallic = m.metallic;
    material.emission = m.emission;
    material.roughness = m.roughness;
    material.sheen = m.sheen;
    material.transmission = m.transmission;
    material.ior = m.ior;
    material.clearcoat = m.clearcoat;
    material.clearcoatRoughness = m.clearcoatRoughness;
    material.medium = m.mediumIndex;
    material.nullSurface = (m.flags & MATERIAL_NULL_SURFACE) != 0u;
    return material;
}

```

10.2 Structure: lobes and weights

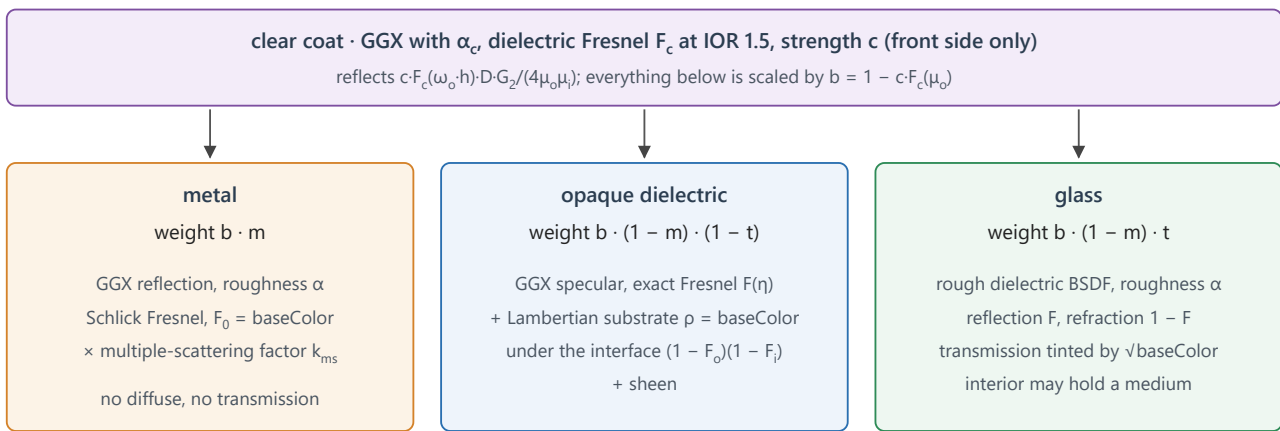


Figure 15. Structure of the material. An optional clear coat sits on top; the base is a blend of three lobes selected by `Metallic` (m) and `Transmission` (t). All base weights share the factor b , the part of the light not reflected by the coat.

Let $\mu_o = |\cos \theta_o|$, $\mu_i = |\cos \theta_i|$, and let c be the clear-coat strength, set to 0 when ω_o is on the back side. The lobes are weighted by

$$b = 1 - c F(\mu_o; 1.5), \quad w_{\text{metal}} = b m, \quad w_{\text{glass}} = b (1 - m) t, \quad w_{\text{opaque}} = b (1 - m)(1 - t). \quad (10.1)$$

KEY IDEA · THE COMPLETE BSDF

$$\begin{aligned}
 f(\omega_o, \omega_i) = & c \frac{F(\omega_o \cdot \mathbf{h}; 1.5) D(\mathbf{h}; \alpha_c) G_2(\omega_o, \omega_i; \alpha_c)}{4 \mu_o \mu_i} \\
 & + w_{\text{metal}} k_{\text{ms}}(\mu_o) \frac{F_{\text{Schlick}}(\rho, \omega_o \cdot \mathbf{h}) D(\mathbf{h}; \alpha) G_2(\omega_o, \omega_i; \alpha)}{4 \mu_o \mu_i} \\
 & + w_{\text{opaque}} \left[\frac{F(\omega_o \cdot \mathbf{h}; \eta) D(\mathbf{h}; \alpha) G_2(\omega_o, \omega_i; \alpha)}{4 \mu_o \mu_i} + \frac{\rho (1 - F(\mu_o; \eta)) (1 - F(\mu_i; \eta))}{\pi (1 - F_{\text{dr}}(\eta))} + C_s (1 - \omega_i \cdot \mathbf{h})^5 \right] \\
 & + w_{\text{glass}} f_{\text{dielectric}}(\omega_o, \omega_i; \alpha, \eta, \sqrt{\rho})
 \end{aligned}$$

The first three lines are reflection lobes: they are zero unless ω_o and ω_i lie on the same side, and are evaluated with both directions mirrored into the upper hemisphere (Section 10.8). The last line is the full rough dielectric BSDF of Sections 9.4–9.5 (reflection and refraction). $k_{\text{ms}} = 1 + \rho (1/E(\mu_o, \alpha) - 1)$ is the metal's multiple-scattering compensation (Section 10.4).

The weights of (10.1) make the blend energy-consistent: the coat takes its Fresnel share cF_c , the base gets the remainder b , and `Metallic` and `Transmission` split b linearly. Intermediate values of m and t are rarely physical (a surface is either metal or not), but they are useful for texture-driven transitions and anti-aliased material boundaries.

10.3 The clear coat

A clear coat is a thin, transparent, non-absorbing dielectric film — car paint lacquer, varnish on wood, the finish of the showcase floor. It reflects with a standard GGX lobe of its own roughness α_c and the exact Fresnel term of IOR 1.5 ($F_0 = 0.04$). The light it does not reflect reaches the base.

A rigorous layered model would attenuate the base by the coat's transmittance on the way in *and* out, refract both directions into the layer, and add the inter-reflections between coat and base (Weidlich & Wilkie 2007; stochastic evaluation: Guo et al. 2018). The renderer uses the common single-scattering approximation: the base is scaled by $b = 1 - cF(\mu_o; 1.5)$, the transmittance of a smooth coat in the view direction.

CAVEAT · RECIPROCITY

Because b depends only on ω_o , the coated BSDF is not reciprocal. That is harmless here: the path tracer always evaluates the BSDF with ω_o toward the camera, both for BSDF sampling and for light sampling, so all estimators see the same function and remain consistent. The furnace test (Section 12) shows no energy gain: a white clear-coated diffuse sphere measures 1.00. The coat is only applied on the front side; seen from behind, as on the back of a single-sided plane, a surface has no coat.

10.4 The metal lobe and multiple-scattering compensation

Conductors reflect at the surface and absorb whatever is transmitted within a few nanometres, so the metal lobe consists of GGX reflection only, with Schlick's Fresnel and $F_0 = \rho$ (Section 8.5). The single-scattering microfacet model, however, loses energy. Its directional albedo for a white conductor ($F = 1$),

$$E(\mu_o, \alpha) = \int_{\mathcal{H}^2} \frac{D G_2}{4\mu_o\mu_i} \mu_i d\omega_i, \quad (10.2)$$

is less than one, because light that is reflected toward another microfacet instead of escaping is simply discarded by $G_2 < G_1$ (and so are samples reflected below the horizon). In reality that light scatters again and mostly escapes. The loss grows quickly with roughness: at $r = 1$ more than two thirds of the energy at normal incidence is lost, and a rough gold sphere would look much darker than a polished one.

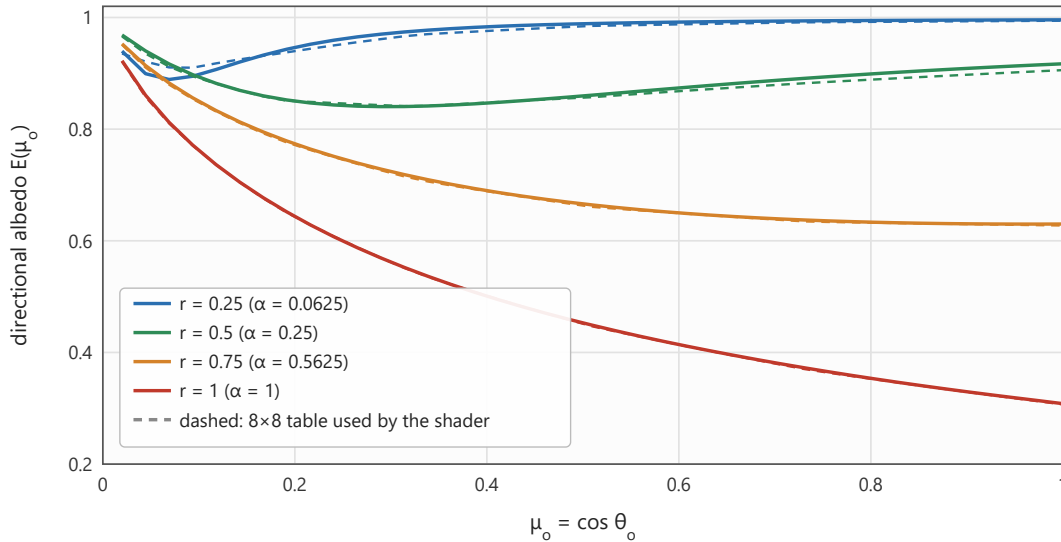


Figure 16. Directional albedo $E(\mu_o)$ of single-scattering GGX with $F = 1$, computed by Monte Carlo integration of the shader's own functions (solid), and the 8×8 table used by the shader (dashed). Everything below 1 is energy that single scattering loses.

Turquin (2019) proposes a simple compensation. Assume that the light which scatters more than once leaves with the same angular distribution as singly scattered light. For a white conductor the missing energy is $1 - E(\mu_o)$, so scaling the lobe by $1/E(\mu_o)$ restores an albedo of exactly one. For a coloured conductor, each additional bounce is attenuated by roughly F_0 , so only the F_0 -weighted part of the missing energy is added back:

$$f_{\text{ms}} = f_{\text{ss}} k_{\text{ms}}, \quad k_{\text{ms}} = 1 + F_0 \frac{1 - E(\mu_o, \alpha)}{E(\mu_o, \alpha)} = 1 + F_0 \left(\frac{1}{E(\mu_o, \alpha)} - 1 \right). \quad (10.3)$$

The factor is exact for $F_0 = 1$ in the white furnace and vanishes when $F_0 \rightarrow 0$, where no multiply scattered light survives anyway. Kulla & Conty (2017) derive a reciprocal alternative with a separate lobe and a more accurate colour term; (10.3) is cheaper and sufficient for a camera path tracer.

IN THE CODE · THE ALBEDO TABLE

$E(\mu, \alpha)$ was precomputed with 2×10^6 visible-normal samples per entry, using a C# port of the shader's GGX functions, and stored as an 8×8 table `GGX_ALBEDO`. Rows are roughness $r = j/7$; columns are $\sqrt{\mu} = i/7$. The square root places more columns near grazing angles, where E changes fastest. `GgxAlbedo` interpolates bilinearly. The mean relative error of the interpolated $1/E$ is 0.37%, the maximum about 3%, at extreme grazing angles on smooth surfaces. `CreateBsdf` stores k_{ms} per colour channel in `metalScale`.

bsdf.glsl · const GGX_ALBEDO

```
const float GGX_ALBEDO[64] = float[(
    0.9796, 0.9987, 0.9999, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
    0.9990, 0.8918, 0.9643, 0.9924, 0.9976, 0.9989, 0.9994, 0.9996,
    0.9997, 0.9490, 0.8864, 0.9164, 0.9595, 0.9803, 0.9887, 0.9924,
    0.9997, 0.9666, 0.9028, 0.8637, 0.8756, 0.9099, 0.9387, 0.9566,
    0.9997, 0.9649, 0.9007, 0.8396, 0.8051, 0.8046, 0.8265, 0.8548,
    0.9995, 0.9545, 0.8759, 0.7966, 0.7323, 0.6911, 0.6743, 0.6779,
    0.9993, 0.9390, 0.8366, 0.7330, 0.6423, 0.5692, 0.5142, 0.4758,
    0.9991, 0.9202, 0.7892, 0.6579, 0.5425, 0.4466, 0.3690, 0.3069);
```

bsdf.glsl · GgxAlbedo

```
float GgxAlbedo(float cosTheta, float alpha)
{
    vec2 p = vec2(sqrt(clamp(cosTheta, 0.0, 1.0)), min(sqrt(alpha), 1.0)) * 7.0;
    ivec2 i = min(ivec2(p), ivec2(6));
    vec2 t = p - vec2(i);
    int row = i.y * 8 + i.x;
    float e0 = mix(GGX_ALBEDO[row], GGX_ALBEDO[row + 1], t.x);
    float e1 = mix(GGX_ALBEDO[row + 8], GGX_ALBEDO[row + 9], t.x);
    return mix(e0, e1, t.y);
}
```

10.5 The opaque dielectric: specular reflection over a diffuse substrate

Plastic, paint, stone, wood and most everyday non-metals are modelled as a dielectric interface (IOR η , roughness α) on top of a Lambertian substrate with albedo $\rho = \text{BaseColor}$. The interface contributes a microfacet reflection lobe with the exact dielectric Fresnel term (9.8). The diffuse part needs more care, because light that enters the layer can be reflected at the interface from below and return to the substrate several times before it escapes.

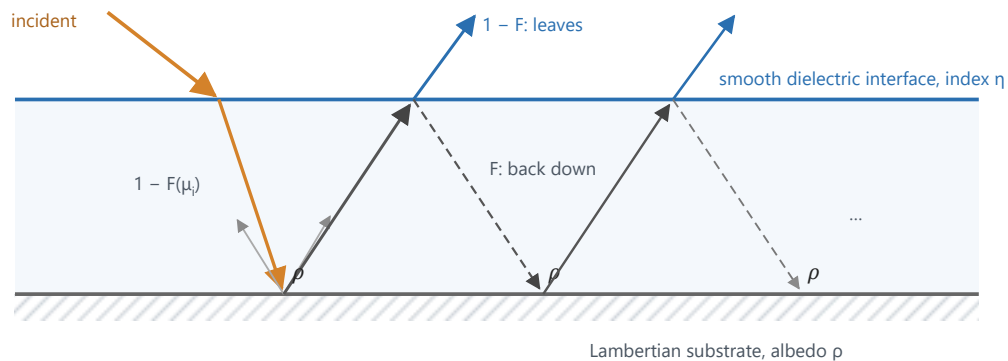


Figure 17. Light enters the layer, is reflected diffusely by the substrate, and at the interface partly escapes and partly returns to the substrate — a geometric series of internal bounces.

DERIVATION · DIFFUSE SUBSTRATE UNDER A SMOOTH INTERFACE

Light from ω_i enters the layer with the fraction $1 - F(\mu_i)$; the transmitted irradiance on the substrate is $E_t = (1 - F(\mu_i)) L_i \cos \theta_i d\omega_i$. The substrate reflects diffusely. Of this diffuse upward flux the interface reflects the fraction F_{dr}^{int} back down (Section 8.6), where it is reflected again with albedo ρ , and so on. The total exitance M of the substrate therefore satisfies

$$M = \rho (E_t + F_{dr}^{int} M) \implies M = \frac{\rho E_t}{1 - \rho F_{dr}^{int}}.$$

The radiance leaving the substrate is M/π (Lambertian). Crossing the interface toward ω_o multiplies it by the transmittance $1 - F(\mu_o)$ and by $1/\eta^2$, because it passes from the dense layer into air (Section 2.3). Dividing by the incident irradiance gives

$$f_d^{exact} = \frac{\rho (1 - F(\mu_i)) (1 - F(\mu_o))}{\pi \eta^2 (1 - \rho F_{dr}^{int})}.$$

For $\rho = 1$ the identity (8.6), $\eta^2(1 - F_{dr}^{int}) = 1 - F_{dr}^{ext}$, simplifies the denominator. The diffuse albedo becomes

$$\int f_d \mu_i d\omega_i = \frac{1 - F(\mu_o)}{\pi(1 - F_{dr}^{ext})} \int (1 - F(\mu_i)) \mu_i d\omega_i = \frac{1 - F(\mu_o)}{\pi(1 - F_{dr}^{ext})} \pi(1 - F_{dr}^{ext}) = 1 - F(\mu_o),$$

which together with the specular reflection $F(\mu_o)$ of a smooth interface is exactly one: the model conserves energy, and it is reciprocal.

The renderer uses the $\rho = 1$ normalisation for every ρ (the "linear" variant of Mitsuba's `plastic` BSDF):

$$f_d = \frac{\rho (1 - F(\mu_o; \eta)) (1 - F(\mu_i; \eta))}{\pi (1 - F_{dr}(\eta))}. \quad (10.4)$$

With the exact form, a base colour of 0.5 behind glass-like varnish would produce a diffuse albedo of only 0.28 at normal incidence, because every internal bounce is absorbed again by the substrate. This is the physical reason why wet or varnished surfaces look darker and more saturated. The linear form gives 0.48, so `BaseColor` stays close to the colour you actually see, which is what material authors expect. For $\eta = 1.5$ the factor $1/(1 - F_{dr})$ is 1.101.

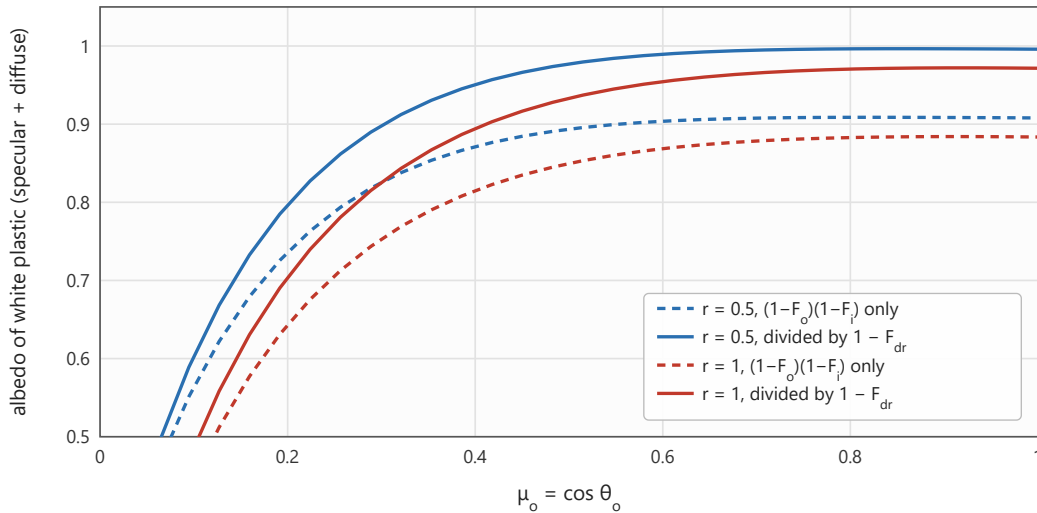


Figure 18. Albedo of white plastic ($\eta = 1.5$) with and without the $1/(1 - F_{dr})$ normalisation. Without it, about 9% of the energy is lost even at normal incidence. The remaining loss of the normalised model comes from the single-scattering specular lobe and from using the macro-surface Fresnel term for the diffuse transmission at grazing angles.

10.6 Sheen

Cloth, velvet and dusty surfaces scatter extra light at grazing angles: fibres standing up from the surface are seen side-on and catch the light. The Disney BSDF models this with an additive sheen term (Burley 2012),

$$f_{sheen} = C_s (1 - \cos \theta_d)^5, \quad \cos \theta_d = \omega_i \cdot \mathbf{h} = \omega_o \cdot \mathbf{h}, \quad (10.5)$$

which vanishes for mirror-like configurations and grows toward grazing ones. It is added to the opaque dielectric lobe and scaled by w_{opaque} . The term is empirical and not energy-conserving — it adds energy instead of redistributing it — so moderate colours should be used. The velvet bunny of the showcase scene combines a dark red base (roughness 1) with white sheen.

10.7 The glass lobe

For transmissive materials the complete rough dielectric BSDF of Sections 9.4–9.5 forms one lobe: microfacet reflection with probability F and refraction with probability $1 - F$, both with the material's roughness and IOR. Rough glass (`Roughness` > 0) blurs the refracted image — the frosted box in the showcase scene. Smooth glass (`Roughness` = 0) refracts sharply.

Transmission is tinted by $\sqrt{\rho}$ at every crossing. A ray that enters and leaves a closed object crosses two interfaces, so a thin object appears tinted by exactly `BaseColor`, independent of its thickness. This is an artistic convenience. Physically coloured glass absorbs inside its volume, so thick parts are more saturated than thin ones; for that behaviour, assign an absorbing medium to the glass (Beer–Lambert, Section 19.3), as in the blue sphere of the showcase scene.

The IOR is clamped to at least 1.01: at $\eta = 1$ the refracted direction is $\omega_i = -\omega_o$ and the generalised half vector $\eta'\omega_i + \omega_o$ degenerates to zero. The glass lobe is evaluated with the true, signed directions, because refraction differs between the two sides: from inside, η becomes $1/\eta$ and total internal reflection appears.

10.8 Two-sided surfaces and light leaks

The reflection lobes are evaluated with both directions mirrored into the upper hemisphere, $\mathbf{o} = (\omega_{o,x}, \omega_{o,y}, |\omega_{o,z}|)$ and likewise for $\mathbf{i}$. A single-sided mesh therefore shades identically from both sides. The walls of the showcase room are single planes, and a ray arriving at their back side behaves as if it arrived at the front.

A second safeguard concerns the difference between shading and geometric normals. Because the shading normal is interpolated, a direction that is a reflection with respect to $\mathbf{n}$ can actually pass through the triangle, and vice versa. Accepting such pairs lets light leak through walls or creates black artefacts (Veach 1997, ch. 5). `EvalBsdf` therefore rejects every pair for which the shading frame and the geometric normal disagree on whether it is a reflection:

$$\text{sign}((\omega_o \cdot \mathbf{n})(\omega_i \cdot \mathbf{n})) \neq \text{sign}((\omega_o \cdot \mathbf{n}_g)(\omega_i \cdot \mathbf{n}_g)) \implies f = 0.$$

This loses a little energy at grazing angles on smooth-shaded meshes, but it never creates light.

10.9 Near-specular materials and light sampling

A random point on a light almost never falls inside a very narrow lobe, so next event estimation would waste a shadow ray and MIS would hand almost all the weight to the BSDF sample anyway. `BsdfHasSmoothLobes` returns false when the material has no diffuse part, $\alpha \leq 0.0025$ ($r \leq 0.05$), and no rough coat. The path loop then skips next event estimation, and an emitter found by the next BSDF sample receives the full weight (Section 14.4). The mirror wall and chrome sphere ($r = 0.02$) and the smooth blue glass sphere ($r = 0$) take this path.

10.10 A worked example: the car paint

The red sphere uses $\rho = (0.7, 0.03, 0.03)$, $r = 0.45$, $c = 1$, $r_c = 0.03$, $\eta = 1.5$, $m = t = 0$. With no metal or glass, $b = 1 - F_c$ and the lobe weights are cF_c (coat), bF_d (specular) and $b(1 - F_d)$ (diffuse), where $F_c = F_d = F(\mu_o; 1.5)$ here. They are also the lobe selection probabilities of Section 11.1:

View angle θ_o	$F(\mu_o; 1.5)$	coat	specular	diffuse
0° (facing the camera)	0.040	0.040	0.038	0.922
78.5° ($\mu_o = 0.2$, near the silhouette)	0.339	0.339	0.224	0.437

At the centre of the sphere almost all samples go to the diffuse lobe; near the silhouette a third of them sample the mirror-like coat, which produces the crisp reflections along the rim.

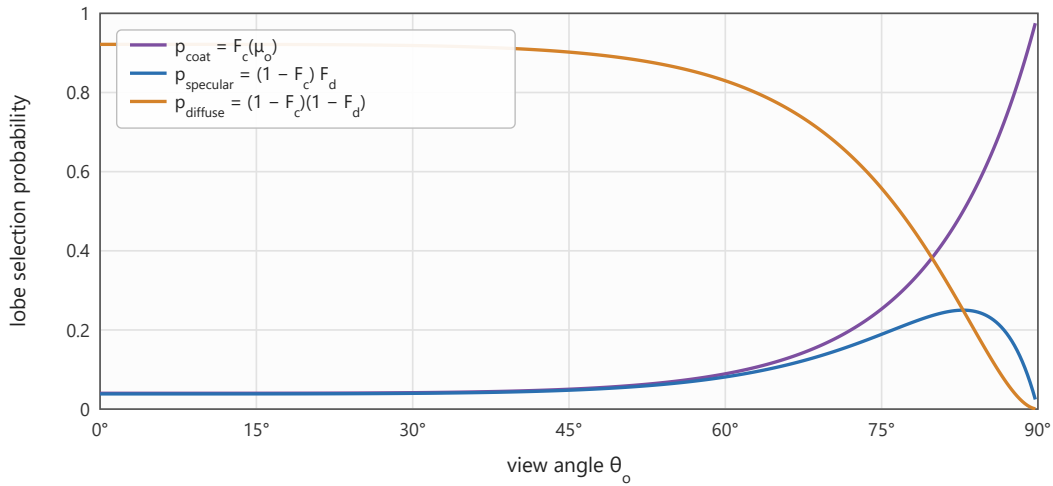


Figure 19. Lobe selection probabilities of the clear-coated car paint as a function of the viewing angle.

11. Sampling the BSDF

11.1 Choosing a lobe

To sample a direction, `SampleBsdf` first chooses one lobe at random, with probabilities that approximate the fraction of energy each lobe reflects in the current view direction:

$$\begin{aligned} p_{\text{coat}} &\propto cF(\mu_o; 1.5), & p_{\text{metal}} &\propto w_{\text{metal}}, & p_{\text{glass}} &\propto w_{\text{glass}}, \\ p_{\text{spec}} &\propto w_{\text{opaque}} F(\mu_o; \eta), & p_{\text{diff}} &\propto w_{\text{opaque}} (1 - F(\mu_o; \eta)). \end{aligned} \quad (11.1)$$

By construction of (10.1) these already sum to $cF_c + b = 1$; the explicit normalisation in `CreateBsdf` only guards against parameters outside $[0, 1]$. A single uniform number ξ is compared against the cumulative sums.

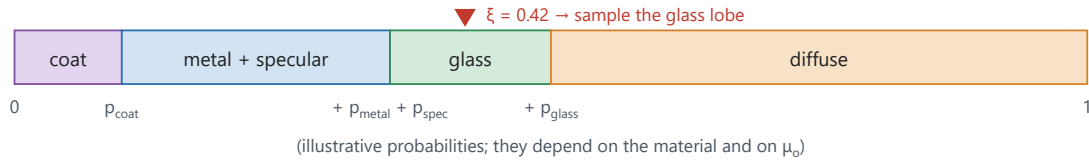


Figure 20. Lobe selection with one uniform random number compared against cumulative probabilities.

11.2 Sampling each lobe

- **Coat, metal and dielectric specular:** sample a visible normal of the lobe's GGX distribution (Section 9.7) — α_c for the coat, α for metal and specular, which share one branch — and reflect ω_o about it. As in evaluation, sampling happens in the mirrored upper hemisphere, and the z component is flipped back afterwards with `side`. Samples that end up below the horizon are rejected.
- **Glass:** `SampleDielectric` (Section 9.8) with the true directions.
- **Diffuse:** cosine-weighted hemisphere sampling, which is proportional to the $\cos \theta_i$ factor of the rendering equation.

DERIVATION · UNIFORM DISK AND COSINE-WEIGHTED HEMISPHERE

Disk. A point is uniform on the unit disk if its density per unit area is $1/\pi$. In polar coordinates the area element is $r dr d\phi$, so the radius must have density $2r$ on $[0, 1]$, with CDF r^2 . Inversion gives $r = \sqrt{\xi_1}$ and $\phi = 2\pi\xi_2$. Without the square root, points would cluster at the centre.

Hemisphere (Malley's method). Lift a uniform disk point vertically onto the hemisphere: $z = \sqrt{1 - r^2} = \cos \theta$. A small patch of the hemisphere with solid angle $d\omega$ projects onto the disk with area $dA = \cos \theta d\omega$. Hence the density per solid angle is

$$p(\omega) = \frac{1}{\pi} \frac{dA}{d\omega} = \frac{\cos \theta}{\pi}. \quad (11.2)$$

For a Lambertian lobe $f = \rho/\pi$ this gives the constant weight $f \cos \theta / p = \rho$.

common.glsl · SampleUniformDisk

```
vec2 SampleUniformDisk(vec2 u)
{
    float r = sqrt(u.x);
    float phi = TWO_PI * u.y;
    return r * vec2(cos(phi), sin(phi));
}
```

common.glsl · SampleCosineHemisphere

```
vec3 SampleCosineHemisphere(vec2 u)
{
    vec2 d = SampleUniformDisk(u);
    return vec3(d, sqrt(max(0.0, 1.0 - dot(d, d))));
}
```

11.3 Why the whole BSDF is evaluated after sampling

After a direction has been generated by one lobe, `SampleBsdf` calls `EvalBsdf`, which returns the sum of *all* lobes and the mixture density

$$p(\omega_i) = p_{\text{coat}} p_r(\omega_i; \alpha_c) + (p_{\text{metal}} + p_{\text{spec}}) p_r(\omega_i; \alpha) + p_{\text{glass}} p_{\text{dielectric}}(\omega_i) + p_{\text{diff}} \frac{\mu_i}{\pi}.$$

This is one-sample MIS with the balance heuristic (Section 4.6). The weight $f|\cos\theta_i|/p$ is unbiased no matter which lobe produced the direction, and a direction that several lobes could have produced — a diffuse sample landing inside the specular highlight, for example — is correctly weighted by all of them. Evaluating only the chosen lobe and dividing by its own density would be wrong, and dividing by its density times its selection probability would give a valid but much noisier estimator. The sample is discarded if the mixture density is below 10^{-12} or the BSDF is zero, which happens for rejected directions (Section 10.8).

11.4 The code

bsdf.glsl · struct Bsdf

```
struct Bsdf
{
    vec3 baseColor;
    vec3 sheen;
    vec3 glassTint;
    vec3 metalScale;
    float diffuseScale;
    float alpha;
    float coatAlpha;
    float ior;
    float clearcoat;
    float wMetal;
    float wOpaque;
    float wGlass;
    float pCoat;
    float pSpecular;
    float pMetal;
    float pGlass;
    float pDiffuse;
};
```

bsdf.glsl · CreateBsdf

```
Bsdf CreateBsdf(SurfaceMaterial material, vec3 wo)
{
    Bsdf b;
    b.baseColor = material.baseColor;
    b.sheen = material.sheen;
    b.glassTint = sqrt(max(material.baseColor, vec3(0.0)));
    b.alpha = max(material.roughness * material.roughness, MIN_ALPHA);
    b.coatAlpha = max(material.clearcoatRoughness * material.clearcoatRoughness, MIN_ALPHA);
    b.ior = max(material.ior, 1.01);
    b.clearcoat = wo.z > 0.0 ? material.clearcoat : 0.0;

    float cos0 = abs(wo.z);
    b.metalScale = 1.0 + material.baseColor * (1.0 / GgxAlbedo(cos0, b.alpha) - 1.0);
    b.diffuseScale = 1.0 / (1.0 - FresnelDiffuseReflectance(b.ior));
    float coatFresnel = b.clearcoat * FresnelDielectric(cos0, COAT_IOR);
```

```

float baseWeight = 1.0 - coatFresnel;
float dielectric = baseWeight * (1.0 - material.metallic);
b.wMetal = baseWeight * material.metallic;
b.wGlass = dielectric * material.transmission;
b.wOpaque = dielectric * (1.0 - material.transmission);

float specularFresnel = FresnelDielectric(cosO, b.ior);
b.pCoat = coatFresnel;
b.pMetal = b.wMetal;
b.pGlass = b.wGlass;
b.pSpecular = b.wOpaque * specularFresnel;
b.pDiffuse = b.wOpaque * (1.0 - specularFresnel);
float total = b.pCoat + b.pMetal + b.pGlass + b.pSpecular + b.pDiffuse;
float scale = total > 0.0 ? 1.0 / total : 0.0;
b.pCoat *= scale;
b.pMetal *= scale;
b.pGlass *= scale;
b.pSpecular *= scale;
b.pDiffuse *= scale;
return b;
}

```

bsdf.glsl · BsdfHasSmoothLobes

```

bool BsdfHasSmoothLobes(Bsdf b)
{
    return b.pDiffuse > 0.0 || b.alpha > NEE_MIN_ALPHA || (b.pCoat > 0.0 && b.coatAlpha > NEE_MIN_ALPHA);
}

```

bsdf.glsl · EvalBsdf

```

vec3 EvalBsdf(Bsdf b, vec3 wo, vec3 wi, vec3 ng, out float pdf)
{
    pdf = 0.0;
    vec3 f = vec3(0.0);
    if (wo.z == 0.0 || wi.z == 0.0)
        return f;
    bool reflection = wo.z * wi.z > 0.0;
    if (reflection != (dot(wo, ng) * dot(wi, ng) > 0.0))
        return f;

    if (reflection)
    {
        vec3 o = vec3(wo.xy, abs(wo.z));
        vec3 i = vec3(wi.xy, abs(wi.z));
        vec3 h = normalize(o + i);
        float oh = max(dot(o, h), 1e-8);
        float denom = 4.0 * o.z * i.z;
        if (b.pCoat > 0.0)
        {
            f += vec3(b.clearcoat * FresnelDielectric(oh, COAT_IOR) * GgxD(h, b.coatAlpha) * GgxG2(o, i, b.coatAlpha) /
            denom);
            pdf += b.pCoat * GgxVisiblePdf(o, h, b.coatAlpha) / (4.0 * oh);
        }

        float D = GgxD(h, b.alpha);
        float G = GgxG2(o, i, b.alpha);
        float specularPdf = GgxVisiblePdf(o, h, b.alpha) / (4.0 * oh);
        if (b.wMetal > 0.0)
        {
            f += b.wMetal * b.metalScale * FresnelSchlick(b.baseColor, oh) * (D * G / denom);
            pdf += b.pMetal * specularPdf;
        }
        if (b.wOpaque > 0.0)
        {
            float fo = FresnelDielectric(o.z, b.ior);
            float fi = FresnelDielectric(i.z, b.ior);
            f += vec3(b.wOpaque * FresnelDielectric(oh, b.ior) * D * G / denom);
            f += b.wOpaque * (b.baseColor * (INV_PI * b.diffuseScale * (1.0 - fo) * (1.0 - fi)) + b.sheen *
            SchlickWeight(dot(i, h)));
            pdf += b.pSpecular * specularPdf + b.pDiffuse * i.z * INV_PI;
        }
    }

    if (b.wGlass > 0.0)
    {
        float glassPdf;

```

```

    f += b.wGlass * EvalDielectric(wo, wi, b.alpha, b.ior, b.glassTint, glassPdf);
    pdf += b.pGlass * glassPdf;
}
return f;
}

```

bsdf.glsl · SampleBsdf

```

bool SampleBsdf(Bsdf b, vec3 wo, vec3 ng, inout uint rng, out vec3 wi, out vec3 f, out float pdf)
{
    float side = wo.z >= 0.0 ? 1.0 : -1.0;
    vec3 o = vec3(wo.xy, abs(wo.z));
    float lobe = Random(rng);
    vec2 u = Random2(rng);
    f = vec3(0.0);
    pdf = 0.0;

    if (lobe < b.pCoat)
    {
        vec3 i = reflect(-o, SampleGgxVisibleNormal(o, b.coatAlpha, u));
        if (i.z <= 0.0)
            return false;
        wi = vec3(i.xy, i.z * side);
    }
    else if (lobe < b.pCoat + b.pMetal + b.pSpecular)
    {
        vec3 i = reflect(-o, SampleGgxVisibleNormal(o, b.alpha, u));
        if (i.z <= 0.0)
            return false;
        wi = vec3(i.xy, i.z * side);
    }
    else if (lobe < b.pCoat + b.pMetal + b.pSpecular + b.pGlass)
    {
        if (!SampleDielectric(wo, b.alpha, b.ior, vec3(u, Random(rng)), wi))
            return false;
    }
    else
    {
        vec3 i = SampleCosineHemisphere(u);
        wi = vec3(i.xy, i.z * side);
    }

    f = EvalBsdf(b, wo, wi, ng, pdf);
    return pdf > 1e-12 && MaxComponent(f) > 0.0;
}

```

12. Verifying energy conservation

12.1 The white furnace test

Place an object in a uniform environment of radiance L_{env} and remove all lights. If every interaction conserves energy and nothing absorbs, the radiance field $L \equiv L_{\text{env}}$ is a solution of the rendering equation: every surface reflects and transmits exactly what it receives. The object becomes invisible against the background, no matter how many bounces the light takes inside it. An object that loses energy appears darker than the background; one that creates energy appears brighter. This "white furnace" test is the standard sanity check for BSDFs and media, because it tests the evaluation, the sampling densities and the path loop at once.

12.2 Setup and results

Eight spheres with white or non-absorbing materials were rendered in a uniform sky of radiance 0.5, with tone mapping disabled so that pixel values are linear, and the average radiance at each sphere's centre was compared with the background. Runs with 64 and with 1024 maximum bounces gave identical numbers. The "first version" column is the renderer before the diffuse normalisation, the metal compensation and the path-level spectral MIS were added.

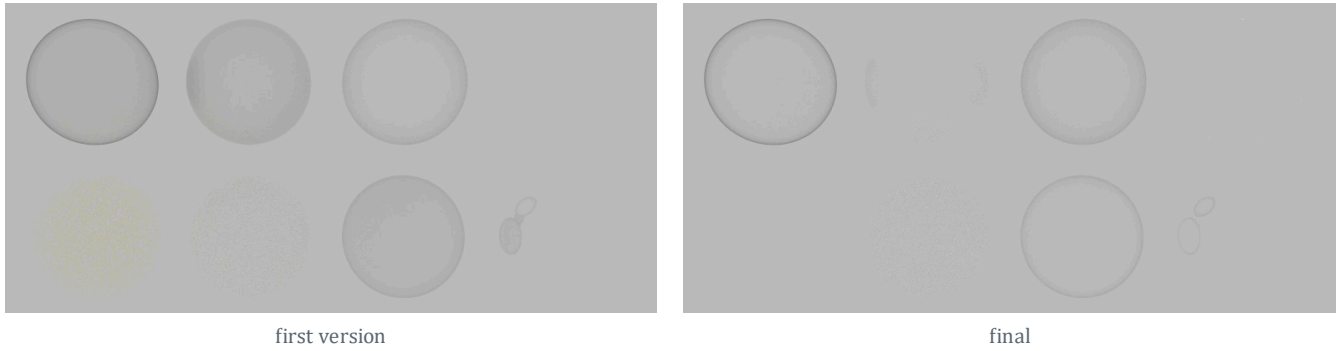


Figure 21. White furnace (uniform sky, no tone mapping). Top row: diffuse $r = 1$, metal $r = 0.5$, rough glass $r = 0.3$, smooth glass. Bottom row: null-bounded medium with strongly coloured scattering $\sigma_s = (1, 5, 20)$ and $g = 0.5$; glass filled with a dense medium, $\sigma_s = 10$, $g = -0.3$; clear coat over white diffuse; metal $r = 0.05$. In the final version the metal, coat and media spheres all but vanish, and the coloured speckles of the chromatic medium are gone (Section 20). The darker ellipses at the bottom right are mirror reflections of the neighbouring spheres, not errors.

Sphere	first version	final	remaining deviation explained by
diffuse (white plastic), $r = 1$	0.88	0.97	single-scatter specular loss; grazing diffuse transmission
metal (white), $r = 0.5$	0.91	1.01	albedo-table interpolation
clear coat over white diffuse, $r = 0.5$	0.91	1.00	—
rough glass, $r = 0.3$	0.99	0.99	uncompensated multiple scattering (0.93 at the rim)
smooth glass	1.00	1.00	—
metal (white), $r = 0.05$	1.01	1.01	albedo-table interpolation
null-bounded chromatic medium	0.99	1.00	—
glass with a dense scattering medium	0.99	1.00	—

Sphere radiance divided by the background radiance (1.00 = perfect energy conservation), measured at the sphere centres.

12.3 A CPU reference for the sampling code

The furnace test cannot distinguish a wrong BSDF from a wrong sampling density that happens to cancel. A separate check therefore ported `GgxD`, `GgxLambda`, `SampleGgxVisibleNormal` and `GgxVisiblePdf` to C# and computed the directional albedo in two independent ways: by visible-normal sampling with the shader's pdf, and by uniform hemisphere sampling, which needs no pdf of the BSDF at all. The two agree to four decimals for all tested roughness values and angles. In addition, the integral of the reflection pdf over the hemisphere equals the fraction of samples that are not rejected below the horizon. Together these confirm that the density returned by `EvalBsdf` is exactly the density that `SampleBsdf` samples — the most common source of subtle bias in path tracers. The same program produced the albedo table of Section 10.4 and the albedo plots.

12.4 What still loses energy

- **Rough glass** and the **clear coat** use single-scattering microfacets without compensation. Compensating a dielectric would need a table over (μ, α, η) for both reflection and transmission (Kulla & Conty 2017).
- The **diffuse transmission** uses the Fresnel term of the macro surface. At grazing angles on very rough surfaces this underestimates the light that enters, so white plastic with $r = 1$ keeps a slightly dark rim (0.95).
- The **shading-normal check** of Section 10.8 discards a small amount of energy at grazing angles on smooth-shaded meshes.

PART IV

Light transport

With the BSDF in place, this part assembles the path tracer: how light sources are sampled directly, how light sampling and BSDF sampling are combined without double counting, how paths are terminated, and how all of it fits into the loop that runs for every pixel.

13. Next event estimation

A path that relies on BSDF sampling alone finds a light only when a sampled direction happens to hit it. For small lights that almost never happens, and the image stays noisy for a very long time. *Next event estimation* (NEE) adds, at every scattering vertex, an explicit connection to a random point on a light — a Monte Carlo estimate of the direct lighting integral (3.4).

13.1 The light list

Lights are ordinary triangles whose material has a non-zero emission. `Scene::Build` walks over all instances, and for every part whose resolved material emits (luminance $Y(L_e) > 0$, and not a null surface) it transforms the part's triangles to world space and appends them to the light list. Each triangle receives the power weight

$$P_k = Y(L_{e,k}) A_k, \quad (13.1)$$

proportional to its emitted flux $\pi L_e A$ (Section 2.4). The list stores the running sum of these weights, normalised to a cumulative distribution function (CDF) whose last entry is forced to exactly 1 so that rounding can never leave a gap at the end.

Scene.cpp · Scene::Build (excerpt: primitives and light list)

```
const glm::mat3 normalMatrix = glm::transpose(glm::inverse(glm::mat3(m)));
for (const ModelPart& part : model.Parts)
{
    const uint32_t index = materialIndex(instance.Material.IsValid() ? instance.Material : part.Material);
    primitives.push_back({ model.VertexAddress, model.IndexAddress + uint64_t(part.FirstIndex) * sizeof(uint32_t), index
});

    const Material& material = m_Materials[index];
    const float luminance = Luminance(material.Emission);
    if (luminance <= 0.0f || material.NullSurface)
        continue;

    for (uint32_t i = part.FirstIndex; i < part.FirstIndex + part.IndexCount; i += 3)
    {
        const glm::vec3& a = model.CpuPositions[model.CpuIndices[i]];
        const glm::vec3& b = model.CpuPositions[model.CpuIndices[i + 1]];
        const glm::vec3& c = model.CpuPositions[model.CpuIndices[i + 2]];
        glm::vec3 p0 = glm::vec3(m * glm::vec4(a, 1.0f));
        glm::vec3 p1 = glm::vec3(m * glm::vec4(b, 1.0f));
        glm::vec3 p2 = glm::vec3(m * glm::vec4(c, 1.0f));
        const glm::vec3 worldCross = glm::cross(p1 - p0, p2 - p0);
        const float area = 0.5f * glm::length(worldCross);
        if (area <= 0.0f)
            continue;
        if (glm::dot(worldCross, normalMatrix * glm::cross(b - a, c - a)) < 0.0f)
            std::swap(p1, p2);

        totalPower += double(luminance) * area;
        lights.push_back({ p0, float(totalPower), p1, 0.0f, p2, 0.0f, material.Emission, 0.0f });
    }
}
```

```

    }
  }
}
for (GpuLightTriangle& light : lights)
  light.Cdf = float(light.Cdf / totalPower);
if (!lights.empty())
  lights.back().Cdf = 1.0f;

```

One subtlety is the orientation of each light triangle. The closest-hit shader computes the geometric normal from the object-space cross product transformed with the inverse transpose (Section 6.3). The light list computes its normal from the world-space vertices. For a mirroring instance transform (negative determinant) these disagree, because $M\mathbf{a} \times M\mathbf{b} = \det(M) M^{-T}(\mathbf{a} \times \mathbf{b})$. The build therefore swaps two vertices whenever the world-space cross product points against the transformed object-space normal, so that "front face" means the same thing in both places.

In the showcase scene the ceiling panel ($L_e = 14$, area 0.315) carries 94% of the total power, the orange orb inside the smoke 4.7% and the cyan orb 1.5%.

13.2 Choosing a point on a light

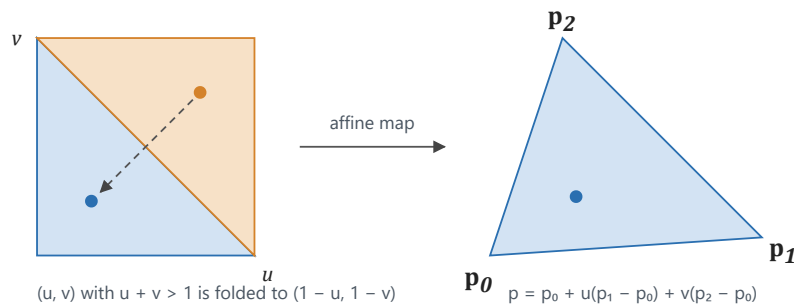


Figure 22. Uniform sampling of a triangle. A uniform point in the unit square is folded into the lower-left half, then mapped affinely onto the triangle.

`SampleLight` draws one random number, finds the first CDF entry that is at least that large by binary search, and thereby selects triangle k with probability $P_k / \sum_j P_j$. It then samples a point uniformly on the triangle with two more random numbers.

DERIVATION · UNIFORM POINT ON A TRIANGLE

Let (u, v) be uniform on the unit square. The half $u + v \leq 1$ is a triangle of area $\frac{1}{2}$; the reflection $(u, v) \mapsto (1 - u, 1 - v)$ maps the other half onto it and preserves area. After folding, (u, v) is uniform on the lower triangle with density 2. The affine map $\mathbf{p} = \mathbf{p}_0 + u(\mathbf{p}_1 - \mathbf{p}_0) + v(\mathbf{p}_2 - \mathbf{p}_0)$ has the constant Jacobian $\|(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)\| = 2A$, so $\mathbf{p}$ is uniform on the triangle with density $2/(2A) = 1/A$. The barycentric coordinates are $(1 - u - v, u, v)$. Unlike the common square-root parameterisation, folding needs no square root and preserves the stratification of (u, v) .

The probability density of the resulting point per unit area is the selection probability times the density on the triangle:

$$p_A(\mathbf{y}) = \frac{Y(L_{e,k}) A_k}{\sum_j P_j} \cdot \frac{1}{A_k} = \frac{Y(L_{e,k})}{\sum_j P_j}. \quad (13.2)$$

KEY IDEA

With power-proportional selection and uniform sampling within each triangle, the area density of a light point depends only on the emitted radiance at that point, not on the triangle's size or on which instance it belongs to. `LightPdfArea(emission)` can therefore recompute the density of any point on any emitter from its material alone — exactly what MIS needs when a BSDF-sampled ray hits an emitter (Section 14).

raygen.rgen · SampleLight

```

bool SampleLight(inout uint rng, out LightSample light)
{
    if (lightCount == 0u || totalLightPower <= 0.0)
        return false;

    float u = Random(rng);
    uint low = 0u;
    uint high = lightCount - 1u;
    while (low < high)
    {
        uint mid = (low + high) / 2u;
        if (lights[mid].cdf < u)
            low = mid + 1u;
        else
            high = mid;
    }

    LightTriangle triangle = lights[low];
    vec3 bary = SampleUniformTriangle(Random2(rng));
    light.position = triangle.p0 * bary.x + triangle.p1 * bary.y + triangle.p2 * bary.z;
    light.normal = normalize(cross(triangle.p1 - triangle.p0, triangle.p2 - triangle.p0));
    light.emission = triangle.emission;
    light.pdfArea = Luminance(triangle.emission) / totalLightPower;
    return true;
}

```

raygen.rgen · LightPdfArea

```

float LightPdfArea(vec3 emission)
{
    return totalLightPower > 0.0 ? Luminance(emission) / totalLightPower : 0.0;
}

```

common.glsf · SampleUniformTriangle

```

vec3 SampleUniformTriangle(vec2 u)
{
    if (u.x + u.y > 1.0)
        u = 1.0 - u;
    return vec3(1.0 - u.x - u.y, u.x, u.y);
}

```

13.3 The estimator

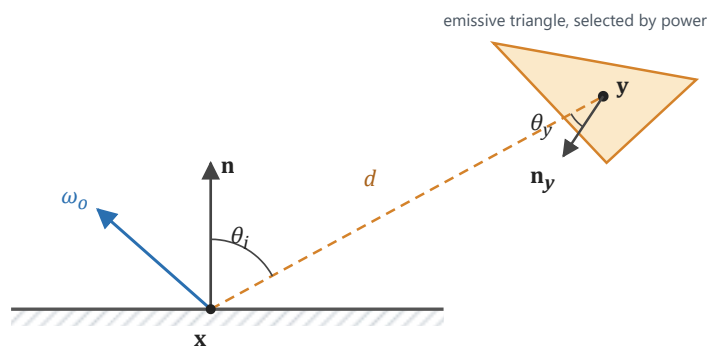


Figure 23. Next event estimation: a point y on a light is chosen, and the shadow ray from x to y decides whether (and how much of) its emission arrives.

Given a light sample y with normal $\mathbf{n}_y$ at distance d in direction ω_i , the area density is converted to solid angle with (4.3), $p_{\text{light}} = p_A d^2 / \cos \theta_y$. The emitter is one-sided: if $\cos \theta_y = \mathbf{n}_y \cdot (-\omega_i) \leq 0$, the sample sees the back of the light and contributes nothing. Otherwise the estimate of the direct light at a surface is

$$\langle L_{\text{direct}} \rangle = \frac{f(\omega_o, \omega_i) |\omega_i \cdot \mathbf{n}| L_e T(\mathbf{x}, \mathbf{y})}{p_{\text{light}}} w_{\text{light}}, \quad (13.3)$$

where T is the transmittance along the shadow ray: 1 for a clear line of sight in vacuum, 0 if an opaque surface blocks it, and the product of the media transmittances along the way otherwise (Section 21). w_{light} is the MIS weight of the next section.

If the light lies on the other side of the surface — possible for rough glass — the shadow ray starts on that side and inside the corresponding medium. `DirectLightingSurface` therefore updates a copy of the medium stack before tracing it.

raygen.rgen · DirectLightingSurface

```
vec3 DirectLightingSurface(vec3 position, vec3 ng, vec3 wo, mat3 frame, BsdF bsdf, int surfaceMedium, MediumStack stack,
inout uint rng)
{
    LightSample light;
    if (!SampleLight(rng, light))
        return vec3(0.0);

    vec3 toLight = light.position - position;
    float distance2 = dot(toLight, toLight);
    vec3 wi = toLight * inversesqrt(distance2);
    float cosLight = dot(light.normal, -wi);
    if (cosLight <= 0.0)
        return vec3(0.0);

    float bsdfPdf;
    vec3 f = EvalBsdF(bsdf, ToLocal(frame, wo), ToLocal(frame, wi), ToLocal(frame, ng), bsdfPdf);
    if (MaxComponent(f) <= 0.0)
        return vec3(0.0);

    bool outside = dot(wi, ng) > 0.0;
    if (outside != (dot(wo, ng) > 0.0))
        CrossBoundary(stack, !outside, surfaceMedium);
    vec3 transmittance = Transmittance(OffsetRay(position, outside ? ng : -ng), light.position, stack);
    if (MaxComponent(transmittance) <= 0.0)
        return vec3(0.0);

    float lightPdf = light.pdfArea * distance2 / cosLight;
    return f * abs(dot(wi, frame[2])) * light.emission * transmittance * (PowerHeuristic(lightPdf, bsdfPdf) / lightPdf);
}
```

13.4 What counts as a light

Only emissive triangles are in the light list. The sky is not: it is never sampled explicitly and contributes only when a BSDF- or phase-sampled ray escapes the scene. In the showcase scene the sky enters through the open front of the room, where BSDF sampling finds it easily; for a scene lit mainly through a small window, environment importance sampling would be needed (Section 26). Because selection is proportional to power, a small light next to a much brighter one receives few samples even where it dominates locally — the orange orb inside the smoke is chosen for only about 5% of the light samples. Many-light methods (light BVHs) address this.

14. Combining light and BSDF sampling

14.1 Two strategies for the same light

After NEE, every emitter can be reached in two ways: by the explicit light sample, and by the BSDF-sampled continuation ray happening to hit it. Adding both contributions naively counts direct light twice. Using only one of them wastes the other's strengths:

- For a **glossy** surface reflecting a **large** light, most light samples fall outside the narrow lobe, and the rare ones inside it carry a huge weight f/p_{light} . BSDF sampling finds the light reliably.
- For a **rough** surface and a **small** light, BSDF sampling almost never hits the light; light sampling always does.

Multiple importance sampling (Section 4.5) weights both so that each dominates where its density is higher.

14.2 Power heuristic weights

With both densities expressed per solid angle of the same direction ω_i ,

$$w_{\text{light}} = \frac{p_{\text{light}}^2}{p_{\text{light}}^2 + p_{\text{bsdf}}^2}, \quad w_{\text{bsdf}} = \frac{p_{\text{bsdf}}^2}{p_{\text{bsdf}}^2 + p_{\text{light}}^2}, \quad w_{\text{light}} + w_{\text{bsdf}} = 1. \quad (14.1)$$

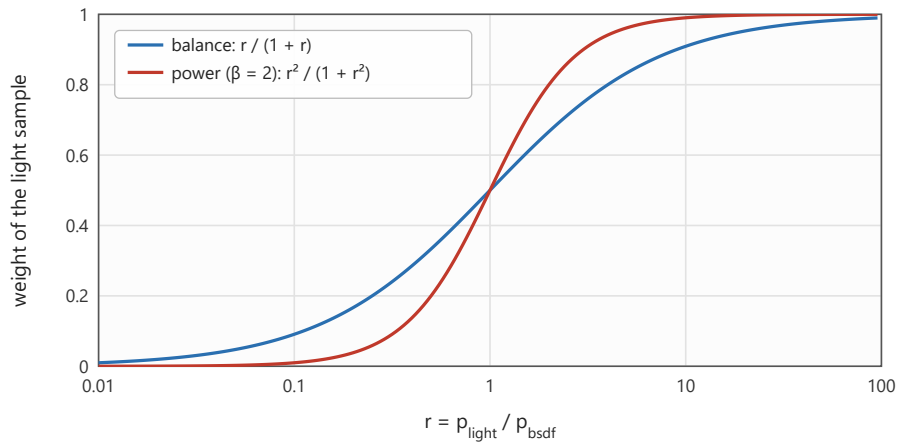


Figure 24. MIS weight of the light sample as a function of the density ratio. The power heuristic (red) hands the sample almost entirely to the better strategy once the densities differ by a factor of a few.

common.glsl · PowerHeuristic

```
float PowerHeuristic(float pdfA, float pdfB)
{
    float a = pdfA * pdfA;
    float b = pdfB * pdfB;
    return a / max(a + b, 1e-30);
}
```

14.3 Implementation

The two weights are applied at two different moments of the path:

1. **At the NEE connection** (`DirectLightingSurface`, `DirectLightingMedium`): p_{light} comes from the light sample, and p_{bsdf} is the mixture density returned by `EvalBsdf` for ω_i (or the phase function value in a medium). The contribution is multiplied by w_{light} .
2. **When the continuation ray hits an emitter** (in `TracePath`): p_{bsdf} is `previousPdf`, the density with which the ray's direction was sampled at the previous scattering vertex. p_{light} is recomputed as `LightPdfArea(emission) · d2 / cos θy` with d measured from `previousPosition`. The emission is multiplied by w_{bsdf} .

Null boundaries and medium segments without a scattering event do not change the ray's direction, so they leave `previousPosition` and `previousPdf` untouched. The distance d is then measured along the same straight line that the shadow ray would have taken, through the same boundaries. Both strategies also estimate the same transmittance: NEE computes it analytically, and the BSDF path estimates it stochastically by free-flight sampling (Section 19). The combination is therefore unbiased: for every direction the two weights sum to one, and each strategy's estimate is unbiased for its weighted share.

IN THE CODE

```
if (frontFace && emission > 0)
    weight = previousNee ? PowerHeuristic(previousPdf, LightPdfArea(emission) · d2 / cos θy) : 1
    radiance += β · emission · weight
```

14.4 When there is only one strategy

If NEE was skipped at the previous vertex — a near-specular BSDF (Section 10.9) — or there was no previous vertex (the camera ray), then BSDF sampling is the only strategy that can find the emitter, and its weight must be 1. The flag `previousNee` records which case applies. At medium scattering events NEE is always performed, so `previousNee` is set to true there.

14.5 The environment

The sky is never sampled by NEE, so a ray that escapes always adds βL_{env} with weight 1.

15. Russian roulette

After the third bounce, a path survives with probability

$$q = \min \left(\max_c \beta_c, 0.95 \right), \quad (15.1)$$

and its throughput is divided by q when it survives. A path whose throughput has dropped to 10% survives one bounce in ten, but then carries ten times the weight, so the expectation is unchanged (Section 4.7). The rule uses the largest colour channel rather than luminance, so that paths carrying strongly coloured light, such as the blue interior of the tinted sphere, are not terminated prematurely. The cap of 0.95 guarantees termination even where the throughput never falls, for example inside a white furnace, where β stays at 1 forever. The first three bounces are always traced, because short paths carry most of the image. Independently of roulette, `maxBounces` (default 64) is a hard limit; it is rarely reached, and the furnace test shows no measurable difference between 64 and 1024.

raygen.rgen · RussianRoulette

```
bool RussianRoulette(inout vec3 beta, uint depth, inout uint rng)
{
    if (depth < 3u)
        return false;
    float survival = min(MaxComponent(beta), 0.95);
    if (survival <= 0.0 || Random(rng) >= survival)
        return true;
    beta /= survival;
    return false;
}
```

16. The complete path loop

16.1 Overview

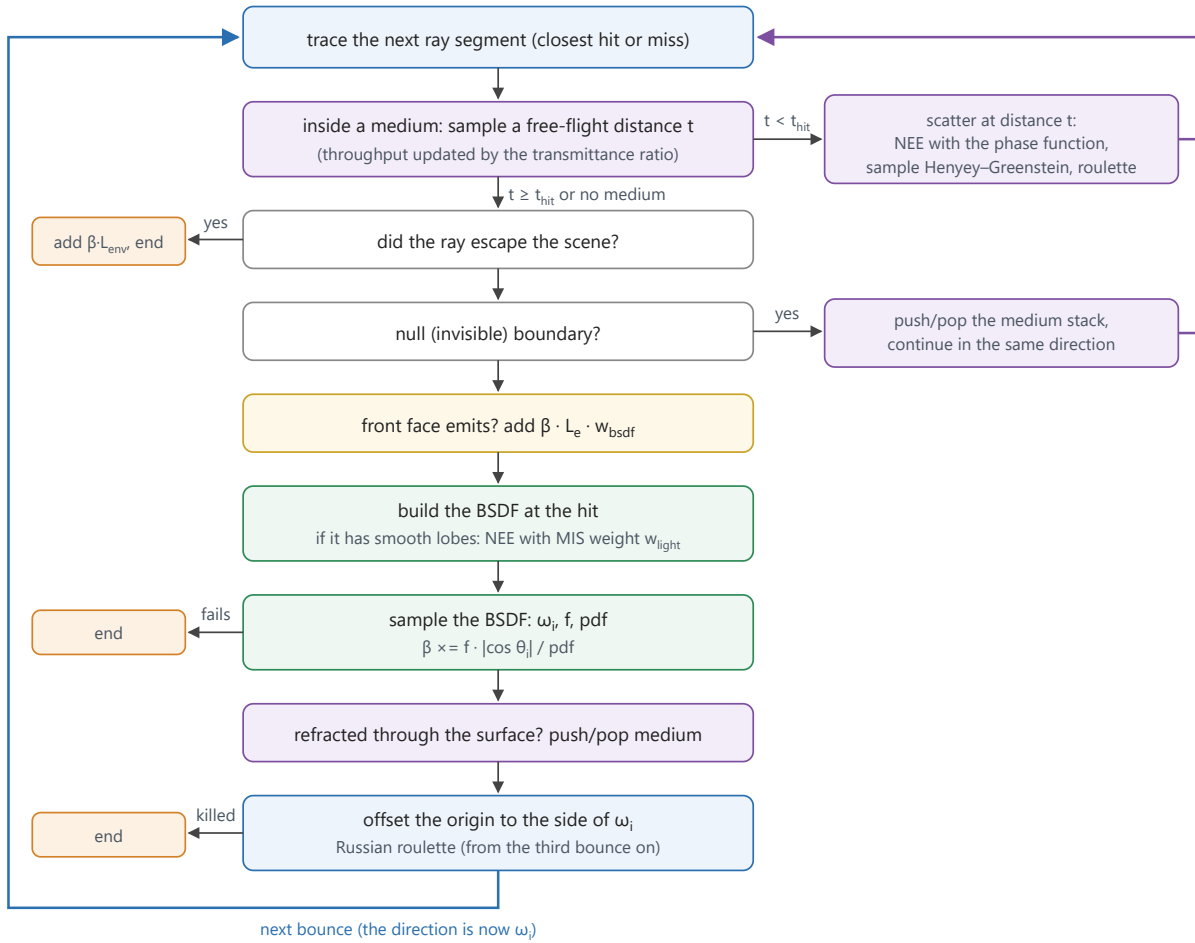


Figure 25. Control flow of `TracePath`. Purple: medium events and boundary crossings; green: surface scattering; orange: path termination. Not shown: every scattering event increments the depth, and the path ends when it exceeds `maxBounces`.

16.2 Step by step

1. **Trace.** `TraceClosest` finds the nearest surface along the ray or reports a miss. The payload is copied into `surface`, because shadow rays traced later overwrite the payload.
2. **Medium.** If the top of the medium stack is a medium, `SampleMediumInteraction` samples a free-flight distance (Section 19). If it falls before the surface, the path scatters inside the medium: NEE with the phase function, a new direction from the Henyey-Greenstein distribution, Russian roulette, and the loop restarts from the scattering point. Otherwise the throughput has been multiplied by the transmittance ratio and the path proceeds to the surface.
3. **Miss.** An escaped ray adds βL_{env} and ends the path.
4. **Null boundary.** An invisible surface only changes the current medium (push when entering through its front face, pop when leaving). The ray continues in the same direction from just behind the surface.
5. **Emission.** A front-facing emitter adds βL_e with the MIS weight of Section 14.3.
6. **BSDF and NEE.** The frame is built around the shading normal, the BSDF is created for ω_o and, if it has smooth lobes, a light sample is connected.
7. **Continue.** A direction is sampled, the throughput is updated, the medium stack is updated if the direction crosses the surface, the new origin is offset to the side of ω_i , and `previousPosition`, `previousPdf` and `previousNee` are recorded for the MIS weight at the next emitter hit. Russian roulette may end the path.

16.3 The code

raygen.rgen · TraceClosest

```
bool TraceClosest(vec3 origin, vec3 direction, float tMax)
{
    payload.t = -1.0;
    traceRayEXT(topLevelAS, gl_RayFlagsOpaqueEXT, 0xFF, 0, 0, 0, origin, 0.0, direction, tMax, 0);
    return payload.t >= 0.0;
}
```

raygen.rgen · TracePath

```
vec3 TracePath(vec3 origin, vec3 direction, inout uint rng)
{
    vec3 radiance = vec3(0.0);
    vec3 beta = vec3(1.0);
    vec3 spectralPdf = vec3(1.0);
    int hero = min(int(Random(rng) * 3.0), 2);
    MediumStack stack = CreateMediumStack();
    vec3 previousPosition = origin;
    float previousPdf = 0.0;
    bool previousNee = false;
    uint depth = 0;
    int nullCrossings = 0;

    while (true)
    {
        bool hit = TraceClosest(origin, direction, INF);
        HitPayload surface = payload;

        int mediumIndex = CurrentMedium(stack);
        if (mediumIndex >= 0)
        {
            Medium medium = media[mediumIndex];
            float t;
            if (SampleMediumInteraction(medium, hit ? surface.t : INF, hero, beta, spectralPdf, rng, t))
            {
                if (++depth > globalData.maxBounces)
                    break;

                vec3 position = origin + direction * t;
                vec3 wo = -direction;
                radiance += beta * DirectLightingMedium(position, wo, medium.g, stack, rng);

                float phasePdf;
                direction = SampleHenyeyGreenstein(wo, medium.g, Random2(rng), phasePdf);
                origin = position;
                previousPosition = position;
                previousPdf = phasePdf;
                previousNee = true;
                if (RussianRoulette(beta, depth, rng))
                    break;
                continue;
            }
            if (MaxComponent(beta) <= 0.0)
                break;
        }

        if (!hit)
        {
            radiance += beta * EnvironmentRadiance(direction);
            break;
        }

        SurfaceMaterial material = FetchMaterial(surface.materialIndex, surface.uv);
        vec3 ng = surface.geometricNormal;
        bool frontFace = surface.frontFace != 0;

        if (material.nullSurface)
        {
            if (++nullCrossings > MAX_NULL_CROSSINGS)
                break;
            CrossBoundary(stack, frontFace, material.medium);
            origin = OffsetRay(surface.position, frontFace ? -ng : ng);
            continue;
        }
    }
}
```

```

if (frontFace && MaxComponent(material.emission) > 0.0)
{
    float weight = 1.0;
    if (previousNee)
    {
        vec3 toLight = surface.position - previousPosition;
        float lightPdf = LightPdfArea(material.emission) * dot(toLight, toLight) / max(dot(ng, -direction), 1e-8);
        weight = PowerHeuristic(previousPdf, lightPdf);
    }
    radiance += beta * material.emission * weight;
}

if (++depth > globalData.maxBounces)
    break;

vec3 wo = -direction;
mat3 frame = BuildFrame(surface.shadingNormal);
vec3 woLocal = ToLocal(frame, wo);
vec3 ngLocal = ToLocal(frame, ng);
Bsdf bsdf = CreateBsdf(material, woLocal);

bool nee = BsdfHasSmoothLobes(bsdf);
if (nee)
    radiance += beta * DirectLightingSurface(surface.position, ng, wo, frame, bsdf, material.medium, stack, rng);

vec3 wiLocal;
vec3 f;
float pdf;
if (!SampleBsdf(bsdf, woLocal, ngLocal, rng, wiLocal, f, pdf))
    break;

vec3 wi = ToWorld(frame, wiLocal);
beta *= f * abs(wiLocal.z) / pdf;

bool outside = dot(wi, ng) > 0.0;
if (outside != (dot(wo, ng) > 0.0))
    CrossBoundary(stack, !outside, material.medium);

origin = OffsetRay(surface.position, outside ? ng : -ng);
direction = wi;
previousPosition = surface.position;
previousPdf = pdf;
previousNee = nee;
if (RussianRoulette(beta, depth, rng))
    break;
}
return radiance;
}

```

16.4 The environment

The sky is procedural and not physically calibrated. It blends from white at the horizon to blue at the zenith, adds a sun lobe $100 \max(0, \omega \cdot \mathbf{s})^{100}$ around $\mathbf{s} = \text{normalize}(1, 1, 1)$, and returns a light grey ground below the horizon. It lights the showcase room through the open front.

scene.glsl · EnvironmentRadiance

```

vec3 EnvironmentRadiance(vec3 dir)
{
    const vec3 sunDir = normalize(vec3(1.0));
    const vec3 groundColor = vec3(0.9);
    const vec3 horizonColor = vec3(1.0);
    const vec3 zenithColor = vec3(0.08, 0.37, 0.73);

    float sun = pow(max(0.0, dot(dir, sunDir)), 100.0) * 100.0;
    float skyGradient = pow(smoothstep(0.0, 0.4, dir.y), 0.45);
    float groundToSky = smoothstep(-0.01, 0.0, dir.y);
    vec3 sky = mix(horizonColor, zenithColor, skyGradient);
    return mix(groundColor, sky, groundToSky) + sun * float(groundToSky >= 1.0);
}

```

PART V

Participating media

Smoke, fog, coloured glass, wax, jade and skin do not only interact with light at their surfaces: light is absorbed and scattered throughout their volume. This part derives the equation that describes such media, the sampling methods that solve it for homogeneous media, the spectral technique that keeps coloured media noise-free, and the bookkeeping that tells a ray which medium it is in.

17. Radiative transfer

17.1 Absorption, scattering and extinction

A participating medium is a volume filled with particles that are too small and numerous to model individually. Along a short distance ds , a beam of light loses the fraction $\sigma_a ds$ to **absorption** (converted to heat) and the fraction $\sigma_s ds$ to **out-scattering** (redirected into other directions). The coefficients have units of inverse length. Their sum is the **extinction coefficient** $\sigma_t = \sigma_a + \sigma_s$, and two derived quantities are more intuitive:

- the **mean free path** $1/\sigma_t$, the average distance light travels between interactions, and
- the **single-scattering albedo** $\varpi = \sigma_s/\sigma_t$, the probability that an interaction scatters rather than absorbs.

The beam also gains radiance by **in-scattering** of light arriving from other directions. Emissive media (fire, glowing gas) would add a fourth term; the renderer does not model volumetric emission.

17.2 The radiative transfer equation

Let $L(\mathbf{x}, \omega)$ be the radiance at $\mathbf{x}$ travelling in direction ω . Its change along the direction of travel is

$$(\omega \cdot \nabla) L(\mathbf{x}, \omega) = -\sigma_t L(\mathbf{x}, \omega) + \sigma_s \int_{S^2} p(\omega' \rightarrow \omega) L(\mathbf{x}, \omega') d\omega', \quad (17.1)$$

where the **phase function** $p(\omega' \rightarrow \omega)$ is the probability density that light travelling in direction ω' is scattered into direction ω (Section 18). The first term is the loss by absorption and out-scattering, the second the gain by in-scattering. Equation (17.1) is the radiative transfer equation (RTE; Chandrasekhar 1960). The rendering equation (3.2) is its boundary condition at surfaces.

17.3 The integral form along a ray

A path tracer needs the radiance arriving at a point from one direction. Integrating (17.1) along the ray $\mathbf{x}_t = \mathbf{x}_0 + t\omega$ from $\mathbf{x}_0$ to the first surface at distance t_s gives the *volume rendering equation*:

$$L_i(\mathbf{x}_0, \omega) = \int_0^{t_s} T(t) \sigma_s L_s(\mathbf{x}_t, -\omega) dt + T(t_s) L_o(\mathbf{x}_{t_s}, -\omega), \quad (17.2)$$

with the **transmittance**

$$T(t) = \exp\left(-\int_0^t \sigma_t(\mathbf{x}_{t'}) dt'\right) \stackrel{\text{homogeneous}}{=} e^{-\sigma_t t}, \quad (17.3)$$

and the **in-scattered radiance** toward the viewer,

$$L_s(\mathbf{x}, \omega_o) = \int_{S^2} p(\omega_o, \omega_i) L_i(\mathbf{x}, \omega_i) d\omega_i. \quad (17.4)$$

Read (17.2) from right to left: the radiance of the surface behind the medium arrives attenuated by $T(t_s)$ (the Beer-Lambert law), and every point along the ray adds the light it scatters toward the viewer, itself attenuated by the medium in between. In a homogeneous medium, $\sigma_t = 5$ per unit means that light is attenuated by $1/e$ every 0.2 units.

18. Phase functions

18.1 Definition

The phase function plays the role of the BSDF inside a medium. It is a probability density over directions, normalised as $\int_{S^2} p(\omega_o, \omega_i) d\omega_i = 1$, and it has no cosine factor, because there is no surface to project onto. Energy loss inside a medium is described entirely by σ_a ; the phase function only redistributes the scattered light. The simplest phase function is isotropic, $p = 1/(4\pi)$.

18.2 The Henyey–Greenstein phase function

The renderer uses the Henyey–Greenstein (HG) phase function (Henyey & Greenstein 1941), which depends only on the scattering angle θ_s between the incoming and the outgoing propagation directions:

$$p_{\text{HG}}(\theta_s) = \frac{1}{4\pi} \frac{1 - g^2}{(1 + g^2 - 2g \cos \theta_s)^{3/2}}. \quad (18.1)$$

The parameter $g \in (-1, 1)$ is the mean cosine of the scattering angle: $g > 0$ scatters forward (light keeps going roughly in its direction), $g < 0$ backward, and $g = 0$ is isotropic. Clouds have $g \approx 0.85$, biological tissue 0.8–0.95, smoke and dust typically 0.2–0.6. The renderer clamps g to $[-0.99, 0.99]$.

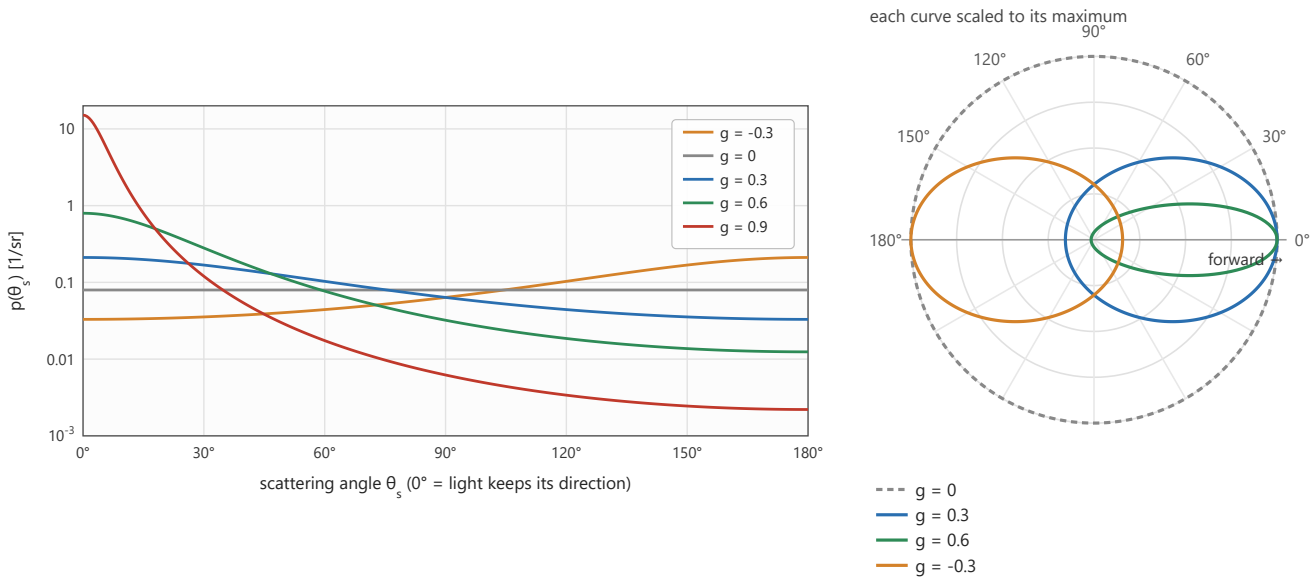


Figure 26. Henyey–Greenstein phase function. Left: $p(\theta_s)$ on a logarithmic scale. Right: polar plot of the lobe shapes (each scaled to its maximum), with light arriving from the left and “forward” to the right.

IN THE CODE · DIRECTION CONVENTION

The shader uses the convention of PBRT-v4. At a scattering point, $\omega_o = -\text{direction}$ points back toward the viewer and ω_i toward where light comes from — both away from the point, as for surfaces. The propagation directions are $-\omega_i$ (incoming light) and ω_o (outgoing light), so $\cos \theta_s = -\omega_o \cdot \omega_i$, and (18.1) becomes

$$p(\omega_o, \omega_i) = \frac{1}{4\pi} \frac{1 - g^2}{(1 + g^2 + 2g(\omega_o \cdot \omega_i))^{3/2}},$$

which is what `HenyeyGreenstein(dot(wo, wi), g)` evaluates. Forward scattering peaks where $\omega_i \approx -\omega_o$: light continues through the medium toward the viewer.

raygen.rgen · HenyeyGreenstein

```
float HenyeyGreenstein(float cosTheta, float g)
{
    float denom = max(1.0 + g * g + 2.0 * g * cosTheta, 1e-12);
    return INV_4PI * (1.0 - g * g) / (denom * sqrt(denom));
}
```

18.3 Sampling Henyey–Greenstein by inversion

The azimuth around the axis is uniform, $\phi = 2\pi\xi_2$. The cosine $\mu = \cos\theta_s$ is sampled by inverting its cumulative distribution:

DERIVATION

The density of μ is $2\pi p_{\text{HG}}(\mu) = \frac{1-g^2}{2(1+g^2-2g\mu)^{3/2}}$ on $[-1, 1]$. With the substitution $w = 1 + g^2 - 2g\mu$,

$$P(\mu) = \int_{-1}^{\mu} \frac{1-g^2}{2(1+g^2-2g\mu')^{3/2}} d\mu' = \frac{1-g^2}{2g} \left(\frac{1}{\sqrt{1+g^2-2g\mu}} - \frac{1}{1+g} \right).$$

Setting $P(\mu) = \xi$ and solving for μ :

$$\mu = \frac{1}{2g} \left(1 + g^2 - \left(\frac{1-g^2}{1-g+2g\xi} \right)^2 \right). \quad (18.2)$$

In the code's convention the cosine is taken with ω_o , i.e. $-\mu$. Replacing ξ by the equally uniform $1 - \xi$ gives the form in `SampleHenyeyGreenstein`: $s = \frac{1-g^2}{1+g-2g\xi}$, $\cos(\omega_o, \omega_i) = -\frac{1+g^2-s^2}{2g}$. For $|g| < 10^{-3}$ the formula divides by nearly zero and the isotropic $\cos = 1 - 2\xi$ is used instead.

The sampled direction is built in an orthonormal frame around ω_o . Because the sampling is exact, the density equals the phase function, $p(\omega_i) = p_{\text{HG}}$, and the phase weight p_{HG}/p is exactly 1: phase-function sampling never changes the throughput. The density is still returned, because MIS needs it (Section 21).

raygen.rgen · SampleHenyeyGreenstein

```
vec3 SampleHenyeyGreenstein(vec3 wo, float g, vec2 u, out float pdf)
{
    float cosTheta;
    if (abs(g) < 1e-3)
    {
        cosTheta = 1.0 - 2.0 * u.x;
    }
    else
    {
        float s = (1.0 - g * g) / (1.0 + g - 2.0 * g * u.x);
        cosTheta = -(1.0 + g * g - s * s) / (2.0 * g);
    }
    cosTheta = clamp(cosTheta, -1.0, 1.0);
    float sinTheta = sqrt(max(0.0, 1.0 - cosTheta * cosTheta));
    float phi = TWO_PI * u.y;
    pdf = HenyeyGreenstein(cosTheta, g);
    return ToWorld(BuildFrame(wo), vec3(sinTheta * cos(phi), sinTheta * sin(phi), cosTheta));
}
```

19. Free-flight distance sampling

19.1 Sampling where the next interaction happens

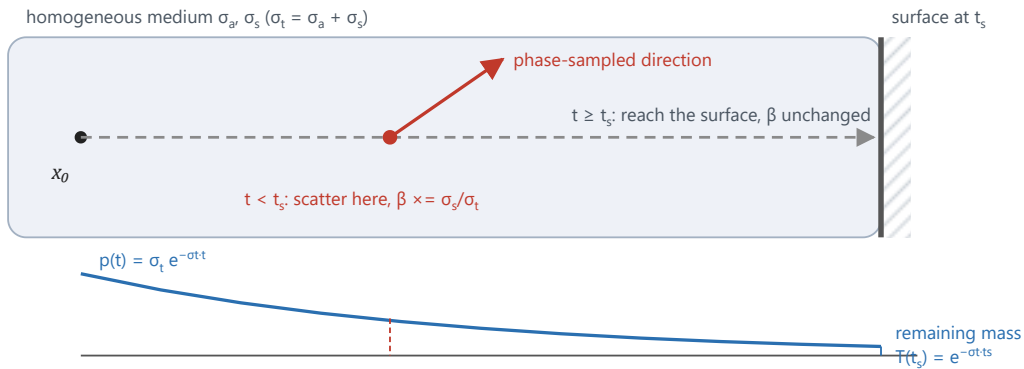


Figure 27. Free-flight sampling. A distance is drawn from the exponential density $p(t)$; if it falls before the surface, the path scatters there. The probability of reaching the surface is the remaining mass $T(t_s)$, exactly the transmittance.

The integral form (17.2) consists of two terms, and free-flight sampling chooses between them at random. For a grey homogeneous medium, draw a distance from the density proportional to the transmittance,

$$p(t) = \sigma_t e^{-\sigma_t t}, \quad t = -\frac{\ln(1 - \xi)}{\sigma_t}. \quad (19.1)$$

The probability that the sample lies beyond the surface is $\int_{t_s}^{\infty} p(t) dt = e^{-\sigma_t t_s} = T(t_s)$.

- $t < t_s$ — **scattering event**. This estimates the in-scattering integral: $\frac{T(t) \sigma_s L_s}{p(t)} = \frac{\sigma_s}{\sigma_t} L_s = \varpi L_s$. The throughput is multiplied by the albedo, and L_s is estimated like a surface bounce: NEE with the phase function plus a phase-sampled continuation.
- $t \geq t_s$ — **no interaction**. This estimates the surface term: $\frac{T(t_s) L_o}{T(t_s)} = L_o$. The throughput is unchanged and the path continues at the surface.

KEY IDEA

In a grey medium, free-flight sampling turns the volume rendering equation into a random walk: fly an exponentially distributed distance, and at every scattering event multiply the throughput by the albedo ϖ and pick a new direction from the phase function. No transmittance is ever multiplied explicitly — the attenuation is expressed by how often paths reach the surface.

19.2 Pure absorbers

If $\sigma_s = 0$ there is no in-scattering, and (17.2) collapses to $L_i = T(t_s) L_o$. No sampling is needed: `SampleMediumInteraction` multiplies the throughput by $e^{-\sigma_a t_s}$ deterministically, which adds no noise at all. This is the physically based way to colour glass. The blue sphere of the showcase scene has $\sigma_a = (3, 1.2, 0.3)$: through its 0.32-unit diameter it transmits (0.38, 0.68, 0.91), while its thin rim transmits almost everything. Thickness-dependent colour is what the thickness-independent glass tint of Section 10.7 cannot express.

19.3 Rays that leave the scene inside a medium

A ray that misses all geometry has $t_s = \infty$ (`INF` = 10^{30} in the code, finite so that $0 \cdot \infty$ cannot produce NaN). In a scattering medium every sampled distance is then finite and the path always scatters; in an absorbing medium $T(\infty) = 0$. Either way, the environment is never reached through an unbounded medium — the reason the global medium is of limited use (Section 22.5).

20. Coloured media: hero channels and spectral MIS

20.1 The problem

Media are specified per RGB channel, and in coloured media σ_t differs between the channels. Distances must be sampled with a single σ , but a density that suits the red channel can be orders of magnitude off for the blue one: sampling with the smallest coefficient wastes steps in the densest channel, and sampling with the largest one almost never samples the long flights of the thinnest channel. Each choice alone gives unbounded weights in the other channels.

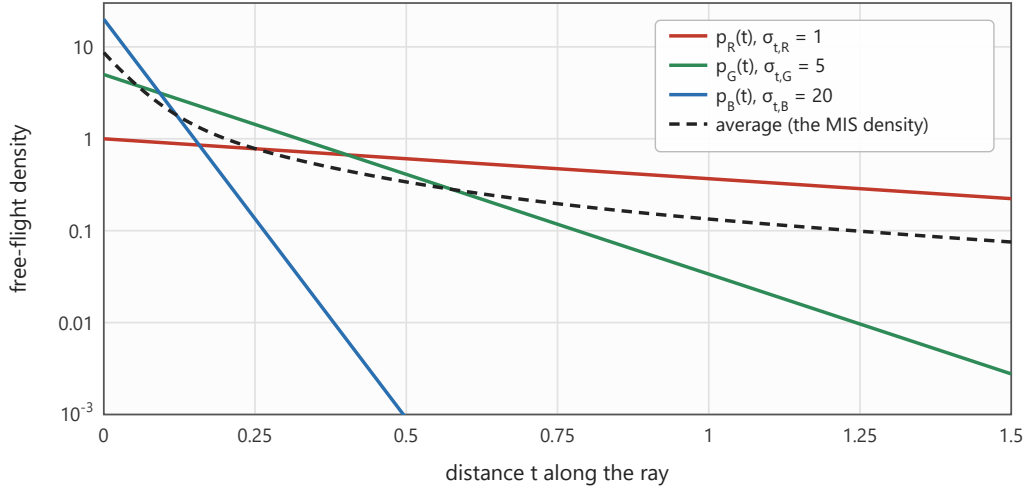


Figure 28. Free-flight densities of the three channels of the furnace-test medium ($\sigma_t = 1, 5, 20$) and their average, which is the density that the MIS estimator effectively uses.

20.2 The channel as a sampling technique

Treat "sample with channel c 's coefficient" as three sampling techniques and combine them with MIS (Section 4.6). Each path picks a *hero channel* h uniformly at random (Wilkie et al. 2014 introduced the idea for spectral rendering) and samples all its free-flight distances with $\sigma_{t,h}$. Let $p_c(\bar{x})$ be the probability density of the path's sampling decisions had channel c been the hero. It is the product over all medium events of $\sigma_{t,c}e^{-\sigma_{t,c}t}$ (scattering) or $e^{-\sigma_{t,c}t_s}$ (passing to a surface), times the direction densities, which are the same for all channels. With the balance heuristic over the three techniques, the one-sample MIS estimator (4.6) for channel λ of the path contribution f_λ is

$$\frac{f_\lambda(\bar{x})}{\frac{1}{3} \sum_c p_c(\bar{x})}. \quad (20.1)$$

The weights are bounded because the denominator contains the density of the channel that generated the path.

20.3 Incremental implementation

A path is built incrementally, so the code tracks two vectors. The first is $\tilde{\beta} = f/p_h$, the ordinary throughput of the hero channel's sampling. The second is `spectralPdf`, the ratios $r_c = p_c/p_h$, starting at $(1, 1, 1)$; its hero component is always exactly 1. By (20.1) the correctly weighted throughput is $\tilde{\beta}/\bar{r}$, where $\bar{r}$ is the average of the three components. Instead of dividing at every contribution, the code stores $\beta = \tilde{\beta}/\bar{r}$ directly — the throughput every other part of the loop uses — and updates it at each medium event:

$$r \leftarrow r \odot q, \quad \beta \leftarrow \beta \cdot a \cdot \frac{\bar{r}_{\text{old}}}{\bar{r}_{\text{new}}}, \quad (20.2)$$

where a is the event's usual factor f/p_h and $q_c = p_c/p_h$ its density ratio:

Event	a (per channel)	q_c
scatter at t	$T(t) \sigma_s / (\sigma_{t,h} T_h(t))$	$\sigma_{t,c} T_c(t) / (\sigma_{t,h} T_h(t))$
pass to the surface	$T(t_s) / T_h(t_s)$	$T_c(t_s) / T_h(t_s)$

DERIVATION · WHY THE UPDATE IS EXACT

Before the event $\beta = \tilde{\beta}/\bar{r}_{\text{old}}$. The event multiplies $\tilde{\beta}$ by a and r by q . Hence

$$\beta_{\text{new}} = \frac{\tilde{\beta} a}{\bar{r}_{\text{new}}} = \frac{\tilde{\beta}}{\bar{r}_{\text{old}}} \cdot a \cdot \frac{\bar{r}_{\text{old}}}{\bar{r}_{\text{new}}} = \beta \cdot a \cdot \frac{\bar{r}_{\text{old}}}{\bar{r}_{\text{new}}}.$$

Surface scattering and phase sampling have the same density for every channel ($q = 1$), so they only multiply β by their usual weight. In a grey medium $q = (1, 1, 1)$, r never changes, and the update reduces to $\beta \times = \varpi$ (scatter) or 1 (pass) — Section 19.1.

raygen.rgen · SampleMediumInteraction

```
bool SampleMediumInteraction(Medium medium, float tMax, int hero, inout vec3 beta, inout vec3 spectralPdf, inout uint rng,
out float t)
{
    t = tMax;
    vec3 sigmaT = medium.sigmaA + medium.sigmaS;
    if (MaxComponent(medium.sigmaS) <= 0.0)
    {
        beta *= exp(-sigmaT * tMax);
        return false;
    }

    float previousPdf = Average(spectralPdf);
    float distance = sigmaT[hero] > 0.0 ? -log(max(1.0 - Random(rng), 1e-12)) / sigmaT[hero] : INF;
    if (distance < tMax)
    {
        vec3 transmittance = exp(-sigmaT * distance);
        vec3 density = sigmaT * transmittance;
        spectralPdf *= density / density[hero];
        beta *= transmittance * medium.sigmaS / density[hero] * (previousPdf / Average(spectralPdf));
        t = distance;
        return true;
    }

    vec3 transmittance = exp(-sigmaT * tMax);
    spectralPdf *= transmittance / transmittance[hero];
    beta *= transmittance / transmittance[hero] * (previousPdf / Average(spectralPdf));
    return false;
}
```

20.4 Why per-event averaging is not enough

The first version of the renderer applied the balance heuristic separately at every event: it sampled the channel anew each time and divided by the per-event average density, $\beta \times = T\sigma_s/\sigma_t\bar{T}$. That estimator is also unbiased, but its weights multiply over the events of a path. In a strongly coloured, optically thick medium they develop heavy tails: most paths contribute slightly too little and rare paths far too much. In the furnace test (Section 12) this showed up as coloured speckles and a 1% deficit that shrinks only very slowly. The path-level form (20.1), also used by PBRT-v4, removed both.

20.5 Numerical safety

Because the hero component of r is always 1, $\bar{r} \geq 1/3$ and the division in (20.2) is safe. A distance sampled with 24-bit random numbers satisfies $\sigma_{t,h}t \leq \ln 2^{24} \approx 16.6$, so the hero channel's density $\sigma_{t,h}e^{-\sigma_{t,h}t}$ never underflows. Components of r for other channels may become very small, which is harmless, or large; the latter happens only on exponentially rare paths, and the per-sample NaN/Inf guard (Section 25) catches the pathological remainder.

21. Direct lighting in media

At a medium scattering event, next event estimation works as at a surface (Section 13), with the phase function in place of $f|\cos\theta_i|$:

$$\langle L_{\text{direct}} \rangle = \frac{p_{\text{HG}}(\omega_o, \omega_i) L_e T(\mathbf{x}, \mathbf{y})}{p_{\text{light}}} w_{\text{light}}, \quad w_{\text{light}} = \frac{p_{\text{light}}^2}{p_{\text{light}}^2 + p_{\text{HG}}^2}. \quad (21.1)$$

The phase function value doubles as the competing density, because phase sampling is exact.

The transmittance along the shadow ray is computed by `Transmittance`. It traces toward the light point (stopping at 99.99% of the distance, so as not to hit the light itself) and multiplies $e^{-\sigma_t \ell}$ for the segment of length ℓ in the current medium. If it hits a null boundary, it updates a local copy of the medium stack and continues behind the boundary. Any other surface blocks the light completely. This handles smoke boxes, nested boundaries and the global medium analytically, without noise.

CAVEAT · REFRACTIVE BOUNDARIES BLOCK SHADOW RAYS

A dielectric boundary refracts light, so the straight shadow ray from a point inside glass to a light is not a valid light path, and `Transmittance` returns 0. Consequently, points inside the jade dragon or behind the glass sphere receive light only through BSDF-sampled paths that happen to exit through the boundary toward a light. Such paths are rare for small lights, and the resulting noise decays slowly: the jade is the noisiest object in the scene. Specialised techniques such as manifold next event estimation (Hanika et al. 2015) address this.

raygen.rgen · DirectLightingMedium

```
vec3 DirectLightingMedium(vec3 position, vec3 wo, float g, MediumStack stack, inout uint rng)
{
    LightSample light;
    if (!SampleLight(rng, light))
        return vec3(0.0);

    vec3 toLight = light.position - position;
    float distance2 = dot(toLight, toLight);
    vec3 wi = toLight * inversesqrt(distance2);
    float cosLight = dot(light.normal, -wi);
    if (cosLight <= 0.0)
        return vec3(0.0);

    vec3 transmittance = Transmittance(position, light.position, stack);
    if (MaxComponent(transmittance) <= 0.0)
        return vec3(0.0);

    float phase = HenyeyGreenstein(dot(wo, wi), g);
    float lightPdf = light.pdfArea * distance2 / cosLight;
    return phase * light.emission * transmittance * (PowerHeuristic(lightPdf, phase) / lightPdf);
}
```

raygen.rgen · Transmittance

```
vec3 Transmittance(vec3 origin, vec3 target, MediumStack stack)
{
    vec3 transmittance = vec3(1.0);
    for (int i = 0; i < MAX_NULL_CROSSINGS; i++)
    {
        vec3 toTarget = target - origin;
        float distance = length(toTarget);
        vec3 direction = toTarget / distance;
        bool hit = TraceClosest(origin, direction, distance * 0.9999);
        float segment = hit ? payload.t : distance;

        int mediumIndex = CurrentMedium(stack);
        if (mediumIndex >= 0)
            transmittance *= exp(-(media[mediumIndex].sigmaA + media[mediumIndex].sigmaS) * segment);
        if (!hit)
            return transmittance;

        Material material = materials[payload.materialIndex];
        if ((material.flags & MATERIAL_NULL_SURFACE) == 0u || MaxComponent(transmittance) <= 0.0)
            return vec3(0.0);

        CrossBoundary(stack, payload.frontFace != 0u, material.mediumIndex);
        vec3 ng = payload.geometricNormal;
        origin = OffsetRay(payload.position, dot(direction, ng) > 0.0 ? ng : -ng);
    }
    return vec3(0.0);
}
```

22. Medium boundaries and the medium stack

22.1 Attaching media to geometry

A medium is attached to a material, `Material::Medium`, and fills the interior of every closed mesh drawn with that material. "Interior" is defined by the geometric normal: meshes must be closed and consistently wound so that $\mathbf{n}_g$ points outward. A ray *enters* an object when it crosses a front face (`frontFace` = the ray came from the outside) and continues on the inside; it *leaves* when it crosses a back face.

22.2 Two kinds of boundaries

	Null boundary (<code>NullSurface = true</code>)	Dielectric boundary (glass material with a medium)
physical meaning	index-matched interface: the medium simply starts, e.g. smoke in air	refractive interface: the medium sits behind a surface with its own IOR, e.g. coloured glass, jade, wax
at the boundary	no reflection, no refraction, no Fresnel; the surface is invisible	full rough dielectric BSDF (Section 10.7)
medium change	on every crossing	only when the sampled direction transmits through the surface
shadow rays	pass through, accumulating transmittance	blocked
convergence	fast: full NEE inside the medium	slow for small lights (Section 21)

22.3 The medium stack

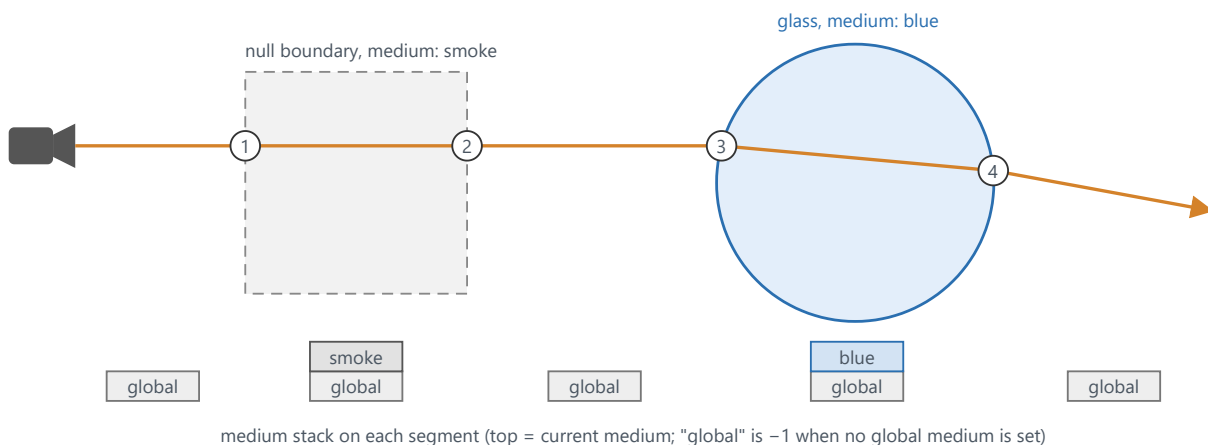


Figure 29. A camera ray passes through a null-bounded smoke box and a glass sphere filled with an absorbing medium. (1) front face of a null boundary: push smoke; (2) back face: pop; (3) refraction into the sphere through a front face: push blue; (4) refraction out through a back face: pop.

Media can be nested — a glass of water inside a foggy room — so the current medium is kept on a small stack. The bottom entry is the global medium, or `-1` for vacuum. Entering an object pushes its medium, leaving pops, and the top entry is the medium the ray currently travels through. An object whose material has no medium pushes `-1`, so the inside of plain glass is vacuum even inside fog. The stack holds four entries: the global medium plus three nested levels.

raygen.rgen · CreateMediumStack

```
MediumStack CreateMediumStack()
{
    MediumStack stack;
    for (int i = 0; i < MEDIUM_STACK_SIZE; i++)
        stack.media[i] = -1;
    stack.media[0] = globalMedium;
    stack.depth = 0;
    return stack;
}
```

raygen.rgen · CurrentMedium

```
int CurrentMedium(MediumStack stack)
{
    return stack.media[stack.depth];
}
```

raygen.rgen · CrossBoundary

```
void CrossBoundary(inout MediumStack stack, bool entering, int medium)
{
    if (entering)
    {
        stack.depth = min(stack.depth + 1, MEDIUM_STACK_SIZE - 1);
        stack.media[stack.depth] = medium;
    }
    else
    {
        stack.depth = max(stack.depth - 1, 0);
    }
}
```

22.4 Rules and limitations

- **Closed, outward-wound meshes.** A mesh with holes lets rays enter without leaving (or vice versa), and the stack gets out of sync. In the showcase assets, `cube.obj`, `sphere.obj` and `dragon.obj` are closed; `bunny.obj` has 42 open boundary edges and cannot hold a medium.
- **The camera starts outside** every object medium; the stack is initialised with the global medium only.
- **Nested, not overlapping.** The stack assumes that boundaries are crossed in last-in, first-out order. Partially overlapping volumes pop the wrong entry.
- **No coplanar boundaries.** Two surfaces at exactly the same place produce ambiguous intersections. The smoke box therefore floats 2 mm above the floor.
- **Limited depth.** Deeper nesting than three levels is clamped.

22.5 The global medium

`Scene::SetGlobalMedium` places a medium at the bottom of the stack, immersing the whole scene — the classic way to add atmospheric fog. The global medium is unbounded, so a ray that escapes the scene travels an infinite distance through it and never reaches the sky (Section 19.3). In a scene lit mainly by the environment, such as the open showcase room, this removes the sky light entirely. For local fog, use a large null-bounded box instead:

```
const MediumId fog = scene.AddMedium({ .Absorption = glm::vec3(0.01f), .Scattering = glm::vec3(0.3f), .Anisotropy = 0.6f });
scene.AddInstance(cube, { .Position = roomCenter, .Scale = roomSize }, scene.AddMaterial({ .Medium = fog, .NullSurface = true }));
```

23. Worked examples



smoke volume with an emitter inside



absorbing blue glass and its caustic



jade: scattering inside rough glass

Figure 30. The three media of the showcase scene (enlarged crops of the title image, 60 s of accumulation). The jade is visibly noisier: its light arrives only through BSDF-sampled exits (Section 21).

Object	Parameters	What they mean
smoke box	$\sigma_s = 5, \sigma_a = 0.2, g = 0.4$; null boundary, 0.32 units wide	$\sigma_t = 5.2$, mean free path 0.19, albedo 0.96, optical thickness across the box 1.7 (19% of the light passes straight through). Forward scattering lets the orange emitter light the smoke in a soft halo. NEE works through the null boundary, so it converges fast.
blue glass sphere	$\sigma_a = (3, 1.2, 0.3), \sigma_s = 0$; smooth glass, IOR 1.5, diameter 0.32	pure absorber, evaluated analytically (no noise). Transmittance through the centre is (0.38, 0.68, 0.91); the thin rim is nearly clear. The refracted light focuses into the caustic on the floor, found by BSDF sampling from the floor.
jade dragon	$\sigma_s = 40, \sigma_a = (4, 0.4, 3), g = 0.3$; rough glass, IOR 1.62, $r = 0.25$	$\sigma_t \approx (44, 40, 43)$, mean free path 0.024, while the body is about 0.1 thick: light scatters dozens of times (subsurface scattering). The albedos (0.91, 0.99, 0.93) look almost white, but after n events each channel is scaled by ϖ^n — after 20 events (0.15, 0.82, 0.23) — so deep light turns green while thin parts stay pale. That is jade's translucency.

CHOOSING MEDIUM COEFFICIENTS

- **Density:** pick the mean free path relative to the object size. $\sigma_t \cdot \text{size} \ll 1$ is a faint haze, ≈ 1 a visible volume, $\gg 1$ looks solid, like milk or wax.
- **Colour:** set it with a small per-channel difference in albedo, not a large one. Multiple scattering raises the albedo to the power of the number of bounces, so in dense media a colour difference of a few percent is already strongly saturated.
- **Direction:** g controls the look against the light. Forward scattering ($g > 0$) gives bright silver linings and halos around lights; backward scattering makes the medium look denser when lit from the front.
- **Absorption:** for coloured glass or liquids, use $\sigma_s = 0$ and choose $\sigma_a = -\ln(T_{\text{desired}})/d$ for a transmittance T at thickness d .

PART VI

Output and practice

The path loop produces radiance, but a monitor displays 8-bit values. This part covers the last steps from the accumulated radiance to a pixel, collects the numerical safeguards that keep a long-running progressive render clean, and lists the limitations of the current implementation together with the natural next steps.

24. From radiance to pixels

24.1 Exposure and tone mapping

The displayed value of a pixel starts from the running mean of Section 5.4 multiplied by the exposure, $x = \text{exposure} \cdot S_N/N$. Radiance in the showcase scene spans several orders of magnitude: the ceiling panel emits 14, the sky reaches 1–2, and shadowed corners receive a few hundredths. Clamping to $[0, 1]$ would burn every highlight into flat white and shift the hue of bright colours abruptly. A tone curve compresses high values smoothly instead.

The renderer uses Stephen Hill's fit of the ACES reference rendering and output transforms (RRT + ODT), as published in Matt Pettineo's BakingLab. The colour is first converted with a matrix into a working space close to ACEScg, including the RRT's saturation adjustment. A rational function then approximates the filmic curve,

$$c(x) = \frac{x(x + 0.0245786) - 0.000090537}{x(0.983729x + 0.4329510) + 0.238081}, \quad (24.1)$$

after which a second matrix converts back to linear sRGB and the result is clamped to $[0, 1]$. Both matrices have unit row sums, so grey stays grey and (24.1) is the complete tone curve for grey values. The curve has a gentle toe, a mid-tone contrast boost and a long shoulder that approaches white asymptotically. Very bright saturated colours are also desaturated toward white — why the cyan emitter appears almost white at its centre — which is a deliberate trait of the ACES look.

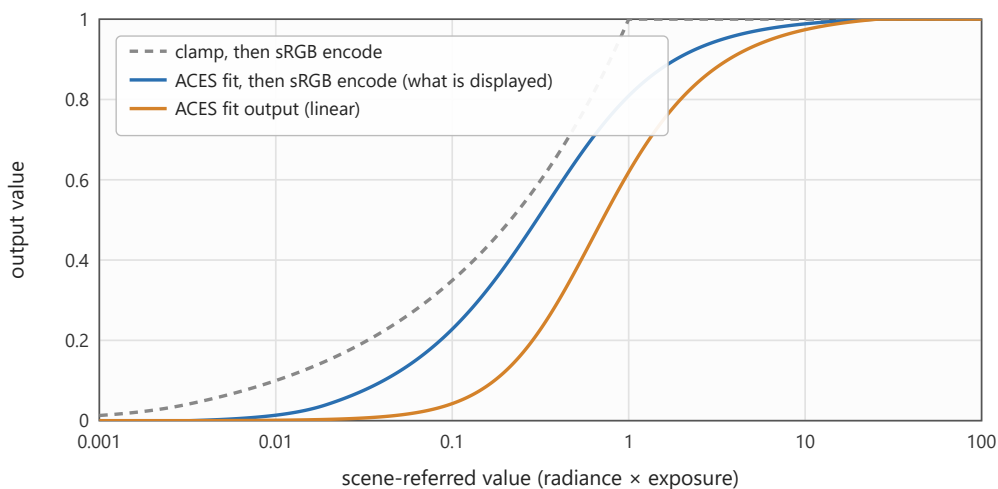


Figure 31. Displayed value as a function of scene value, for plain clamping and for the ACES fit, both followed by the sRGB encoding. The fit compresses the range from 1 to about 20 into the top of the display range instead of clipping it.

raygen.rgen · TonemapAces

```
vec3 TonemapAces(vec3 color)
{
    const mat3 inputMatrix = mat3(0.59719, 0.07600, 0.02840, 0.35458, 0.90834, 0.13383, 0.04823, 0.01566, 0.83777);
    const mat3 outputMatrix = mat3(1.60475, -0.10208, -0.00327, -0.53108, 1.10813, -0.07276, -0.07367, -0.00605, 1.07602);
    color = inputMatrix * color;
    vec3 a = color * (color + 0.0245786) - 0.000090537;
    vec3 b = color * (0.983729 * color + 0.4329510) + 0.238081;
    return clamp(outputMatrix * (a / b), 0.0, 1.0);
}
```

24.2 sRGB encoding

Displays expect values encoded with the sRGB transfer function (IEC 61966-2-1), which allocates more code values to dark tones, matching the eye's sensitivity:

$$V = \begin{cases} 12.92 x & x \leq 0.0031308 \\ 1.055 x^{1/2.4} - 0.055 & \text{otherwise.} \end{cases} \quad (24.2)$$

Storage images cannot use sRGB formats, so the shader applies (24.2) itself and writes the result to the `RGBA8` output image. The swapchain uses the UNORM format `B8G8R8A8_UNORM`, so the composite pass forwards the already encoded values unchanged, and screenshots taken with the G key are correctly encoded PNGs.

raygen.rgen · LinearToSrgb

```
vec3 LinearToSrgb(vec3 color)
{
    return mix(color * 12.92, 1.055 * pow(color, vec3(1.0 / 2.4)) - 0.055, step(vec3(0.0031308), color));
}
```

24.3 Linear lighting and sRGB textures

The rendering equation is linear in radiance and in reflectance, so all lighting must happen on linear values. Colour textures are stored with sRGB encoding (8 bits per channel spend their precision on dark tones) and are created with the `R8G8B8A8_SRGB` format. The texture unit decodes them to linear values before filtering, so `BaseColor` and texture colours enter the BSDF linearly. Material colours specified in code are linear values as well: the showcase floor's checker texture stores 0x40 and 0xD0, which decode to linear reflectances of 0.05 and 0.63.

25. Numerical robustness

A progressive renderer accumulates millions of samples into a running sum. A single NaN or infinity would poison a pixel permanently, and rare numerical corner cases occur sooner or later. The implementation guards against them at the following places:

Guard	Purpose
<code>MIN_ALPHA = 10⁻³</code>	keeps GGX finite; mirrors are very sharp GGX lobes instead of delta distributions
<code>max(w.z , 1e-8), cos²θ ≥ 10⁻¹² in Δ</code>	no division by zero for grazing directions
sample rejected if pdf ≤ 10 ⁻¹² or $f = 0$	no huge weights from vanishing densities
<code>max(dot(ng, -direction), 1e-8)</code> in the emission MIS weight	emitters hit at grazing angles
24-bit random numbers, <code>max(1 - ξ, 1e-12)</code>	$\ln(1 - \xi)$ stays finite in free-flight sampling
<code>INF = 10³⁰</code> (finite)	$0 \cdot \text{INF} = 0$ instead of NaN in transmittance computations
IOR clamped to ≥ 1.01	the refraction half vector cannot degenerate
g clamped to ± 0.99 ; isotropic fallback for $ g < 10^{-3}$	the HG inversion is well conditioned
ULP-based ray offsets, <code>tMin = 0</code> (Section 6.5)	no self-intersection at any scene scale
<code>MAX_NULL_CROSSINGS = 64</code> , stack depth clamp	no endless loops at overlapping or broken null boundaries
last light-CDF entry forced to 1	the binary search always finds a triangle
lobe probabilities renormalised	robust to parameters outside $[0, 1]$
per-sample NaN/Inf guard in <code>main</code>	a pathological sample is replaced by zero instead of corrupting the running sum; the bias is negligible because such samples are extremely rare

26. Limitations and extensions

26.1 Light transport

- **Environment importance sampling.** The sky is only found by BSDF sampling. Sampling it in proportion to its radiance (a 2D distribution over a latitude–longitude map) and combining that with MIS would make sun-lit and window-lit scenes converge much faster.
- **Many lights.** Power-proportional selection ignores distance and orientation. A light BVH (Conty Estevez & Kulla 2018) or reservoir resampling (ReSTIR; Bitterli et al. 2020) picks lights that matter at the shading point.
- **Light through refractive boundaries.** Caustics and the interiors of glass-bounded media rely on BSDF sampling. Manifold next event estimation (Hanika et al. 2015), bidirectional methods or path guiding address this class of paths.
- **Sample quality.** Low-discrepancy sequences — Sobol with Owen scrambling (Burley 2020) or blue-noise dithered sequences — reduce the error of the low-dimensional parts of the integrand compared to independent random numbers.
- **Denoising.** A denoiser (for example Intel Open Image Denoise, or a spatio-temporal filter for interactive use) would give clean previews after few samples.

26.2 Materials

- Energy compensation for rough dielectrics and the clear coat (tables over μ , α and η ; Kulla & Conty 2017).
- A reciprocal layered model with inter-reflections between coat and base (stochastic layering; Guo et al. 2018).
- Anisotropic roughness (α_x, α_y) — GGX, Smith masking and visible-normal sampling generalise directly.
- Conductor Fresnel with edge tint or complex IOR (Gulbrandsen 2014), thin-film interference, textures for roughness, metallic and normals, and texture filtering for secondary rays (ray cones; Akenine-Möller et al. 2021).

26.3 Media

- Volumetric emission (flames, glowing gas) — one more term in (17.1).
- More realistic phase functions: a sum of two HG lobes for clouds, or Mie scattering.
- A camera inside a medium: initialise the stack by finding the media that contain the camera.
- Overlapping volumes and nested dielectrics with priorities (Schmidt & Budge 2002).

26.4 Towards heterogeneous media

In a heterogeneous medium, $\sigma_t(\mathbf{x})$ varies in space, the transmittance integral (17.3) has no closed form, and inverting it for free-flight sampling is no longer possible. The standard solution adds fictitious "null" particles that do nothing but bring the extinction up to a constant *majorant* $\bar{\sigma} \geq \sigma_t(\mathbf{x})$ along the ray:

- **Delta (Woodcock) tracking** samples tentative collisions with the constant density $\bar{\sigma}e^{-\bar{\sigma}t}$ — exactly the homogeneous code of Section 19. At each tentative collision at $\mathbf{x}$ it accepts a real collision with probability $\sigma_t(\mathbf{x})/\bar{\sigma}$ and otherwise continues. The accepted distances follow the correct heterogeneous free-flight distribution without ever evaluating the transmittance integral.
- **Ratio tracking** estimates shadow-ray transmittance as $\prod_k (1 - \sigma_t(\mathbf{x}_k)/\bar{\sigma})$ over the tentative collisions (Novák et al. 2014).
- The null-scattering path integral (Miller et al. 2019) and PBRT-v4 put these into one framework, including the spectral MIS of Section 20. The ratio vector r gains a factor per null collision, and light sampling gets its own ratio vector.

The homogeneous renderer is the special case $\bar{\sigma} = \sigma_t$, in which every tentative collision is real. The density itself need not be a dense 3D texture:

- a procedural function (noise, analytic shapes) evaluated at each tentative collision costs no memory at all;
- sparse voxel structures such as NanoVDB (Museth 2021) store only occupied regions;
- a coarse grid of per-cell majorants gives tight $\bar{\sigma}$ bounds, and therefore few wasted null collisions, even when the density comes from a function.

APPENDICES

Reference material

A compact summary of every formula the renderer implements, the parameters and constants, a map from shader functions to sections of this document, the longer derivations, and the literature.

A. Formula summary

Light transport

$$L_o(\mathbf{x}, \omega_o) = L_e + \int_{\mathcal{S}^2} f L_i |\cos \theta_i| d\omega_i, \quad d\omega_i = \frac{|\cos \theta_y|}{\|\mathbf{x} - \mathbf{y}\|^2} dA_y, \quad p_\omega = p_A \frac{\|\mathbf{x} - \mathbf{y}\|^2}{|\cos \theta_y|}$$

$$\langle I \rangle = \frac{1}{N} \sum \frac{f(X_k)}{p(X_k)}, \quad \beta = \prod \frac{f|\cos \theta_i|}{p}, \quad w_a = \frac{p_a^2}{p_a^2 + p_b^2}, \quad q_{RR} = \min(\max_c \beta_c, 0.95)$$

Fresnel

$$r_{\parallel} = \frac{\eta \cos \theta_1 - \cos \theta_2}{\eta \cos \theta_1 + \cos \theta_2}, \quad r_{\perp} = \frac{\cos \theta_1 - \eta \cos \theta_2}{\cos \theta_1 + \eta \cos \theta_2}, \quad F = \frac{1}{2}(r_{\parallel}^2 + r_{\perp}^2), \quad F_0 = \left(\frac{\eta - 1}{\eta + 1}\right)^2$$

$$F_{\text{Schlick}} = F_0 + (1 - F_0)(1 - \cos \theta)^5, \quad F_{dr} = 2 \int_0^1 F \mu d\mu, \quad 1 - F_{dr}^{\text{int}} = \frac{1 - F_{dr}^{\text{ext}}}{\eta^2}$$

GGX microfacets

$$D = \frac{\alpha^2}{\pi((\mathbf{m} \cdot \mathbf{n})^2(\alpha^2 - 1) + 1)^2}, \quad \Lambda = \frac{-1 + \sqrt{1 + \alpha^2 \tan^2 \theta}}{2}, \quad G_1 = \frac{1}{1 + \Lambda}, \quad G_2 = \frac{1}{1 + \Lambda(\omega_o) + \Lambda(\omega_i)}$$

$$f_r = \frac{F D G_2}{4|\cos \theta_o||\cos \theta_i|}, \quad f_t = \frac{|\omega_i \cdot \mathbf{h}||\omega_o \cdot \mathbf{h}|}{|\cos \theta_i||\cos \theta_o|} \frac{(1 - F) D G_2}{(\omega_i \cdot \mathbf{h} + \omega_o \cdot \mathbf{h}/\eta')^2} \frac{1}{\eta^2}$$

$$D_{\omega_o}(\mathbf{m}) = \frac{G_1(\omega_o) \max(0, \omega_o \cdot \mathbf{m}) D(\mathbf{m})}{\cos \theta_o}, \quad p_r = \frac{D_{\omega_o}(\mathbf{h})}{4|\omega_o \cdot \mathbf{h}|}, \quad p_t = D_{\omega_o}(\mathbf{h}) \frac{|\omega_i \cdot \mathbf{h}|}{(\omega_i \cdot \mathbf{h} + \omega_o \cdot \mathbf{h}/\eta')^2}, \quad \frac{f_r \cos \theta_i}{p_r} = F \frac{G_2}{G_1}$$

Material

$$b = 1 - cF(\mu_o; 1.5), \quad w_{\text{metal}} = bm, \quad w_{\text{glass}} = b(1 - m)t, \quad w_{\text{opaque}} = b(1 - m)(1 - t)$$

$$k_{\text{ms}} = 1 + \rho \left(\frac{1}{E(\mu_o, \alpha)} - 1 \right), \quad f_d = \frac{\rho(1 - F(\mu_o))(1 - F(\mu_i))}{\pi(1 - F_{dr}(\eta))}, \quad f_{\text{sheen}} = C_s(1 - \omega_i \cdot \mathbf{h})^5$$

$$p_{\text{coat}} : p_{\text{metal}} : p_{\text{glass}} : p_{\text{spec}} : p_{\text{diff}} = cF_c : w_{\text{metal}} : w_{\text{glass}} : w_{\text{opaque}} F : w_{\text{opaque}}(1 - F)$$

Sampling

$$\text{disk: } r = \sqrt{\xi_1}, \phi = 2\pi\xi_2; \quad \text{cosine hemisphere: } p = \frac{\cos \theta}{\pi}; \quad \text{triangle: fold } (u, v), p_A = \frac{1}{A}; \quad p_A^{\text{light}} = \frac{Y(L_e)}{\sum_j Y(L_{e,j}) A_j}$$

Media

$$L_i = \int_0^{t_s} T(t) \sigma_s L_s dt + T(t_s) L_o, \quad T(t) = e^{-\sigma_t t}, \quad t = -\frac{\ln(1 - \xi)}{\sigma_t}, \quad \text{scatter: } \beta \times = \frac{\sigma_s}{\sigma_t}$$

$$p_{\text{HG}} = \frac{1}{4\pi} \frac{1 - g^2}{(1 + g^2 + 2g \omega_o \cdot \omega_i)^{3/2}}, \quad s = \frac{1 - g^2}{1 + g - 2g\xi}, \quad \cos(\omega_o, \omega_i) = -\frac{1 + g^2 - s^2}{2g}$$

$$\text{spectral MIS: } r \leftarrow r \odot q, \quad \beta \leftarrow \beta a \bar{r}_{\text{old}} / \bar{r}_{\text{new}}, \quad \text{estimator } f / \frac{1}{3} \sum_c p_c$$

Output

$$c(x) = \frac{x(x + 0.0245786) - 0.000090537}{x(0.983729x + 0.4329510) + 0.238081}, \quad V = \begin{cases} 12.92x & x \leq 0.0031308 \\ 1.055x^{1/2.4} - 0.055 & \text{else} \end{cases}$$

B. Parameter reference

Material field	Default	Notes
BaseColor	(1, 1, 1)	linear; multiplied by Texture
Emission	(0, 0, 0)	radiance; front face only; triangles with $Y > 0$ become lights
Metallic	0	
Roughness	0.5	$\alpha = \max(r^2, 10^{-3})$
Transmission	0	
IOR	1.5	clamped to ≥ 1.01 in the shader
Clearcoat	0	coat IOR fixed at 1.5 (COAT_IOR)
ClearcoatRoughness	0.03	
Sheen	(0, 0, 0)	additive
Texture	none $\rightarrow$ white	RGBA8 sRGB, loaded with LoadTexture or AddTexture
Medium	none	interior medium of a closed mesh
NullSurface	false	invisible boundary; emission ignored
Medium field	Default	Notes
Absorption σ_a	(0, 0, 0)	per scene unit; negative values clamped to 0
Scattering σ_s	(0, 0, 0)	per scene unit; all zero means a pure absorber (analytic)
Anisotropy g	0	Henyey–Greenstein; clamped to $[-0.99, 0.99]$
Frame parameter	Default	Notes
MaxBounces	64	surface and medium scattering events; null crossings do not count
Exposure	1	multiplier before tone mapping
SamplesPerPixel	1	paths per pixel and frame
Aperture, FocusDistance	0, 1	thin lens radius and focus distance along the view axis; 0 = pinhole
Shader constant	Value and meaning	
MIN_ALPHA	10^{-3} , smallest GGX width	
NEE_MIN_ALPHA	0.0025, below this (without diffuse or rough coat) NEE is skipped	
COAT_IOR	1.5	
GGX_ALBEDO	8×8 table of $E(\mu, \alpha)$ over $(\sqrt{\mu}, r)$	
MEDIUM_STACK_SIZE	4 (global medium + 3 nested levels)	
MAX_NULL_CROSSINGS	64 per path and per shadow ray	
INF	10^{30}	
Russian roulette	from depth 3, survival $\min(\max_c \beta_c, 0.95)$	
OffsetRay	256 ULPs along $\mathbf{n}_g$; absolute 2^{-16} for coordinates below $1/32$	

C. Code map

Function	File	Section
PcgHash, Random, Random2	common.glsl	5.5 Random numbers
BuildFrame, ToLocal, ToWorld	common.glsl	7.1 Orthonormal basis
SampleUniformDisk, SampleCosineHemisphere	common.glsl	11.2 Sampling each lobe
SampleUniformTriangle	common.glsl	13.2 Choosing a point on a light
PowerHeuristic	common.glsl	14.2 Power heuristic
OffsetRay	common.glsl	6.5 Ray offsets
EnvironmentRadiance	scene.glsl	16.4 The environment
FresnelDielectric, FresnelSchlick, FresnelDiffuseReflectance	bsdf.glsl	8 Fresnel reflectance
GgxD, GgxLambda, GgxG1, GgxG2	bsdf.glsl	9.2–9.3 GGX, Smith
GgxVisiblePdf, SampleGgxVisibleNormal	bsdf.glsl	9.6–9.8 Visible normals
Refract	bsdf.glsl	7.3 Reflection and refraction
EvalDielectric, SampleDielectric	bsdf.glsl	9.5, 9.8 Rough dielectric
GgxAlbedo, GGX_ALBEDO	bsdf.glsl	10.4 Energy compensation
CreateBsdf, EvalBsdf, SampleBsdf, BsdfHasSmoothLobes	bsdf.glsl	10–11 Material model and sampling
main	rayclosesthit.rchit	6.2 Reconstructing the hit
main	raygen.rgen	5.6 Entry point
TraceClosest, TracePath	raygen.rgen	16 The complete path loop
FetchMaterial	raygen.rgen	10.1 Parameters
SampleLight, LightPdfArea	raygen.rgen	13.2 Light sampling
DirectLightingSurface	raygen.rgen	13.3 The NEE estimator
DirectLightingMedium, Transmittance	raygen.rgen	21 Direct lighting in media
HenryGreenstein, SampleHenryGreenstein	raygen.rgen	18 Phase functions
SampleMediumInteraction	raygen.rgen	19–20 Free-flight sampling, spectral MIS
CreateMediumStack, CurrentMedium, CrossBoundary	raygen.rgen	22.3 The medium stack
RussianRoulette	raygen.rgen	15 Russian roulette
TonemapAces, LinearToSrgb	raygen.rgen	24 From radiance to pixels
Scene::Build	Scene.cpp	6.1 Instancing, 13.1 Light list

D. Derivations

D.1 The half-vector Jacobian for reflection

Fix ω_i and place the polar axis of a spherical coordinate system along it. A half vector $\mathbf{h}$ at polar angle θ_h and azimuth ϕ reflects ω_i into the direction $\omega_o = 2(\omega_i \cdot \mathbf{h})\mathbf{h} - \omega_i$, which lies in the same plane at twice the angle: polar angle $2\theta_h$, azimuth ϕ . Hence

$$d\omega_o = \sin(2\theta_h) d(2\theta_h) d\phi = 4 \sin \theta_h \cos \theta_h d\theta_h d\phi = 4 \cos \theta_h d\omega_h,$$

with $\cos \theta_h = \omega_i \cdot \mathbf{h} = \omega_o \cdot \mathbf{h}$. The roles of the two directions are symmetric, so

$$\frac{d\omega_h}{d\omega_i} = \frac{1}{4|\omega_o \cdot \mathbf{h}|}.$$

D.2 A general half-vector Jacobian, and refraction

Let $\mathbf{h} = \mathbf{H}/\|\mathbf{H}\|$ with $\mathbf{H} = a\omega_i + b\omega_o$ for fixed ω_o and constants a, b . As ω_i sweeps a small solid angle $d\omega_i$, the point $\mathbf{H}$ sweeps an area $a^2 d\omega_i$ on the sphere of radius $|a|$ around $b\omega_o$. The normal of that sphere at $\mathbf{H}$ is ω_i . Seen from the origin at distance $\|\mathbf{H}\|$, this patch subtends the solid angle

$$d\omega_h = \frac{a^2 |\omega_i \cdot \mathbf{h}|}{\|\mathbf{H}\|^2} d\omega_i.$$

Reflection ($a = b = 1$): $\|\omega_i + \omega_o\| = 2|\omega_i \cdot \mathbf{h}|$, which recovers D.1. **Refraction** ($a = \eta_i$, $b = \eta_o$; the sign of $\mathbf{h}_t$ does not matter): because $\mathbf{H}$ is parallel to $\mathbf{h}$, $\|\mathbf{H}\| = |\mathbf{H} \cdot \mathbf{h}| = |\eta_i \omega_i \cdot \mathbf{h} + \eta_o \omega_o \cdot \mathbf{h}|$, and

$$\frac{d\omega_h}{d\omega_i} = \frac{\eta_i^2 |\omega_i \cdot \mathbf{h}|}{(\eta_i \omega_i \cdot \mathbf{h} + \eta_o \omega_o \cdot \mathbf{h})^2} = \frac{|\omega_i \cdot \mathbf{h}|}{(\omega_i \cdot \mathbf{h} + \omega_o \cdot \mathbf{h}/\eta')^2}, \quad \eta' = \frac{\eta_i}{\eta_o}.$$

This is the factor in the transmission pdf (9.14) and in `EvalDielectric`. Repeating the flux argument of Section 9.4 with this Jacobian, the transmitted fraction $1 - F$, and the radiance change $1/\eta'^2$ across the interface gives the BTDF (9.10).

D.3 Free-flight sampling is unbiased

For a grey homogeneous medium, the free-flight estimator returns $\varpi L_s(\mathbf{x}_t)$ if $t < t_s$ and L_o otherwise. Its expectation over $t \sim \sigma_t e^{-\sigma_t t}$ is

$$\int_0^{t_s} \sigma_t e^{-\sigma_t t} \frac{\sigma_s}{\sigma_t} L_s(\mathbf{x}_t) dt + \Pr[t \geq t_s] L_o = \int_0^{t_s} e^{-\sigma_t t} \sigma_s L_s(\mathbf{x}_t) dt + e^{-\sigma_t t_s} L_o,$$

which is exactly the volume rendering equation (17.2). With chromatic coefficients the same holds for each channel of the MIS-weighted estimator (20.1), since the balance heuristic weights of the three channel techniques sum to one for every path.

E. References

1. T. Akenine-Möller, C. Crassin, J. Boksansky, L. Belcour, A. Panteleev, O. Wright. Improved Shader and Texture Level of Detail Using Ray Cones. *Journal of Computer Graphics Techniques* 10(1), 2021.
2. B. Bitterli, C. Wyman, M. Pharr, P. Shirley, A. Lefohn, W. Jarosz. Spatiotemporal Reservoir Resampling for Real-Time Ray Tracing with Dynamic Direct Lighting. *ACM Transactions on Graphics* 39(4), 2020.
3. B. Burley. Physically-Based Shading at Disney. SIGGRAPH Course: Practical Physically Based Shading in Film and Game Production, 2012.
4. B. Burley. Practical Hash-based Owen Scrambling. *Journal of Computer Graphics Techniques* 9(4), 2020.
5. S. Chandrasekhar. *Radiative Transfer*. Dover, 1960.
6. A. Conty Estevez, C. Kulla. Importance Sampling of Many Lights with Adaptive Tree Splitting. *Proc. ACM on Computer Graphics and Interactive Techniques* 1(2), 2018.
7. E. d'Eon, G. Irving. A Quantized-Diffusion Model for Rendering Translucent Materials. *ACM Transactions on Graphics* 30(4), 2011.
8. T. Duff, J. Burgess, P. Christensen, C. Hery, A. Kensler, M. Liani, R. Villemain. Building an Orthonormal Basis, Revisited. *Journal of Computer Graphics Techniques* 6(1), 2017.
9. O. Gulbrandsen. Artist Friendly Metallic Fresnel. *Journal of Computer Graphics Techniques* 3(4), 2014.
10. Y. Guo, M. Hašan, S. Zhao. Position-Free Monte Carlo Simulation for Arbitrary Layered BSDFs. *ACM Transactions on Graphics* 37(6), 2018.
11. J. Hanika, M. Droske, L. Fascione. Manifold Next Event Estimation. *Computer Graphics Forum* 34(4), 2015.
12. E. Heitz. Understanding the Masking-Shadowing Function in Microfacet-Based BRDFs. *Journal of Computer Graphics Techniques* 3(2), 2014.
13. E. Heitz. Sampling the GGX Distribution of Visible Normals. *Journal of Computer Graphics Techniques* 7(4), 2018.
14. E. Heitz, E. d'Eon. Importance Sampling Microfacet-Based BSDFs using the Distribution of Visible Normals. *Computer Graphics Forum* 33(4), 2014.
15. L. G. Henyey, J. L. Greenstein. Diffuse Radiation in the Galaxy. *Astrophysical Journal* 93, 1941.
16. S. Hill. ACES filmic tone mapping fit ("ACESFitted"), published in M. Pettineo's BakingLab, 2016.
17. IEC 61966-2-1:1999. Multimedia systems and equipment — Colour measurement and management — Default RGB colour space — sRGB.
18. W. Jakob. Mitsuba renderer (plastic BSDF, diffuse Fresnel reflectance), 2010.
19. M. Jarzynski, M. Olano. Hash Functions for GPU Rendering. *Journal of Computer Graphics Techniques* 9(3), 2020.
20. J. T. Kajiya. The Rendering Equation. *Computer Graphics (Proc. SIGGRAPH)* 20(4), 1986.
21. C. Kulla, A. Conty. Revisiting Physically Based Shading at Imageworks. SIGGRAPH Course: Physically Based Shading in Theory and Practice, 2017.

22. P. Kutz, R. Habel, Y. K. Li, J. Novák. Spectral and Decomposition Tracking for Rendering Heterogeneous Volumes. *ACM Transactions on Graphics* 36(4), 2017.
23. B. Miller, I. Georgiev, W. Jarosz. A Null-Scattering Path Integral Formulation of Light Transport. *ACM Transactions on Graphics* 38(4), 2019.
24. K. Museth. NanoVDB: A GPU-Friendly and Portable VDB Data Structure for Real-Time Rendering and Simulation. SIGGRAPH Talks, 2021.
25. J. Novák, I. Georgiev, J. Hanika, W. Jarosz. Monte Carlo Methods for Volumetric Light Transport Simulation. *Computer Graphics Forum* 37(2), 2018.
26. J. Novák, A. Selle, W. Jarosz. Residual Ratio Tracking for Estimating Attenuation in Participating Media. *ACM Transactions on Graphics* 33(6), 2014.
27. M. Pharr, W. Jakob, G. Humphreys. *Physically Based Rendering: From Theory to Implementation*, 4th edition. MIT Press, 2023. (pbr-book.org)
28. C. Schlick. An Inexpensive BRDF Model for Physically-based Rendering. *Computer Graphics Forum* 13(3), 1994.
29. C. M. Schmidt, B. Budge. Simple Nested Dielectrics in Ray Traced Images. *Journal of Graphics Tools* 7(2), 2002.
30. K. E. Torrance, E. M. Sparrow. Theory for Off-Specular Reflection from Roughened Surfaces. *Journal of the Optical Society of America* 57(9), 1967.
31. T. S. Trowbridge, K. P. Reitz. Average Irregularity Representation of a Rough Surface for Ray Reflection. *Journal of the Optical Society of America* 65(5), 1975.
32. E. Turquin. Practical Multiple Scattering Compensation for Microfacet Models. Technical report, Industrial Light & Magic, 2019.
33. E. Veach. *Robust Monte Carlo Methods for Light Transport Simulation*. PhD thesis, Stanford University, 1997.
34. E. Veach, L. J. Guibas. Optimally Combining Sampling Techniques for Monte Carlo Rendering. *Proc. SIGGRAPH*, 1995.
35. C. Wächter, N. Binder. A Fast and Robust Method for Avoiding Self-Intersection. In *Ray Tracing Gems*, chapter 6. Apress, 2019.
36. B. Walter, S. R. Marschner, H. Li, K. E. Torrance. Microfacet Models for Refraction through Rough Surfaces. *Eurographics Symposium on Rendering*, 2007.
37. A. Weidlich, A. Wilkie. Arbitrarily Layered Micro-Facet Surfaces. *Proc. GRAPHITE*, 2007.
38. A. Wilkie, S. Nawaz, M. Droske, A. Weidlich, J. Hanika. Hero Wavelength Spectral Sampling. *Computer Graphics Forum* 33(4), 2014.